

Programski jezici i paradigme

Zbirka zadataka

IVAN RISTOVIĆ
ivan.ristovic@math.rs

MILAN ČUGUROVIĆ
milan.cugurovic@math.rs

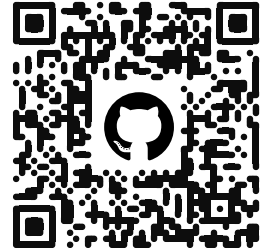
MILENA VUJOŠEVIĆ JANIČIĆ
milena.vujosevic.janicic@math.rs

Sadržaj

1	Programiranje ograničenja	1
1.1	Kombinatorni problemi	2
1.2	Kriptoaritmetike	6
1.3	Matematičke slagalice	13
1.4	Optimizacioni problemi	17
1.5	Ispitni zadaci	20
2	Komponentno programiranje	35
2.1	Osnovni elementi korisničkog interfejsa	36
2.2	Interaktivne komponente	43
2.3	Ispitni zadaci	60
3	Funkcionalno programiranje	77
3.1	Elementarni koncepti funkcionalne paradigme	79
3.2	Liste i funkcije višeg reda	83
3.3	Algebarski tipovi, klase i svojstva	95
3.4	Ispitni zadaci	107

Glava 1

Programiranje ograničenja



Uvod

Programiranje ograničenja (eng. *constraint programming*) je deklarativna programska paradigma zasnovana na definisanju uslova ili ograničenja koja problem mora da ispuni, umesto da se eksplicitno navode koraci za njegovo rešavanje. Programer definiše skup pravila ili zahteva, a sistem automatski pronalazi rešenja koja zadovoljavaju ta ograničenja. Ovaj pristup omogućava da se fokus prebaci sa načina rešavanja na opis samog problema, čime se često postiže veća preglednost i fleksibilnost koda. Posebno je pogodan za situacije u kojima postoji veliki broj mogućih rešenja, ali i složeni odnosi između promenljivih koje treba zadovoljiti.

Programiranje ograničenja je naročito korisno za rešavanje kombinatornih i optimizacionih problema, i situacije u kojima treba pronaći rešenja sa složenim uslovima, kao što su raspoređivanje resursa, planiranje ili rešavanje zagonetki. Razvoj programiranja ograničenja počeo je tokom 1980-ih godina, kao prirodni nastavak i dopuna logičkog programiranja, sa ciljem da se pojednostavi rešavanje komplikovanih problema sa mnogo uslova i promenljivih. Danas je programiranje ograničenja podržano u različitim programskim jezicima, među kojima su *Prolog* i *Python*, kao i kroz specijalizovane biblioteke (npr. *Google OR-Tools*) i alate (npr. *Microsoft Z3*) koji dodatno unapređuju efikasnost i primenu ove paradigme u softverskim sistemima.

1.1 Kombinatorni problemi

Zadatak 1.1.1

Napisati program koji na standardni izlaz ispisuje sve kombinacije oblika $x y z$, gde je $x \in \{a, b, c\}$, $y \in \{1, 2, 3\}$ i $z \in \{0.1, 0.2, 0.3\}$ tako da važi uslov $10 \cdot z = y$.

Primer 1

IZLAZ: a 3 0.3

Primer 2

IZLAZ: b 2 0.2

Primer 3

IZLAZ: c 1 0.1

Zadatak 1.1.2

Napisati program koji pronalazi trocifren broj ABC (gde su A, B i C cifre tog broja) tako da je količnik $ABC / (A + B + C)$ minimalan. Pri tome, cifre A, B i C moraju biti međusobno različite. Na standardni izlaz ispisati pronađeni trocifren broj ABC .

Primer 1

IZLAZ: 189

Zadatak 1.1.3

Napisati program koji pronalazi sve vrednosti promenljivih X, Y i Z za koje važi:

$$X \geq Z \quad \text{i} \quad X * 2 + Y * X + Z \leq 34$$

pri čemu promenljive pripadaju sledećim domenima:

- $X \in \{1, 2, \dots, 90\}$
- $Y \in \{2, 4, 6, \dots, 60\}$ (parni brojevi)
- $Z \in \{1, 4, 9, 16, \dots, 100\}$ (kvadrati prirodnih brojeva)

Sve rezultate ispisati na standardni izlaz u formatu:

$X = \text{broj}, Y = \text{broj}, Z = \text{broj}.$

Primer 1

IZLAZ: X = 7, Y = 2, Z = 4

Primer 2

IZLAZ: X = 6, Y = 2, Z = 4

Primer 3

IzLAZ: X = 5, Y = 4, Z = 4

Primer 4

IzLAZ: X = 5, Y = 2, Z = 4

Primer 5

IzLAZ: X = 4, Y = 4, Z = 4

Primer 6

IzLAZ: X = 4, Y = 2, Z = 4

Primer 7

IzLAZ: X = 3, Y = 8, Z = 1

Primer 8

IzLAZ: X = 3, Y = 6, Z = 1

Primer 9

IzLAZ: X = 3, Y = 2, Z = 1

Zadatak 1.1.4

Napisati program koji pronalazi sve vrednosti promenljivih X, Y, Z i W za koje važe sledeći uslovi:

- $X \geq 2 \cdot W$
- $3 + Y \leq Z$
- $X - 11 \cdot W + Y + 11 \cdot Z \leq 100$

Domeni promenljivih su:

- $X \in \{1, 2, \dots, 110\}$
- $Y \in \{1, 3, 5, \dots, 51\}$
- $Z \in \{10, 20, 30, \dots, 100\}$
- $W \in \{1, 8, 27, 64, 125, 216, 343, 512, 729, 1000\}$

Sve ispravne kombinacije ispisati na standardni izlaz u formatu:
X:broj, Y:broj, Z:broj, W:broj

Primer 1

IzLAZ: X:87, Y:15, Z:20, W:27

Primer 2

IzLAZ: X:85, Y:15, Z:20, W:27

Primer 3

IzLAZ: X:58, Y:15, Z:20, W:27

Primer 4

IzLAZ: X:56, Y:15, Z:20, W:27

Primer 5

```
IzLAZ: X:54, Y:15, Z:20, W:27
```

Primer 6

```
IzLAZ: X:109, Y:13, Z:20, W:27
```

Primer 7

```
IzLAZ: X:79, Y:3, Z:20, W:27
```

Primer 8

```
IzLAZ: X:77, Y:3, Z:20, W:27
```

Primer 9

```
IzLAZ: X:75, Y:3, Z:20, W:27
```

Primer 10

```
IzLAZ: X:55, Y:1, Z:10, W:8
```

Zadatak 1.1.5

Dati su novčići od 1, 2, 5, 10 i 20 dinara. Napisati program koji pronalazi sve moguće kombinacije ovih novčića tako da njihov zbir iznosi tačno 50 dinara. Sve kombinacije ispisati na standardni izlaz.

Primer 1

```
IzLAZ: 1 din: 0
        2 din: 0
        5 din: 0
        10 din: 1
        20 din: 2
        Ukupno: 50
```

Primer 2

```
IzLAZ: 1 din: 0
        2 din: 0
        5 din: 2
        10 din: 0
        20 din: 2
        Ukupno: 50
```

Primer 3

```
IzLAZ: 1 din: 1
        2 din: 2
        5 din: 1
        10 din: 0
        20 din: 2
        Ukupno: 50
```

Primer 4

```
IzLAZ: 1 din: 3
        2 din: 1
        5 din: 1
        10 din: 0
        20 din: 2
        Ukupno: 50
```

Primer 5

```
IzLAZ: 1 din: 5
        2 din: 0
        5 din: 1
        10 din: 0
        20 din: 2
        Ukupno: 50
```

Primer 6

```
IzLAZ: 1 din: 0
        2 din: 5
        5 din: 0
        10 din: 0
        20 din: 2
        Ukupno: 50
```


Zadatak 1.1.6

Napisati program koji učitava ceo broj n i ispisuje magičnu sekvencu S brojeva od 0 do $n - 1$. Sekvenca $S = (x_0, x_1, \dots, x_{n-1})$ je magična ako za svako $i = 0, 1, \dots, n - 1$ važi da se broj i pojavljuje tačno x_i puta u sekvenci S . Sve rezultate ispisati na standardni izlaz.

Primer 1

```
ULAZ : Duzina sekvence: 5  
IZLAZ: [2, 1, 2, 0, 0]
```

Primer 2

```
ULAZ : Duzina sekvence: 7  
IZLAZ: [3, 2, 1, 1, 0, 0, 0]
```

Primer 3

```
ULAZ : Duzina sekvence: 4  
IZLAZ: [2, 0, 2, 0]  
IZLAZ: [1, 2, 1, 0]
```

Primer 4

```
ULAZ : Duzina sekvence: 6  
IZLAZ: Ne postoji magicna  
       sekvenca duzine 6
```

1.2 Kriptoaritmetike

Postavke zadatka

U svim zadacima u okviru ovog poglavlja, potrebno je napisati program koji dodeljuje različite cifre različitim karakterima tako da su jednačine u datim zadacima tačne. Svakom slovu odgovara cifra od 0 do 9. Cifra koja odgovara prvom karakteru ne može biti nula. Pronaći sve moguće kombinacije i ispisati ih na standardni izlaz u odgovarajućem obliku aritmetičke operacije.

Zadatak 1.2.1

$$\begin{array}{r} \text{TWO} \\ + \text{TWO} \\ \hline \text{FOUR} \end{array}$$

Primer 1

```
IzLAZ:  765
        +765
        =1530
```

Primer 2

```
IzLAZ:  734
        +734
        =1468
```

Primer 3

```
IzLAZ:  867
        +867
        =1734
```

Primer 4

```
IzLAZ:  846
        +846
        =1692
```

Primer 5

```
IzLAZ:  836
        +836
        =1672
```

Primer 6

```
IzLAZ:  928
        +928
        =1856
```

Zadatak 1.2.2

$$SEE + YOU = SOON$$

Primer 1

```
IzLAZ: 199 + 803 = 1002
```

Primer 2

```
IzLAZ: 199 + 804 = 1003
```

Primer 3

```
IzLAZ: 199 + 805 = 1004
```

Zadatak 1.2.3

$$GREEN + ORANGE = COLORS$$

Primer 1

$$\text{IZLAZ: } 83446 + 135684 = 219130$$

Zadatak 1.2.4

$$NO + GUN + NO = HUNT$$

Primer 1

$$\text{IZLAZ: } 87 + 908 + 87 = 1082$$

Zadatak 1.2.5

$$SIX + SIX + SIX = NINE + NINE$$

Primer 1

$$\text{IZLAZ: } 942 + 942 + 942 = 1413 + 1413$$

Zadatak 1.2.6

$$COMPLEX + LAPLACE = CALCULUS$$

Zadatak 1.2.7

$$THIS + IS + VERY = EASY$$

Zadatak 1.2.8

$$CROSS + ROADS = DANGER$$

Zadatak 1.2.9

$$FATHER + MOTHER = PARENT$$

Zadatak 1.2.10

$$WE + WANT + NO + NEW + ATOMIC = WEAPON$$

Zadatak 1.2.11

$$EARTH + AIR + FIRE + WATER = NATURE$$

Zadatak 1.2.12

$$SATURN + URANUS + NEPTUNE + PLUTO = PLANETS$$

Zadatak 1.2.13

$$WHEN + IN + ROME + BE + A = ROMAN$$

Zadatak 1.2.14

$$DONT + STOP + THE = DANCE$$

Zadatak 1.2.15

$$HERE + THEY + GO = AGAIN$$

Zadatak 1.2.16

$$OSAKA + HAIKU + SUSHI = JAPAN$$

Zadatak 1.2.17

$$MACHU + PICCHU = INDIAN$$

Zadatak 1.2.18

$$SHE + KNOWS + HOW + IT = WORKS$$

Zadatak 1.2.19

$$COPY + PASTE + SAVE = TOOLS$$

Zadatak 1.2.20

$$THREE + THREE + ONE = SEVEN$$

Zadatak 1.2.21

$$NINE + LESS + TWO = SEVEN$$

Zadatak 1.2.22

$$ONE + THREE + FOUR = EIGHT$$

Zadatak 1.2.23

$$THREE + THREE + TWO + TWO + ONE = ELEVEN$$

Zadatak 1.2.24

$$SEVEN + SEVEN + SIX = TWENTY$$

Zadatak 1.2.25

$$ONE + ONE + ONE + THREE + THREE + ELEVEN = TWENTY$$

Zadatak 1.2.26

$$EIGHT + EIGHT + TWO + ONE + ONE = TWENTY$$

Zadatak 1.2.27

$$ELEVEN + NINE + FIVE + FIVE = THIRTY$$

Zadatak 1.2.28

$$NINE + SEVEN + SEVEN + SEVEN = THIRTY$$

Zadatak 1.2.29

$$TEN + SEVEN + SEVEN + SEVEN + FOUR + \\ FOUR + ONE = FORTY$$

Zadatak 1.2.30

$$TEN + TEN + NINE + EIGHT + THREE = FORTY$$

Zadatak 1.2.31

$$FOURTEEN + TEN + TEN + SEVEN = FORTYONE$$

Zadatak 1.2.32

$$NINETEEN + THIRTEEN + THREE + TWO + TWO + \\ ONE + ONE + ONE = FORTYTWO$$

Zadatak 1.2.33

$$FORTY + TEN + TEN = SIXTY$$

Zadatak 1.2.34

$$SIXTEEN + TWENTY + TWENTY + TEN + \\ TWO + TWO = SEVENTY$$

Zadatak 1.2.35

$$SIXTEEN + TWELVE + TWELVE + TWELVE + \\ NINE + NINE = SEVENTY$$

Zadatak 1.2.36

$$TWENTY + TWENTY + THIRTY = SEVENTY$$

Zadatak 1.2.37

$$FIFTY + EIGHT + EIGHT + TEN + TWO + TWO = EIGHTY$$

Zadatak 1.2.38

$$FIVE + FIVE + TEN + TEN + TEN + TEN + THIRTY = EIGHTY$$

Zadatak 1.2.39

$$SIXTY + EIGHT + THREE + NINE + TEN = NINETY$$

Zadatak 1.2.40

$$ONE + NINE + TWENTY + THIRTY + THIRTY = NINETY$$

Zadatak 1.2.41

$$MEN * AND = WOMEN$$

Zadatak 1.2.42

$$COGITO = ERGO * SUM$$

Zadatak 1.2.43

$$FERMAT * S = LAST + THEOREM$$

Zadatak 1.2.44

$$WINNIE / THE = POOH$$

Zadatak 1.2.45

$$TWO * TWO + EIGHT = TWELVE$$

1.3 Matematičke slagalice

Zadatak 1.3.1

Magičan kvadrat je kvadrat dimenzija 3×3 takav da je suma svih brojeva u svakom redu, svakoj koloni i svakoj dijagonali jednaka 15, a svi brojevi od 1 do 9 su različiti. Program treba da pronade sve moguće magične kvadrate i ispiše ih na standardni izlaz.

Primer 1

```
IzLAZ: 6 1 8
        7 5 3
        2 9 4
```

Primer 2

```
IzLAZ: 6 7 2
        1 5 9
        8 3 4
```

Primer 3

```
IzLAZ: 8 1 6
        3 5 7
        4 9 2
```

Primer 4

```
IzLAZ: 8 3 4
        1 5 9
        6 7 2
```

Primer 5

```
IzLAZ: 4 3 8
        9 5 1
        2 7 6
```

Primer 6

```
IzLAZ: 4 9 2
        3 5 7
        8 1 6
```

Zadatak 1.3.2

Napisati program koji raspoređuje brojeve od 1 do 9 u dve linije koje se ukrštaju u jednom broju. Svaka linija treba da sadrži tačno 5 brojeva, svi brojevi moraju biti međusobno različiti i iz skupa $\{1, 2, \dots, 9\}$. Brojevi unutar svake linije moraju biti poređani u rastućem redosledu, a zbir svih brojeva u svakoj liniji mora biti jednak 25. Sve ispravne kombinacije treba ispisati na standardni izlaz u obliku koji odgovara obliku slova X:

```

  A           G
    B       F
      C
    D       H
  E           I
```

Primer 1

```
IzLAZ: 1           3
        2       4
          5
        6       8
      7           9
```

Primer 2

```
IzLAZ: 1           2
        4       3
          5
        6       7
      9           8
```

Primer 3

```
IzLAZ: 1           2
        4       3
          5
        7       6
      8           9
```

Zadatak 1.3.3

Napisati program koji pronalazi vrednosti promenljivih A, B, C, D, E, F, G, H, I, J, K, L, M, N, O, P, Q, R, S (svako slovo predstavlja različit broj) koje su poređane u obliku heksagona na sledeći način:

```

  A B C
D E F G
H I J K L
M N O P
  Q R S

```

tako da zbir duž svake horizontalne i dijagonalne linije bude 38, tj:

$$\begin{aligned}
 38 &= A + B + C \\
 &= D + E + F + G \\
 &= H + I + J + K + L \\
 &= M + N + O + P \\
 &= Q + R + S
 \end{aligned}$$

$$\begin{aligned}
 38 &= A + D + H \\
 &= B + E + I + M \\
 &= C + F + J + N + Q \\
 &= G + K + O + R \\
 &= L + P + S
 \end{aligned}$$

$$\begin{aligned}
 38 &= C + G + L \\
 &= B + F + K + P \\
 &= A + E + J + O + S \\
 &= D + I + N + R \\
 &= H + M + Q
 \end{aligned}$$

Rešenje ispisati u prikazanom heksagonalnom obliku.

NAPOMENA: *Ovako definisan heksagon se naziva magični heksagon reda tri, i ima jedinstveno rešenje.*

Primer 1

IzLAZ:	3 17 18
	19 7 1 11
	16 2 5 6 9
	12 4 8 14
	10 13 15

Zadatak 1.3.4

Napisati program koji raspoređuje 8 topova na šahovskoj tabli dimenzija 8×8 tako da se nikoja dva topa međusobno ne napadaju. Topovi se međusobno napadaju ukoliko se nalaze u istoj vrsti ili koloni. Potrebno je pronaći i prikazati sve moguće rasporede topova koji zadovoljavaju uslov da se međusobno ne napadaju.

Sve rezultate ispisati na standardni izlaz, jedan rezultat predstavlja jedno moguće rešenje i sastoji se od osam redova sa po osam karaktera, pri čemu karakter T označava poziciju topa, a znak - prazno polje.

Primer 1

```
IzLAZ: T-----
        -----T
        -T-----
        -----T-
        --T-----
        ---T-----
        -----T--
        -----T--
```

Primer 2

```
IzLAZ: T-----
        -----T
        -T-----
        -----T-
        --T-----
        ---T-----
        -----T--
        -----T--
```

Zadatak 1.3.5

Napisati program koji raspoređuje 8 dama na šahovsku tablu dimenzija 8×8 tako da se nikoje dve dame međusobno ne napadaju. Dame se međusobno napadaju ukoliko se nalaze u istoj vrsti, koloni ili dijagonali. Potrebno je pronaći i prikazati sve moguće rasporede dama koji zadovoljavaju ovaj uslov.

Svaki rezultat treba ispisati na standardni izlaz, pri čemu jedan raspored predstavlja jedno rešenje i sastoji se od osam redova sa po osam karaktera. Karakter D označava poziciju dame, a karakter - prazno polje.

Primer 1

```
IzLAZ: --D-----
        -----D--
        -D-----
        -----D-
        D-----
        ---D-----
        -----D
        -----D--
```

Primer 2

```
IzLAZ: --D-----
        -----D--
        -----D
        D-----
        ---D-----
        -----D-
        -----D--
        -D-----
```

Zadatak 1.3.6

Napisati program koji učitava tablu za igru Sudoku iz datoteke čije ime se zadaje sa standardnog ulaza i rešava Sudoku zagonetku koristeći ograničenja. Sudoku je kvadratna tabla dimenzija 9×9 podeljena na devet 3×3 kvadrata. Svaki red, svaka kolona i svaki od devet manjih kvadrata mora da sadrži brojeve od 1 do 9 bez ponavljanja. Ulaz programa predstavlja ime datoteke u kojoj se nalazi tabla, gde su nule predstavljene kao prazna polja, a izlaz rešena Sudoku tabla ispisana na standardni izlaz.

Primer 1

ULAZ : Unesite ime datoteke sa tablom za sudoku: sudoku2.json

ULAZNA DATOTEKA sudoku2.json:

```
[[6, 3, 8, 7, 0, 0, 0, 0, 0],
 [0, 2, 7, 0, 0, 0, 5, 0, 0],
 [5, 4, 0, 6, 0, 9, 2, 0, 0],
 [0, 1, 0, 5, 9, 0, 3, 0, 0],
 [0, 0, 9, 0, 7, 0, 4, 0, 0],
 [0, 0, 6, 0, 2, 4, 0, 1, 0],
 [0, 0, 5, 3, 0, 1, 0, 2, 7],
 [0, 0, 2, 0, 0, 0, 8, 3, 0],
 [0, 0, 0, 0, 0, 7, 9, 4, 5]]
```

IZLAZ: =====

```
|638 | 752 | 194 |
|927 | 418 | 563 |
|541 | 639 | 278 |
```

```
-----
|714 | 596 | 382 |
|289 | 173 | 456 |
|356 | 824 | 719 |
```

```
-----
|895 | 341 | 627 |
|472 | 965 | 831 |
|163 | 287 | 945 |
=====
```

1.4 Optimizacioni problemi

Zadatak 1.4.1

Pekara *Kiflica* proizvodi hleb i kifle. Za mešenje hleba potrebno je 10 minuta, dok je za kiflu potrebno 12 minuta. Vreme potrebno za pečenje ćemo zanemariti. Testo za hleb sadrži 300g brašna, a testo za kiflu sadrži 120g brašna. Zarada koja se ostvari prilikom prodaje jednog hleba je 7 dinara, a prilikom prodaje jedne kifle je 9 dinara. Ukoliko pekara ima 20 radnih sati za mešenje peciva i 20 kg brašna, koliko komada hleba i kifli treba da se umesi kako bi se ostvarila maksimalna zarada (pod pretpostavkom da će pekara sve prodati)? Na standardni izlaz ispisati maksimalnu zaradu koju je moguće ostvariti, kao i koliko komada hleba a koliko komada kifli je potrebno umesiti da bi se ostvarila maksimalna zarada.

Primer 1

IZLAZ: Maksimalna zarada je 876 dinara, za nju je potrebno 39 hlebova i 67 kifli.

Zadatak 1.4.2

Kompanija *Start* ima 250 zaposlenih radnika. Rukovodstvo kompanije je odlučilo da svojim radnicima obezbedi dodatnu edukaciju. Za obuku u programskom jeziku *Elixir* potrebno je platiti 100 evra po osobi, a nakon obuke radnik ostvaruje 150 projekat/sati mesečno, što kompaniji donosi dobit od 5 evra po satu. Za obuku u programskom jeziku *Dart* potrebno je platiti 105 evra po osobi, a nakon obuke radnik ostvaruje 170 projekat/sati mesečno, što kompaniji donosi dobit od 6 evra po satu. Kompanija ima na raspolaganju ukupno 26000 evra za edukaciju i 51200 projekat/sati mesečno koje može iskoristiti. Napisati program koji određuje koliko radnika treba poslati na kurs *Elixir*, a koliko na kurs *Dart*, tako da ukupna dobit bude maksimalna. Pretpostaviti da kompanija može obučiti najviše 250 radnika.

Primer 1

IZLAZ: Maksimalna zarada je 166000, broj radnika koje treba poslati na kurs *Elixir* je 50, a broj radnika koje treba poslati na kurs *Dart* je 200.

Zadatak 1.4.3

Čistačica Mica sređuje i čisti kuće i stanove. Da bi sredila i počistila jedan stan potrebno joj je 1 sat, dok joj je za kuću potrebno 1.5 sati. Prilikom čišćenja, Mica potroši određenu količinu deterdženta: 120 ml po stanu, odnosno 100 ml po kući. Mica zaradi 1000 dinara po svakom stanu, odnosno 1500 dinara po kući. Ukoliko Mica radi 40 sati nedeljno i ima 5 l deterdženta na raspolaganju. Na standardni izlaz ispisati koliko stanova i kuća je potrebno da očisti kako bi imala najveću zaradu kao i kolika je najveća moguća zarada.

Primer 1

IZLAZ: Najveća moguća zarada je 40000 i ona se ostvaruje kada se očisti 1 stan(ova) i 26 kuća.

Zadatak 1.4.4

Marija se bavi grnčarstvom i pravi šolje i tanjire. Da bi se napravila šolja, potrebno je 6 minuta, dok je za tanjir potrebno 3 minuta. Pri pravljenju šolje potroši se 75 g, dok se za tanjir potroši 100 g gline. Ukoliko ima 20 sati na raspolaganju za izradu svih proizvoda i 250 kg gline, a zarada koju ostvari iznosi 2 evra po svakoj šolji i 1.5 evra po tanjiru. Na standardni izlaz ispisati koliko šolja i tanjira Marija mora da napravi kako bi ostvarila maksimalnu zaradu.

Primer 1

IZLAZ: Najveća moguća zarada je 600.00 i ona se ostvaruje kada se napravi 0 solja i 400 tanjira.

Zadatak 1.4.5

Jovanin komšija preprodaje računare i računarsku opremu. Očekuje isporuku računara i štampača. Pri tom, računari su spakovani tako da njihova kutija zauzima 360 kubnih decimetara prostora, dok se štampači pakuju u kutijama koje zauzimaju 240 kubnih decimetara prostora. Komšija se trudi da mesečno proda najmanje 30 računara i da taj broj bude bar za 50% veći od broja prodatih štampača. Računari koštaju 200 evra po nabavnoj ceni, a prodaju se po ceni od 400 evra, dok štampači koštaju u nabavci 60 evra i prodaju se za 140 evra. Magacin kojim komšija raspolaže ima svega 30000 dm^3 prostora i mesečno može da nabavi robu u iznosu od najviše 14000 evra. Na standardni izlaz ispisati koliko računara, a koliko štampača komšija treba da proda kako bi se maksimalno obogatio.

Primer 1

IzLAZ: Maksimalna zarada je: 14680, broj racunara: 59,
broj stampaca: 36

1.5 Ispitni zadaci

Zadatak 1.5.1

Napisati program koji pronalazi sva rešenja jednačine^a:

$$((JE + PENSE) - DONC) + JE = SUIS$$

Različitim slovima odgovaraju različiti brojevi. Brojevi ne mogu početi cifrom 0. Na standardni izlaz ispisati sva rešenja. Rešenja je potrebno numerisati pri ispisu.

Primer 1

```
IzLAZ: 1. ((47 + 17587) - 9653) + 47 = 8028
        2. ((47 + 17587) - 9053) + 47 = 8628
        3. ((47 + 17597) - 8652) + 47 = 9039
        4. ((47 + 17597) - 8052) + 47 = 9639
        5. ((35 + 15065) - 8709) + 35 = 6426
        6. ((35 + 15065) - 8409) + 35 = 6726
        7. ((83 + 13473) - 6542) + 83 = 7097
        8. ((83 + 13473) - 6042) + 83 = 7597
        9. ((35 + 15085) - 6207) + 35 = 8948
        10. ((35 + 15085) - 6907) + 35 = 8248
        11. ((30 + 10740) - 5976) + 30 = 4824
        12. ((30 + 10740) - 5876) + 30 = 4924
        13. ((52 + 12072) - 4309) + 52 = 7867
        14. ((52 + 12072) - 4809) + 52 = 7367
```

^aJe pense, donc je suis, fr. Mislím, dakle postojim.

Zadatak 1.5.2

Napisati program koji pronalazi (i ispisuje na standardni izlaz) jedno rešenje jednačine:

$$MANET + MATISSE + MIRO + MONET + RENOIR = ARTISTS.$$

Različitim slovima odgovaraju različiti brojevi. Nijedan broj ne sme počinjati cifrom 0. Na standardni izlaz ispisati jedno (bilo koje ispravno) rešenje u abecednom redosledu.

Primer 1

```
IzLAZ: [1,2,0,3,4,5,6,7]
```

Primer 2

```
IzLAZ: [1,2,0,3,4,5,6,8]
```


Zadatak 1.5.3

Na Milanovoj farmi uzgajaju se samo zečevi i kokoške. Na farmi ima ukupno 18 glava i 56 nogu. Napisati program koji određuje i na standardni izlaz ispisuje koliko ima životinja koje vrste na farmi.

Primer 1

```
IzLAZ: Kokoske: 7, Zecevi: 11
```

Zadatak 1.5.4

Ivan kreće na put da proda svoje proizvode (viski, parfem, čaše). Njegov ranac ima 9 delova. Viski zauzima 4 mesta i košta 15 evra, parfem zauzima 3 mesta i košta 10 evra, dok čaša zauzima 2 mesta i košta 7 evra. Napisati program koji određuje koliko kog proizvoda Ivan treba da ponese kako bi maksimizovao profit.

Primer 1

```
IzLAZ: Viski: 1, Parfem: 1, Casa: 1
```

Zadatak 1.5.5

Hyper-6 predstavlja šestougao koji pored šest spoljašnjih temena ima i jedno unutrašnje teme, koje je unutrašnjim ivicama povezano sa svakim od njih. Ukoliko su sve ivice ove figure jedinične, onda se ona naziva *ravni Hyper-6*. Primer ravnog *Hyper-6* dat je na slcii ispod, pri čemu su sa A...G označena njegova temena.

```

      A   B
     C   D   E
      F   G
  
```

Napisati program koji u ivice *ravnog Hyper-6* upisuje brojeve od 1 do 7, pri čemu je zbir brojeva u temenima svakog od jediničnih jednakostraničnih trouglova manji od 11, dok je zbir brojeva u temenima svakog od jediničnih rombova manji ili jednak 16.

Primer 1

```
IzLAZ:  2 7
        5 1 3
        4 6
```

Primer 2

```
IzLAZ:  3 5
        7 1 4
        2 6
```

Zadatak 1.5.6

Hyper-6 predstavlja šestougao koji pored šest spoljašnjih temena ima i jedno unutrašnje teme koje je unutrašnjim ivicama povezano sa svakim od njih. Ukoliko su sve ivice ove figure jedinične, onda se ona naziva *ravni Hyper-6*. Glavna paralela *Hyper-6* jeste jedina njegova paralela koja sadrži tri temena i paralelna je osi Ox (podrazumevamo položaj u ravni tako da su dve stranice paralelne osi Ox). Primer ravnog *Hyper-6* dat je na slcii ispod, pri čemu su sa $A \dots G$ označena njegova temena (pri ovim oznakama, D je unutrašnje teme a glavna paralela je $C - D - E$).

```

      A   B
     C   D   E
      F   G

```

Napisati program koji raspoređuje različite brojeve od 1 do 7 u svako teme ravnog *Hyper-6*, pri čemu važi da zbir svakog od spoljašnjih temena sa susedima nije veći od 15. Pronaći takav raspored gde su elementi na glavnoj paraleli takvi da im je zbir maksimalan a proizvod minimalan.

Primer 1

```

IZLAZ:  4 7
        3 1 2
        6 5

```

Zadatak 1.5.7

Čuveni kuvar Gordon Remzi stigao je u Beograd na poslovni sastanak sa ciljem da otvori svoj novi restoran. U sred noći probudio se gladan i rešio je da sam sebi pripremi obrok — proteinsku palačinku. Otišao je do obližnje radnje i shvatio da ima dva problema — radnja nije tako bogata sastojcima, a u džepu ima samo 10 evra, koje može da zameni po kursu od 117 dinara (iako je zvanični kurs Narodne banke Srbije 117.52 dinara za evro!).

Sastojke kupuje u kesicama od po 100 grama. Može da kupi najviše 10 kesica, pri čemu želi da napravi palačinku sa što više proteina, a što manje masti i šećera. Ukupna količina masti mora biti manja od 500 grama, dok šećera ne sme biti više od 150 grama.

Koristeći podatke iz tabele, izračunati maksimalnu količinu proteina koju Gordon može da postigne, poštujući sva ograničenja. Na standardni izlaz ispisati **samo** jedan broj: maksimalnu količinu proteina. Gordon ima ukupno 1170 dinara ($10 \text{ evra} \times 117 \text{ RSD}$).

Sastojak	Cena (din/kesica)	Broj kesica u radnji	Protein (g)	Masti (g)	Šećer (g)
brašno	30	10	20	30	5
plazma	300	20	15	10	30
jaja	50	7	70	150	2
mleko	170	5	40	32	15
višnja	400	3	23	3	45
nutela	450	9	7	15	68

Primer 1

IZLAZ: 355

Zadatak 1.5.8

Projekat *DataForge* koristi razne *Python* biblioteke. Svaka biblioteka može zavisiti od drugih biblioteka, ali može imati i konflikte sa njima. Cilj je instalirati što je više biblioteka moguće, uz maksimalnu kompatibilnost među bibliotekama (tj. ispoštovati sve zavisnosti i izbeći konflikte), a pri tome maksimizovati sumu prioriteta instaliranih biblioteka. Ako je biblioteka instalirana, sve biblioteke od kojih ona zavisi takođe moraju biti instalirane. Dve biblioteke koje su u konfliktu ne mogu biti instalirane istovremeno. Sve biblioteke sa sobom nose određenu cenu resursa koje troše, a ukupan trošak resursa (cena) instaliranih biblioteka ne sme prelaziti 15. Takođe, želimo da instaliramo bar dve biblioteke. Koristeći podatke iz naredne tabele pronaći kombinaciju instaliranih biblioteka tako da se postigne maksimalna moguća vrednost sume njihovih prioriteta. Na standardni izlaz ispisati instalirane biblioteke.

Biblioteka	Zavisí od	Konflikti	Cena	Prioritet
<i>NumPy</i>	-	<i>Keras</i>	3	2
<i>Keras</i>	-	<i>NumPy, Pandas</i>	5	3
<i>Pandas</i>	-	<i>Keras</i>	2	1
<i>Seaborn</i>	<i>NumPy</i>	-	4	2
<i>OpenCV</i>	<i>Keras, Pandas</i>	-	3	3

Tabela 1.1: Zavisnosti i konflikti softverskih biblioteka

Primer 1

```
IZLAZ: Instalirane biblioteke: NumPy Pandas Seaborn.
```

Zadatak 1.5.9

Problem ranca predstavlja jedan od najpoznatijih problema oblasti matematičke optimizacije. Potrebno je u ranac kapaciteta W staviti predmete, od kojih je svaki vrednosti v_i tako da zbir svih v_i u rancu bude maksimalan. Za svaki predmet i poznata je njegova vrednost v_i , njegova težina w_i i koliko komada tog predmeta nam je dostupno c_i (za predmet i u ranac nije moguće staviti više od c_i komada predmeta).

1. Napisati predikat `get(X, I, Y)` koji iz liste `X` vraća element `Y` koji se nalazi na indeksu `I`. Pretpostaviti da indeksiranje počinje od 0.
2. Napisati predikat `ranac(X, Brojaci, Tezine, Vrednosti, Kapacitet)` koji rešava problem ranca tako što u listu `X` upisuje koliko instanci kog predmeta će biti odabrano i ispisuje kolika su zarada i težina ranca pri odabiru tih predmeta.

Sledeća tabela predstavlja jednu instancu opisanog problema koju je potrebno rešiti. Pretpostaviti da postoje tačno 4 predmeta: laptop, dijamant, pantalone i četkica za zube.

ime	laptop	dijamant	pantalone	četkica
i	0	1	2	3
v	3400	1800	200	10
w	500	7	55	1
c	2	3	5	4

Na primer, za $W = 128$, rešenje je uzeti 3 dijamanta, 1 pantalone i 4 četkice za zube.

Primer 1

```

ULAZ : | ?- ranac(X, [2, 3, 5, 4], [500, 7, 55, 1],
           [3400, 1800, 200, 10], 128)
IZLAZ: Zarada 5640
           Tezina 80
           X = [0,3,1,4]

```

Zadatak 1.5.10

Doktor Mile kreira novu terapiju za jačanje imuniteta. Doktor želi da unošenjem vitamina i minerala poveća količinu hemoglobina u organizmu. Na raspolaganju ima 9 tableta vitamina B, 19 tableta vitamina C, 6 tableta vitamina D, 4 tablete magnezijuma, samo dve kapsule selen i 8 tableta cinka. Potrebno je odabrati, za svaki dan u nedelji po najviše jednu tabletu, kojom će se povećati količina hemoglobina u krvi. Trajanje sprint terapije vitaminima i mineralima jeste sedam dana.

Doktor Mile na raspolaganju ima 100 evra (koje može da zameni po kursu od 118 dinara za evro), pri čemu tableta vitamina B košta 130 dinara, vitamina C 800 dinara, vitamina D 150 dinara, magnezijuma i selen 370 odnosno 490 dinara redom, dok tableta cinka košta isto koliko i tableta vitamina D.

Kako unos vitamina i minerala može dovesti do neravnoteže u organizmu, doktor mora voditi računa da njegova terapija ne poveća sintezu proteina za više od 100 g, niti stimuliše sintetisanje više od 200 g amino-kiselina. Količina proteina i amino-kiselina koje tableta svakog od pomenutih suplemenata sintetise (u gramima) data je u tabeli:

Sastojak	Stimulacija sinteze proteina	Stimulacija sinteze amino-kiselina
vitamin B	15	33
vitamin C	11	31
vitamin D	10	20
magnezijum	22	18
selen	1	21
cink	13	16

Potrebno je kreirati sprint terapiju tako da se maksimizuje sinteza hemoglobina, pri čemu jedna tableta vitamina B sintetise 92.5 g hemoglobina, vitamina C 155.5 g, vitamina D 79.6 g, magnezijuma 156.2 g, seleni čak 413 g i cinka 137.7 g. Na standardni izlaz ispisati, zaokruženo na dve decimale, maksimalnu količinu hemoglobina koju je moguće sintetisati pri navedenim uslovima.

Primer 1

IZLAZ: 1605.60

Zadatak 1.5.11

Marko se zaljubio. Srećom, on radi u fabrici čokolade gde na raspolaganju ima nekoliko vrsta čokolade. Svojoj devojci Marko želi napraviti najslađu čokoladu koju može spakovati u torbu. Da devojka ne bi primetila čokoladu ukoliko se slučajno sretnu pre nego što Marko pripremi poklon, on može kupiti najviše 3kg čokolade. U torbu može staviti najviše 1 dm^3 čokolade. Za kupovinu čokolade Marko na raspolaganju ima 300 evra.

Koristeći podatke iz sledeće tabele, izračunati koliko koje čokolade Marko mora da uzme kako bi napravio poklon maksimalne slatkoće, vodeći računa o ograničenjima mase, zapremine i cene. Na standardni izlaz ispisati ukupnu slatkoću Markovog poklona, kao i njegov sastav.

Tip čokolade	Masa (g)	Dimenzije (cm)	Cena (EUR)	Slatkoća
Čokolada A	100	$8 \times 2.5 \times 0.5$	20	8
Čokolada B	45	$7 \times 2 \times 0.5$	16	6.8
Čokolada C	10	$3 \times 2 \times 0.5$	9	4
Čokolada D	25	$3 \times 3 \times 0.5$	7	3

Primer 1

```

IZLAZ: Ukupna slatkoća: 365.0
        Čokolada A: 27
        Čokolada B: 2
        Čokolada C: 16
        Čokolada D: 2

```

Zadatak 1.5.12

Nikola se sprema za takmičenje u bodibildingu, i potrebno mu je da svoju ishranu dovede do savršenstva. Na raspolaganju ima četiri suplementa: whey protein, kreatin monohidrat, i anaboličke steroide: testosteron i danazol. Prema planu ishrane, Nikola može pojesti najviše 3 kg suplementacije. Jedna doza whey proteina teži 100 g, kreatina 45 g, a anabolika — testosterona i danazola — 10 g i 25 g. Svaki suplement povećava vezivanje vode u organizmu, koje ne sme preći 1 dm^3 . Doza whey proteina vezuje 3 cm^3 vode, kreatin 10 cm^3 , testosteron 15 cm^3 , a danazol 20 cm^3 . Nikola na raspolaganju ima po 20 doza svakog suplementa, i ukupni budžet iznosi 300 EUR. Cena jedne doze je:

- whey protein — 8 EUR
- kreatin — 6 EUR
- testosteron — 14 EUR
- danazol — 11 EUR

Efekat na povećanje mišićne mase po dozi je:

- whey protein — 5 g
- kreatin — 11 g
- testosteron — 20 g
- danazol — 15 g

Izračunati i na standardni izlaz ispisati koliko doza kog suplementa Nikola treba da unese kako bi maksimizirao ukupnu dobijenu mišićnu masu, poštujući ograničenja mase, zapremine, cene i broja doza.

Primer 1

```
IZLAZ: Maksimalno povecanje misicne mase: 454
Plan ishrane:
  1 doza whey proteina,
  19 doza kreatina monohidrata,
  0 doza testosterona,
  16 doza danazola
```


Zadatak 1.5.13

Ferarijevi stručnjaci rade na poboljšanju formule za ovogodišnju F1 trku na *Hungaroringu*. Potrebno je na bolid dodati nove komponente koje bi povećale njegovu snagu i brzinu. Na raspolaganju su:

- 4 katalizatora D17,
- 31 filter za sagorevanje R35,
- 14 turbo pumpi Z,
- 3 pogonska motora *Nitro Scalar*.

Ubrzanja koje daju komponente su sledeća:

- katalizator (D17): 9.3 sekundi,
- filter za sagorevanje (R35): 9.9 sekundi,
- turbo pumpa (Z): 2.17 sekundi,
- Nitro Scalar motor: 303.5 sekundi.

Trkački kolegijum doneo je sledeća ograničenja:

- Ukupna težina komponenti ne sme preći 20 000g, pri čemu su težine komponenti redom:

480g, 3980g, 290g, 6600g.

- Ukupna zapremina dodatnih komponenti ne sme preći 3500cm³, pri čemu su dimenzije komponenti (u *cm*):

6×14 , 2.4×7.2 , 2×3 , 24.9×110 .

- Budžet je ograničen na 20 000 evra, sa cenama po komadu:

800, 1300, 120, 9000 evra.

Napisati program koji računa i ispisuje maksimalno moguće ukupno ubrzanje bolida (u sekundama) koje se može postići korišćenjem dostupnih komponenti, a pri tome poštuje sva navedena ograničenja. Na standardni izlaz ispisati i odgovarajuće komponente.

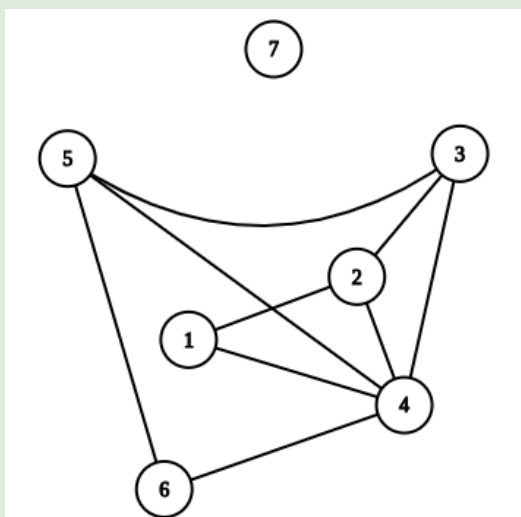
Primer 1

```
IzLAZ: Moze se postici ubrzanje od 386.54s. Potrebne komponente:
        Nitro Solar: 1,
        Katalizator D17: 4,
        Turbo Pumpa: 12,
        Filter R35: 2
```

Zadatak 1.5.14

Mnogi problemi u računarstvu (poput socijalnih mreža, kompilatorskih internih reprezentacija, struktura podataka, modela neuronskih mreža itd.) mogu se modelovati grafovima. Problem bojenja grafa izučava se od 70-tih godina prošlog veka. Zbog eksponencijalne složenosti jedan je od težih problema koji se rešava uz pomoć raznih heuristika.

Milan pokušava da reši problem PP-bojenja grafa sa slike 1.1. Napisati program koji pronalazi sva validna PP-bojenja zadanog grafa (boje kodirati brojevima: 0 – crvena, 1 – plava, 2 – bela, a rezultat ispisati kao vektor gde indeksi u vektoru odgovaraju rednim brojevima čvorova grafa).



Slika 1.1: Graf koji Milan želi da oboji.

NAPOMENA: PP-bojenje grafa sa slike 1.1 podrazumeva bojenje grafa sa 3 boje (crvenom, plavom i belom), pri čemu je svakom od čvorova dodeljena boja i nikoga dva susedna čvora nisu obojena istom bojom. Dodatno, čvor broj 7 mora biti obojen crvenom bojom, dok čvor broj 3 ne sme biti obojen zelenom bojom.

Primer 1

IzLAZ: [1,1,0,2,1,2,0]

Primer 2

IzLAZ: [1,1,2,0,1,2,0]

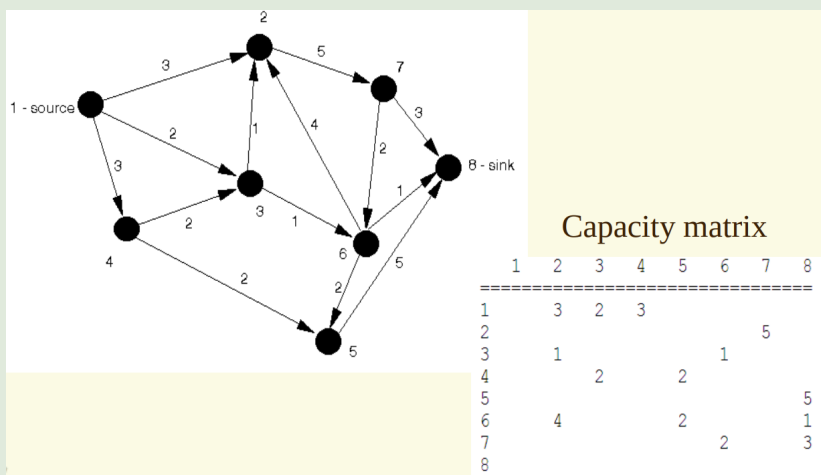
Zadatak 1.5.15

Neka je G usmereni graf $G = (V, E)$, gde je V skup njegovih čvorova a E skup usmerenih grana između njih ($E \subseteq V \times V$). Kapaciteti grana definišu se funkcijom $c : V \times V \rightarrow \mathbb{R}^+$. U grafu G definišu se dva istaknuta čvora: izvor (eng. source, s) i ponor (eng. sink, t). Protok kroz graf G definiše se kao funkcija $f : V \times V \rightarrow \mathbb{R}$ koja zadovoljava:

- $(\forall u, v \in V) f(u, v) \leq c(u, v)$ (ograničenje kapaciteta)
- $(\forall u, v \in V) f(u, v) = -f(v, u)$
- $(\forall u \in V \setminus \{s, t\}) \sum_{v \in V} f(u, v) = 0$

Vrednost protoka kroz graf G definiše se kao: $|f| = \sum_{v \in V} f(s, v)$. Problem pronalaska maksimalnog protoka definiše se kao problem pronalaska maksimalnog protoka koji može biti poslat između izvora (čvora s) i ponora (čvora t).

Napisati program koji rešava problem maksimalnog protoka. Program na standardni izlaz ispisuje maksimalni protok koji se može postići za konkretan graf definisan na slici 1.2.



Slika 1.2: Protok kroz test graf.

Primer 1

IZLAZ: 7

Zadatak 1.5.16

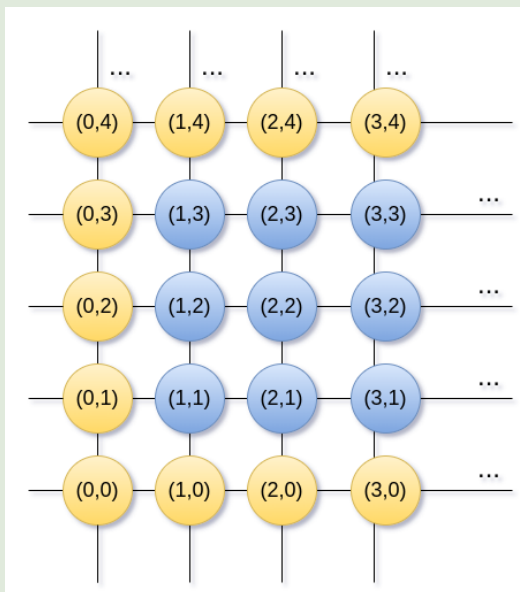
Celobrojna rešetka predstavlja jedan od osnovnih pojmova geometrije brojeva. Definisana je formulom:

$$C = \{ f(m, n) \in \mathbb{R} \mid m, n \in \mathbb{Z} \}$$

gde je $f : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{R}$. Podmreža celobrojne rešetke C_1 dimenzije 3×3 definiše se kao:

$$C_1 = \{ g(m, n) \in \mathbb{R} \mid m, n \in \mathbb{Z}, 1 \leq m, n \leq 3 \}$$

gde je g proizvoljna funkcija $g : \mathbb{Z} \times \mathbb{Z} \rightarrow \mathbb{R}$ (označena plavom bojom na narednoj slici).



Slika 1.3: Primer podrešetke C_1

Napisati program koji raspoređuje brojeve $1, 2, \dots, 9$ u čvorove rešetke C_1 , pri čemu je u svaki čvor upisan broj **različite parnosti** od zbira koordinata tog čvora. Na izlaz ispisati jedno validno popunjavanje rešetke u tri reda.

Primer 1

```

IZLAZ:  3 6 9
         2 5 8
         1 4 7
  
```

Zadatak 1.5.17

Na standardni izlaz ispisati dvodimenzioni vektor X iz celobrojne rešetke $[0, 10]^2$, koji zadovoljava sledeći maksimizacioni problem:

$$\max(\|X\|_2),$$

pri uslovu $AX < b$, gde je

$$A = \begin{bmatrix} 2 & 3 \\ 45 & -34 \end{bmatrix}, \quad b = \begin{bmatrix} 23 & 12 \end{bmatrix}.$$

Primer 1

IZLAZ: $x = [0, 7]$

Zadatak 1.5.18

Igra *Kakuro*, nastala u Japanu, je matematička verzija ukrštenice koja koristi zbir cifara umesto reči. Cilj je popuniti bele kvadrate ciframa od 1 do 9 tako da svaka "reč" (vrsta ili kolona) ima dati zbir. Na primer, na slici 1.4 mora važiti $X_1 + X_2 = 5$, $X_3 + X_4 + X_5 + X_6 = 17$, itd. Nijedna cifra se ne sme koristiti više od jednom u svakoj "reči".

Napisati program koji rešava Kakuro zagonetku prikazanu na slici 1.4. Na standardni izlaz ispisati vektor u kome vrednost na indeksu i odgovara vrednosti promenljive X_{i+1} sa slike 1.4.

			11	4		
	5	x1	x2			
14				10		
17	x3	x4	x5	x6		3
6	x7	x8	4	x9	x10	
		3				
	10	x11	x12	x13	x14	
		3	x15	x16		

Slika 1.4: Kakuro zagonetka.

Primer 1

IZLAZ: $[2, 3, 9, 5, 1, 2, 5, 1, 3, 1, 3, 1, 4, 2, 2, 1]$

Zadatak 1.5.19

Sudoku kvadrat je kvadrat veličine 9×9 , podeljen na 9 manjih kvadrata dimenzija 3×3 . Svaki od ovih kvadrata, kao i svaki red i kolona, mora da sadrži sve 9 brojeva od 1 do 9 bez ponavljanja.

Pronaći i na standardni izlaz ispisati **sva** rešenja (razdvojena belinom) sudoku zagonetke date sledećom početnom konfiguracijom (nepunjene ćelije označene su zvezdicom *):

*	9	*	7	*	*	8	6	*
*	3	1	*	*	5	*	2	*
8	*	6	*	*	*	*	*	*
*	*	7	*	5	*	*	*	6
*	*	*	3	*	7	*	*	*
5	*	*	*	1	*	7	*	*
*	*	*	*	*	*	1	*	9
*	2	*	6	*	*	*	5	*
*	5	4	*	*	8	*	7	*

Primer 1

```

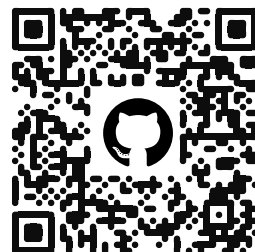
IZLAZ: 2 9 5 7 4 3 8 6 1
        4 3 1 8 6 5 9 2 7
        8 7 6 1 9 2 3 4 5
        3 8 7 4 5 9 2 1 6
        6 1 2 3 8 7 5 9 4
        5 4 9 2 1 6 7 8 3
        7 6 8 5 2 4 1 3 9
        9 2 3 6 7 1 4 5 8
        1 5 4 9 3 8 6 7 2

        2 9 5 7 4 3 8 6 1
        4 3 1 8 6 5 9 2 7
        8 7 6 1 9 2 3 4 5
        3 8 7 4 5 9 2 1 6
        6 1 2 3 8 7 5 9 4
        5 4 9 2 1 6 7 3 8
        7 6 3 5 2 4 1 8 9
        9 2 8 6 7 1 4 5 3
        1 5 4 9 3 8 6 7 2

```

Glava 2

Komponentno programiranje



Uvod

Komponentno programiranje predstavlja proširenje objektno orijentisane paradigme sa ciljem poboljšanja razdvojenosti logike različitih delova softverskog rešenja. Delovi, tj. *komponente*, su odvojene samostalne jedinice koda čijim se vezivanjem gradi složeniji sistem. Komponente se dizajniraju tako da se mogu razvijati nezavisno jedne od drugih, sa unapred dogovorenim interfejsom koji se održava tokom razvoja. U praksi, komponente mogu imati različite uloge i nivoe složenosti — od jednostavnih elemenata korisničkog interfejsa, kao što su dugmad, forme ili navigacioni paneli, do složenijih logičkih celina koje enkapsuliraju određenu funkcionalnost i mogu se ponovo koristiti u različitim delovima aplikacije.

Na višem nivou apstrakcije, komponente mogu predstavljati čitave delove (module) sistema ili čak zasebne programe koji komuniciraju međusobno. U savremenim arhitekturama, posebno kod mikroservisa, komponenta može predstavljati izolovani servis koji pruža funkcionalnost putem programabilnog interfejsa (engl. *API*). Ovakav pristup omogućava lakše održavanje, skaliranje i zamenu pojedinih delova sistema bez uticaja na sistem u celosti, čime se postiže veća fleksibilnost i robusnost softverskog rešenja.

Konvencije

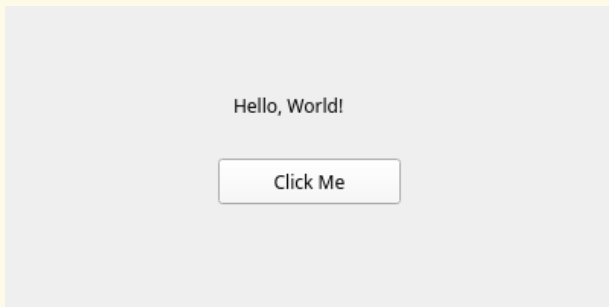
Ovo poglavlje uvodi komponentnu programsku paradigmu kroz jednostavne zadatke koji za cilj imaju dizajn aplikacija koji se sastoji od najmanje dva dela — *prednjeg* (eng. *frontend*) dela korisničkog interfejsa i *zadnjeg* (eng. *backend*) dela koja implementira logiku aplikacije. Delovi aplikacije se razvijaju nezavisno, uz dogovoreni interfejs i interakcije između njih. Međusloj aplikacije definiše veze između prednjeg i zadnjeg dela povezivanjem elemenata prednjeg dela sa odgovarajućim akcijama zadnjeg dela. Prednji i zadnji deo mogu biti posmatrani kao komponente ili kao skupovi komponenata koje međusobno komuniciraju. Elementi korisničkog interfejsa se najčešće sklapaju od gotovih komponenti^a.

^aNa primer, popularni radni okvir *React* se najvećim delom zasniva na komponentnom programiranju.

2.1 Osnovni elementi korisničkog interfejsa

Zadatak 2.1.1: *Hello, World!*

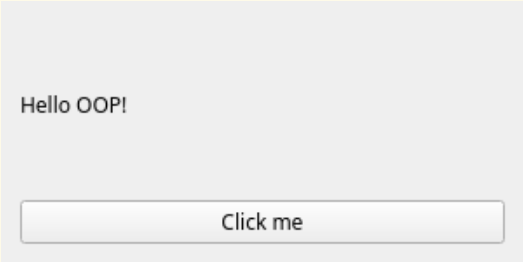
Implementirati program sa komponentom *Label* koja prikazuje tekst *Hello, World!* i dugmetom sa tekстом *Click Me*. Raspored komponenti je prikazan na Slici 2.1. Klikom na dugme *Click Me* se zatvara prozor.



Slika 2.1: Aplikacija *Hello World!*

Zadatak 2.1.2: *Hello, OOP!*

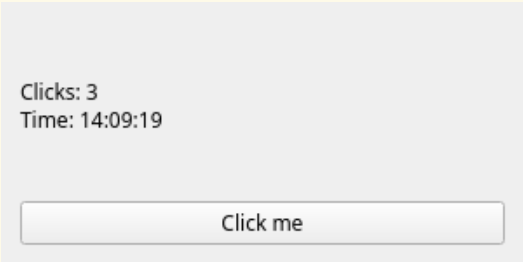
Implementirati program sa komponentom *Label* koja prikazuje tekst **Hello OOP!** i dugmetom sa tekstom **Click Me**, sa izgledom kao na Slici 2.2a. Klikom na dugme **Click Me** se tekst labela menja tako da prikazuje broj klikova dugmeta i vreme poslednjeg klika, kao na Slici 2.2b.



Hello OOP!

Click me

(a)



Clicks: 3
Time: 14:09:19

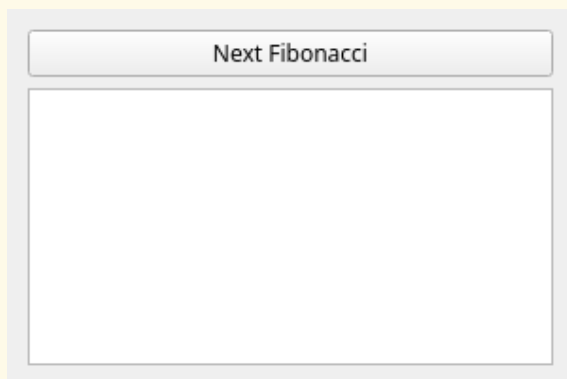
Click me

(b)

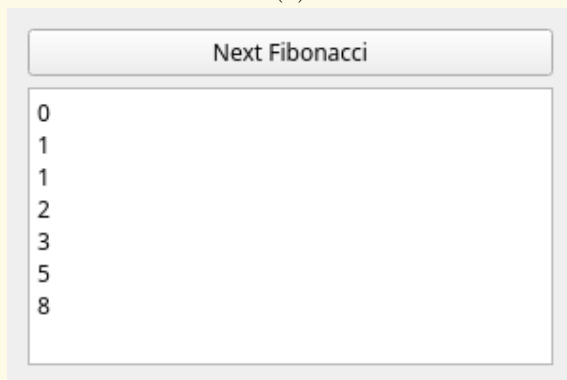
Slika 2.2: Aplikacija *Hello OOP!*

Zadatak 2.1.3: Generator Fibonačijevih brojeva

Implementirati generator Fibonačijevih brojeva sa izgledom kao na Slici 2.3. Aplikacija se sastoji od dugmeta **Next Fibonacci** i polja za čitanje (*read-only*) sa rasporedom kao na slici 2.3a. Klikom na dugme **Next Fibonacci** se generiše naredni Fibonačijev broj i dodaje u polje za čitanje, kao na Slici 2.3b.



(a)

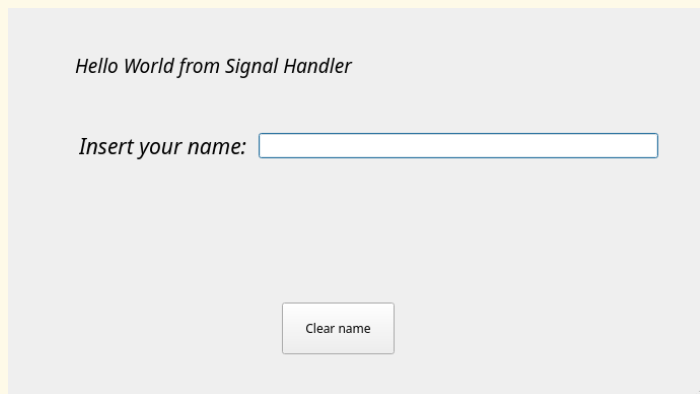


(b)

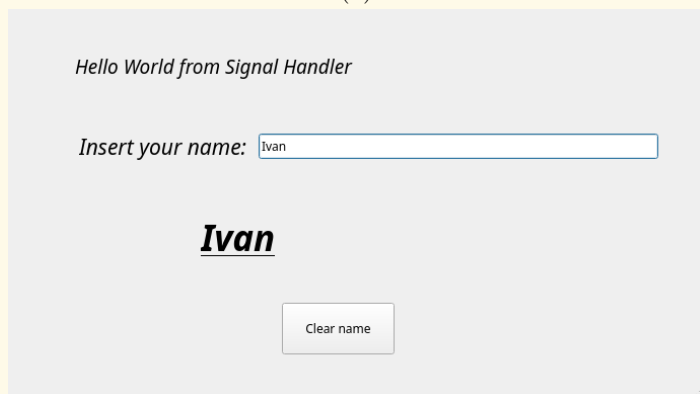
Slika 2.3: Generator Fibonačijevih brojeva

Zadatak 2.1.4: *Hello World from Signal Handler*

Implementirati program sa rasporedom komponenti kao na Slici 2.4a. U jednolinijsko polje za unos se unosi tekst koji aplikacija ispisuje u labelu ispod jednolinijskog polja za unos koja se ažurira u realnom vremenu kako se tekst u polju za unos menja (Slika 2.4b). Klikom na dugme **Clear name** se briše tekst unet u polje za unos i izgled aplikacije se vraća na početni izgled (Slika 2.4a).



(a)

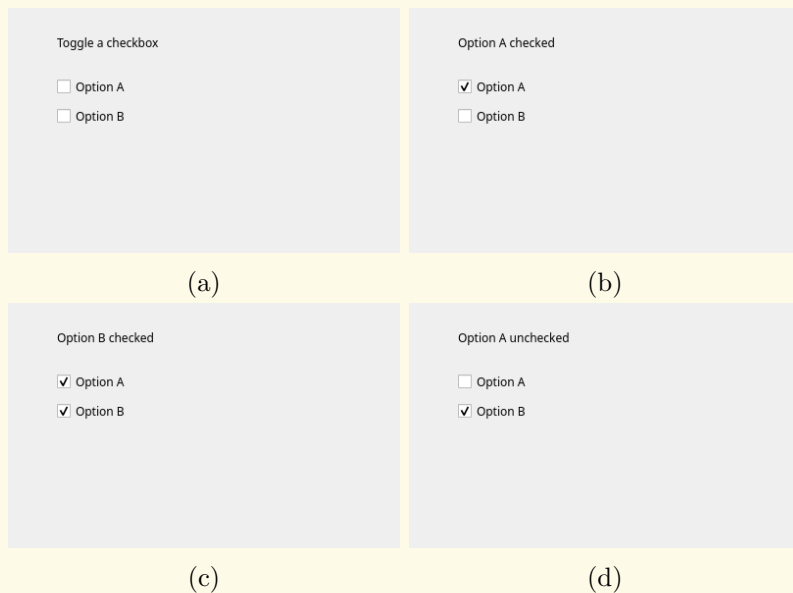


(b)

Slika 2.4: Aplikacija *Hello World from Signal Handler*

Zadatak 2.1.5: *Checkbox toggle*

Implementirati program *Checkbox toggle* sa dve komponente za odabir kao na Slici 2.5a. Odabirom opcija se ažurira tekst labela tako da opisuje poslednju interakciju. Na primer, prilikom odabira opcije A (Slika 2.5b), tekst se ažurira na **Option A checked**. Prilikom odabira opcije B (Slika 2.5c), tekst se ažurira na **Option B checked**. Prilikom uklanjanja odabira, tekst se ažurira na **Option A unchecked** (Slika 2.5d), tj. **Option B unchecked**.



Slika 2.5: Aplikacija *Checkbox toggle*

Zadatak 2.1.6: *Multibox toggle*

Implementirati program *Multibox toggle* sa rasporedom komponenti kao na Slici 2.6a. Klikom na dugme **Show Selected** se prikazuju odabrane opcije redom, kao na Slici 2.6b.

Two screenshots of a Multibox toggle application. Screenshot (a) shows the initial state with three unselected options (A, B, C) and a 'Show Selected' button. Below the button, it says 'No selection'. Screenshot (b) shows the state after clicking 'Show Selected', where options A and C are now checked, and the text below the button reads 'Selected: Option A, Option C'.

(a)

(b)

Slika 2.6: Aplikacija *Multibox toggle***Zadatak 2.1.7: *Radio toggle***

Implementirati program *Radio toggle* sa rasporedom komponenti za jedinstveni odabir kao na Slici 2.7a. Prilikom odabira neke od opcija, tekst labela se ažurira tako da prikaže odabranu opciju, kao na slici 2.7.

Two screenshots of a Radio toggle application. Screenshot (a) shows the initial state with three unselected radio options (A, B, C) and a 'Show Selected' button. Below the button, it says 'No selection'. Screenshot (b) shows the state after clicking 'Show Selected', where option B is now selected, and the text below the button reads 'Selected: Option B'.

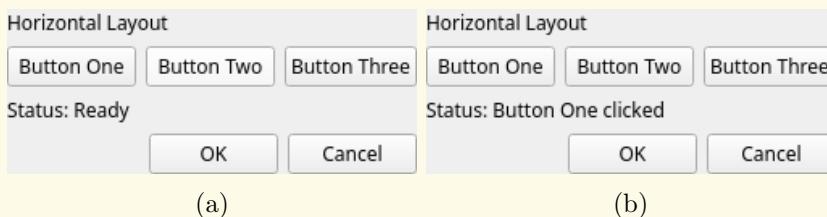
(a)

(b)

Slika 2.7: Aplikacija *Radio toggle*

Zadatak 2.1.8: *Horizontal layout*

Implementirati program *Horizontal layout* sa horizontalnim rasporedom komponenti kao na Slici 2.8. Klikom na bilo koje od tri dugmeta se ažurira tekst labela po uzoru na Sliku 2.8b. Klikom na dugme OK, aplikacija uspešno završava izvršavanje, dok klikom na dugme Cancel aplikacija neuspešno završava izvršavanje.



Slika 2.8: Aplikacija *Horizontal layout*

Zadatak 2.1.9: *Grid layout*

Implementirati program *Grid layout* sa rasporedom komponenti kao na Slici 2.9. Klikom na bilo koje dugme se ispisuje identifikator dugmeta na standardni izlaz kao u primeru ispod.



Slika 2.9: Aplikacija *Grid layout*

Primer izlaza, u slučaju gde su dugmad kliknuta redom od broja 1 do broja 5:

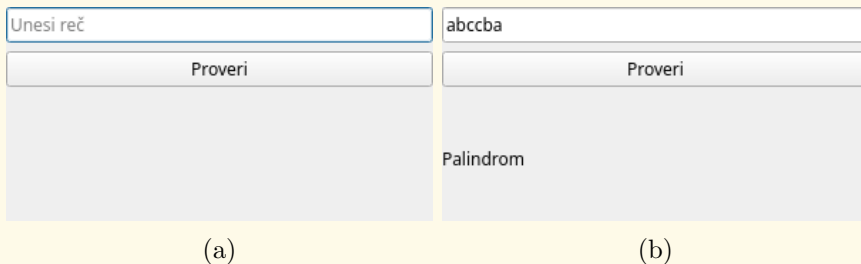
Primer 1

```
IzLAZ: 1
        2
        3
        4
        5
```

2.2 Interaktivne komponente

Zadatak 2.2.1: *Palindrom*

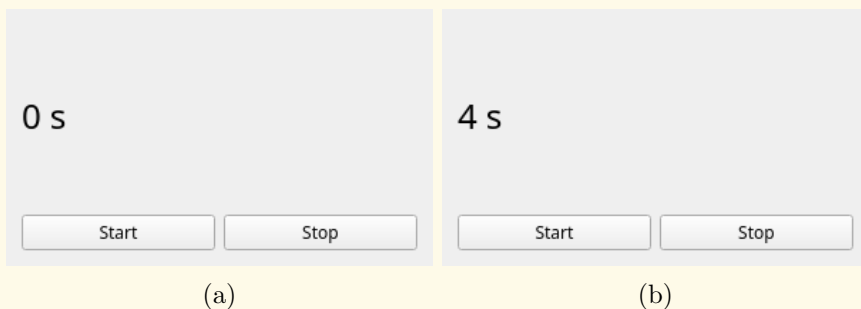
Implementirati aplikaciju sa rasporedom komponenti i izgledom kao na Slici 2.10a, koja testira da li je uneta reč palindrom klikom na dugme **Proveri**. Prikazati rezultat po uzoru na Sliku 2.10b.



Slika 2.10: Aplikacija *Palindrom*

Zadatak 2.2.2: *Štoperica*

Implementirati štopericu sa rasporedom komponenti i izgledom kao na Slici 2.11a. Rezultat prikazati po uzoru na sliku 2.11b.



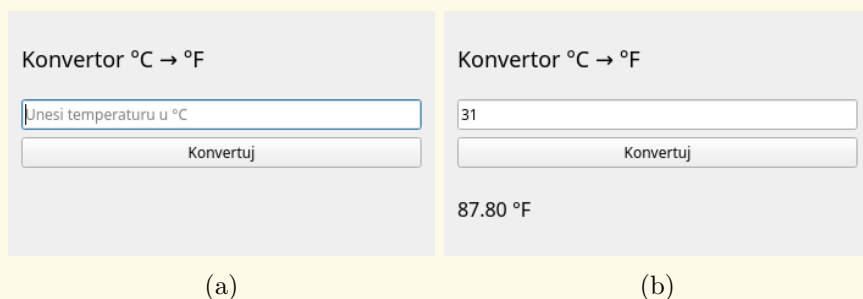
Slika 2.11: Aplikacija *Štoperica*

Zadatak 2.2.3: Konverter temperature

Implementirati konverter temperature sa rasporedom komponenti i izgledom kao na Slici 2.12a. Temperatura se unosi u stepenima Celzija i konvertuje u stepene Farenhajta klikom na dugme Konvertuj. Rezultat prikazati po uzoru na Sliku 2.12b.

Formula za konverziju:

$$F = C \cdot \frac{9}{5} + 32$$



Slika 2.12: Aplikacija *Konverter temperature*

Zadatak 2.2.4: Brojač klikova

Implementirati aplikaciju sa rasporedom komponenti i izgledom kao na Slici 2.13a. Svaki klik dugmeta **Klikni me** povećava brojač ispisan u labeli iznad dugmeta. Posle 20 klikova dugme postaje onemogućeno i tekst **Dovoljno!** se ispisuje u labeli ispod dugmeta (Slika 2.13b).



Slika 2.13: Aplikacija *Brojač klikova*

Zadatak 2.2.5: Generator lozinki

Implementirati generator lozinki sa rasporedom komponenti i izgledom kao na Slici 2.14a. Klikom na dugme **Generate Password** se ispisuje nasumično generisana lozinka u labeli ispod dugmeta po uzoru na Sliku 2.14b.

Two screenshots of a password generator interface. Screenshot (a) shows the form with all checkboxes selected and an empty output field. Screenshot (b) shows the same form with the output field containing the password 'itOxj7h83cQm'.

(a) (b)

Slika 2.14: Aplikacija *Generator lozinki***Zadatak 2.2.6: Analizator teksta**

Implementirati aplikaciju sa rasporedom komponenti i izgledom kao na Slici 2.15a, koja broji karaktere i reči u unetom tekstu i ispisuje rezultat po uzoru na Sliku 2.15b. Aplikacija treba da podržava bogati unos teksta (*rich text*), uključujući masna (*bold*), kurziv (*italic*), i podvučena slova (*underline*).

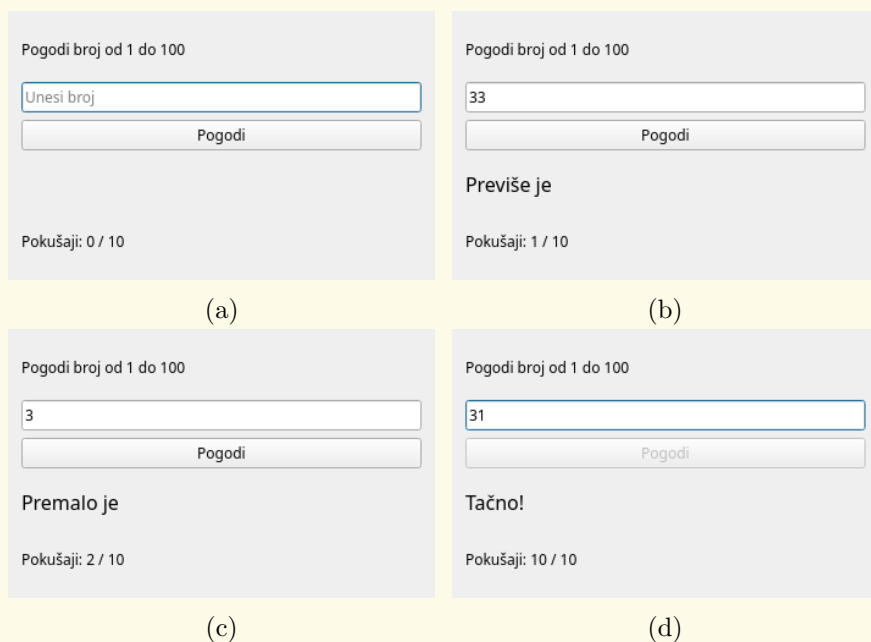
Two screenshots of a text analyzer interface. Screenshot (a) shows an empty text input area and statistics: Karakteri: 0, Reči: 0. Screenshot (b) shows the same interface with sample text entered and statistics: Karakteri: 445, Reči: 69.

(a) (b)

Slika 2.15: Aplikacija *Analizator teksta*

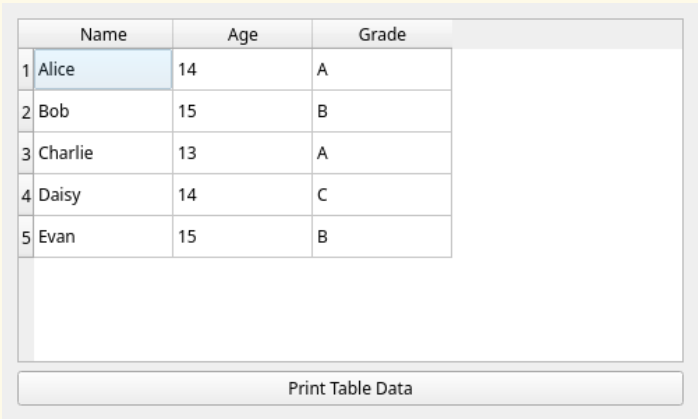
Zadatak 2.2.7: Igra „Pogodi broj”

Implementirati igru *Pogodi broj* sa rasporedom komponenti i izgledom kao na Slici 2.16a. Aplikacija određuje nasumično jedan broj u intervalu $[1, 100]$ kao zamišljeni broj. Korisnik unosi broj u polje za unos i klikom na dugme **Pogodi** se proverava da li je uneti broj jednak zamišljenom. Pri svakom pokušaju, aplikacija odgovara sa **Previše je** (Slika 2.16b), **Premalo je** (Slika 2.16c), ili **Tačno!** (Slika 2.16d) u labeli ispod dugmeta. Igra prestaje ukoliko korisnik pogodi broj ili ukoliko iskoristi 10 pokušaja bez pogotka. Broj neuspelih pokušaja se ispisuje u labeli na dnu aplikacije.

Slika 2.16: Igra *Pogodi broj*

Zadatak 2.2.8: *Tabelarni editor*

Implementirati aplikaciju za tabelarni unos podataka o studentima sa izgledom kao na Slici 2.17. Klikom na dugme **Print Table Data** se ispisuje stanje tabele na standardni izlaz u formatu *CSV*. Omogućiti unos i izmenu podataka u realnom vremenu.



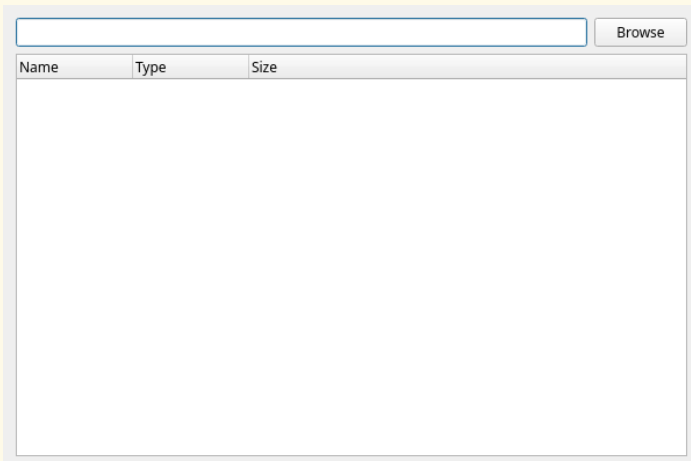
Slika 2.17: Aplikacija *Tabelarni editor*

Primer 1

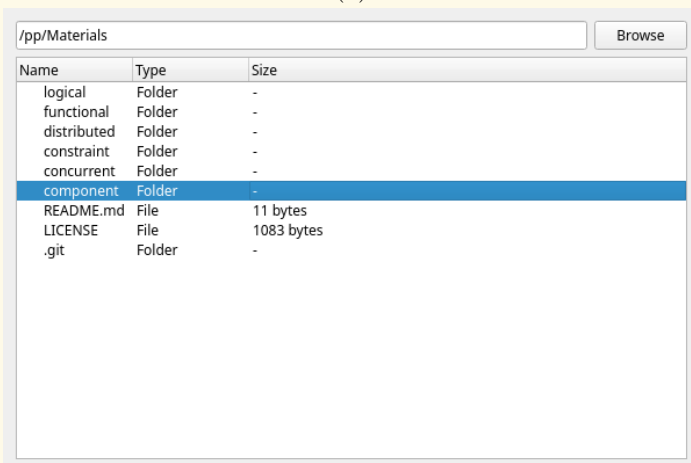
```
IZLAZ: Name, Age, Grade
'Alice', '14', 'A'
'Bob', '15', 'B'
'Charlie', '13', 'A'
'Daisy', '14', 'C'
'Evan', '15', 'B'
```

Zadatak 2.2.9: Navigator fajlsistema

Implementirati aplikaciju za navigaciju fajlsistema sa izgledom kao na Slici 2.18a. Klikom na dugme **Browse** se otvara dijalog za odabir početnog direktorijuma. Nakon odabira, sadržaj odabranog direktorijuma se prikazuje u glavnom polju za prikaz po uzoru na Sliku 2.18b.



(a)

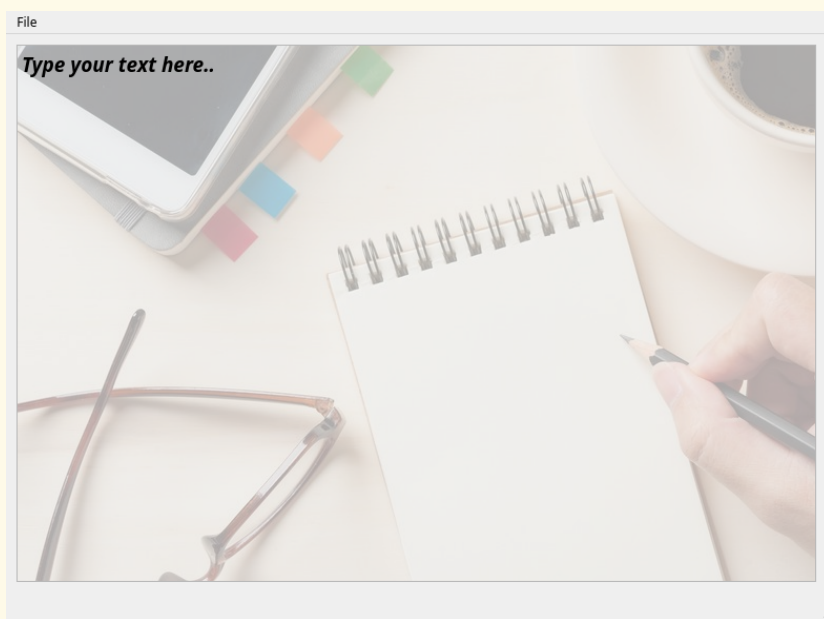


(b)

Slika 2.18: Aplikacija *Navigator fajlsistema*

Zadatak 2.2.10: „Simple Pad” editor

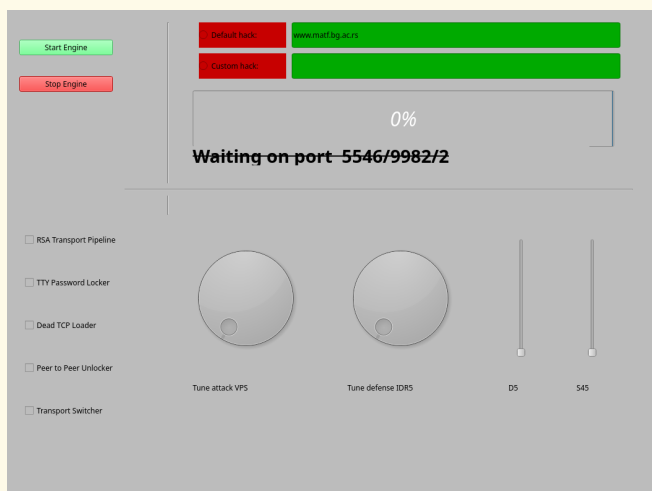
Napisati program koji implementira tekstualni editor sa nazivom *SimplePad*, sa rasporedom komponenti i izgledom kao na Slici 2.19. *SimplePad* treba da podržava bogati unos teksta (*rich text*), uključujući masna (*bold*), kurziv (*italic*), i podvučena slova (*underline*). *SimplePad* treba da pruži osnovne komande (kreiranja, otvaranja i čuvanja dokumenata), kao i opcije kopiranja (**Copy**), isecanja (**Cut**), i lepljenja (**Paste**) delova teksta. U padajućem meniju **File** se nalaze osnovne komande, dok se prečice **Ctrl+C**, **Ctrl+X**, i **Ctrl+V** koriste za odabir opcija kopiranja, isecanja, i lepljenja teksta, redom.



Slika 2.19: Aplikacija *SimplePad*

Zadatak 2.2.11: *HackSim*

Napisati aplikaciju *HackSim* koja simulira hakerske napade, sa izgledom kao na Slici 2.20. Aplikacija klikom na dugme **Start Engine** ažurira progres bar (od 0 do 100%), svake sekunde napredujući za po jedan procenat. Dodatno, aplikacija podržava odabir dodatnih opcija, kao i podešavanje kontinualnih parametara.



(a)



(b)

Slika 2.20: Aplikacija *HackSim*

Zadatak 2.2.12: „*TODO*” aplikacija

Implementirati aplikaciju *ToDoApp* za održavanje liste zadataka sa izgledom kao na Slici 2.21a. Klikom na dugme Add se u listu dodaje zadatak sa opisom unetim u polje za unos teksta (Slika 2.21b). Aplikacija prikazuje sve zadatke u glavnom polju za prikaz. Klikom na dugme Delete Selected se briše odabrani zadatak iz liste zadataka (Slika 2.21c prikazuje odabir stavke za brisanje, dok Slika 2.21d prikazuje stanje nakon brisanja odabrane stavke). Aplikacija čuva listu zadataka u fajlu `tasks.json`, u formatu *JSON*.



Slika 2.21: Aplikacija *ToDoApp*

Zadatak 2.2.13: Kalkulator

Implementirati kalkulator sa rasporedom komponenti i izgledom kao na Slici 2.22a. Kalkulator podržava osnovne aritmetičke operacije sabiranja, oduzimanja, množenja i deljenja (Slika 2.22b). Klikom na **Clear** dugme se resetuje stanje kalkulatora.

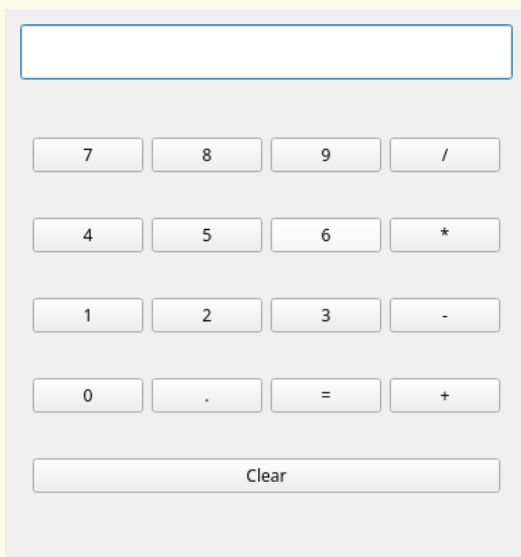


Figure 2.22a shows the initial state of the calculator application. It features a large rectangular display area at the top, which is currently empty. Below the display is a grid of buttons arranged in four rows and four columns. The first row contains buttons for digits 7, 8, 9, and a forward slash (/). The second row contains buttons for digits 4, 5, 6, and an asterisk (*). The third row contains buttons for digits 1, 2, 3, and a minus sign (-). The fourth row contains buttons for digits 0, a decimal point (.), an equals sign (=), and a plus sign (+). Below this grid is a single, wider button labeled "Clear".

(a)



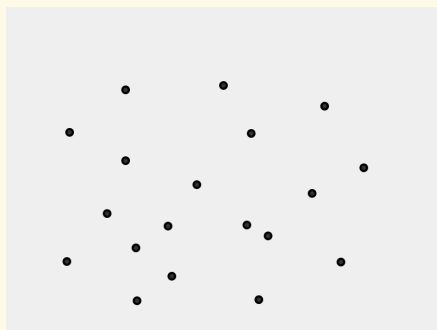
Figure 2.22b shows the calculator application after performing a calculation. The display area now contains the text "3.0*6.2/9". The button layout remains identical to the one in Figure 2.22a, with buttons for digits 0-9, arithmetic operators, and a "Clear" button.

(b)

Slika 2.22: Aplikacija *Kalkulator*

Zadatak 2.2.14: Igra „Bullet Hole”

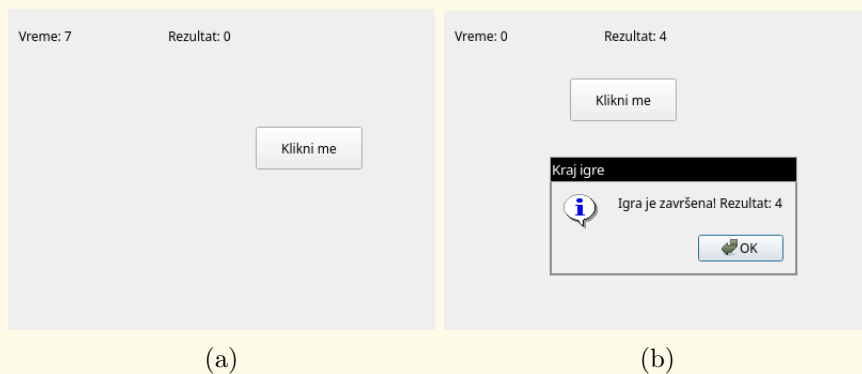
Implementirati igru *BulletHole*^a sa izgledom kao na Slici 2.23, koja omogućava igraču da, klikom na površinu unutar aplikacije, ispaljuje *metke*. Svaki klik kreira *metak* simuliran slikom rupe metka učitanoj iz vektorskog fajla `hole.svg` sa diska.

Slika 2.23: Igra *BulletHole*

^aIgra *BulletHole* je inspirisana popularnom igrom *Desktop Destroyer*.

Zadatak 2.2.15: Igra „Brzi refleksi”

Implementirati igru *Brzi refleksi* sa rasporedom komponenti i izgledom kao na Slici 2.24a. Igra prikazuje dugme koje menja svoju poziciju svakih 0.5 sekundi. Korisnik treba da klikne dugme pre nego što ono promeni poziciju. Finalni skor se računa kao broj uspešnih klikova u 10 sekundi (Slika 2.24b).

Slika 2.24: Igra *Brzi refleksi*

Zadatak 2.2.16: Igra „Vešala”

Napisati jednostavnu desktop aplikaciju koja predstavlja najjednostavniju verziju igre „Vešala” (eng. *Hangman*). Aplikacija treba da zadovolji sledeće uslove:

1. Nasumičan izbor reči

Program treba da ima unapred definisanu listu reči, od koje se pri pokretanju nasumično bira jedna reč koja se pogađa.

2. Prikaz reči

Nepogođena slova treba da budu prikazana kao donje crte (-), a pogođena slova se u njima zamenjuju odgovarajućim slovima.
s

3. Unos slova

Korisnik unosi jedno slovo u polje za unos i potvrđuje pritiskom na dugme „Pogodi”.

4. Provera pogodaka

Ako uneseno slovo postoji u reči, program ažurira prikaz reči. Ako slovo ne postoji, brojač grešaka se povećava.

5. Ograničen broj grešaka

Dozvoljeno je najviše 6 pogrešnih pokušaja. Nakon šest grešaka, igra se završava porukom o porazu i prikazuje se tačna reč.

6. Pobeda

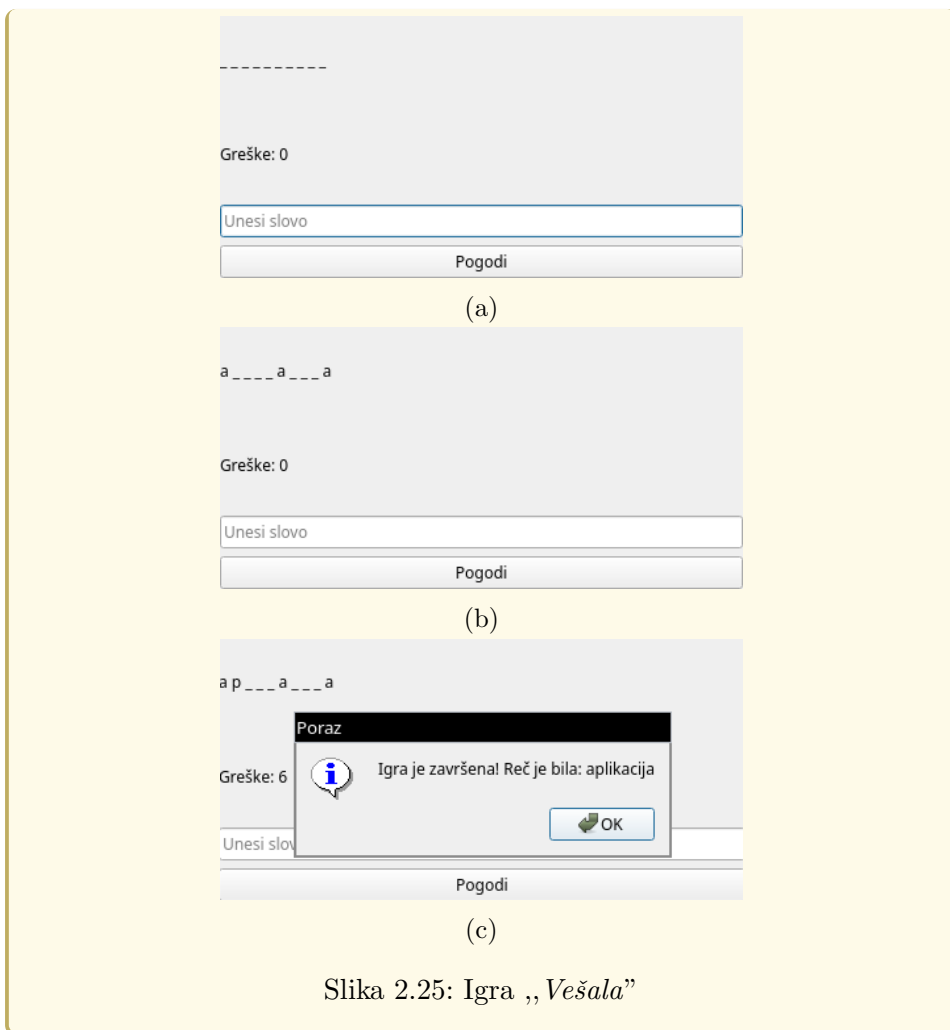
Kada su sva slova pogođena, program prikazuje poruku o pobedi.

7. Jednostavan interfejs

Interfejs treba da sadrži:

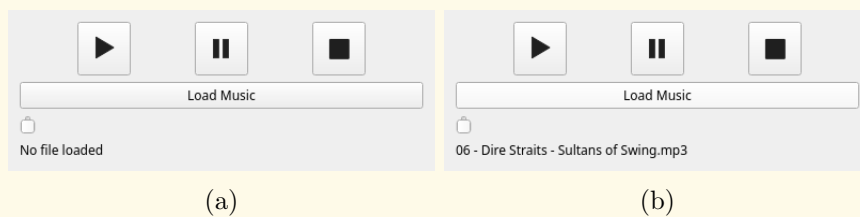
- prikaz trenutnog stanja reči
- prikaz broja pogrešnih pokušaja
- polje za unos slova
- dugme „Pogodi”

Primer interfejsa je prikazan na Slici 2.25.



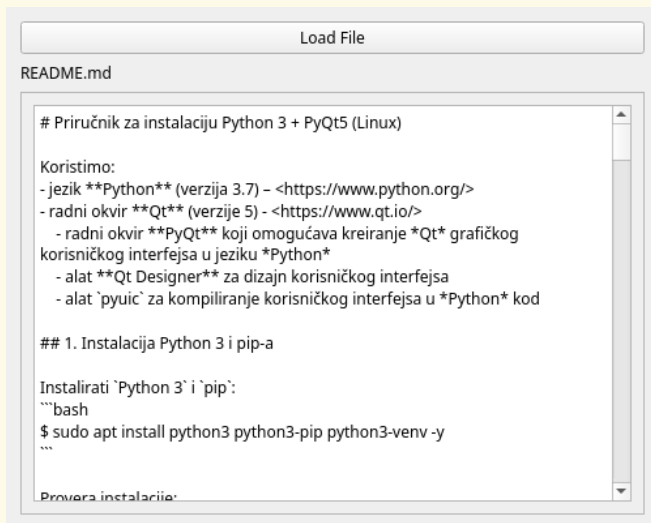
Zadatak 2.2.17: Muzički plejer

Implementirati muzički plejer sa rasporedom komponenti i izgledom kao na Slici 2.26.



Zadatak 2.2.18: Pregledač tekstualnih fajlova i slika

Implementirati pregledač tekstualnih fajlova i slika sa rasporedom komponenti i izgledom kao na Slici 2.27. Pregledač prikazuje fajlove u polju za prikaz. Tekstualni fajlovi (Slika 2.27a) se prikazuju tako što se njihov sadržaj ispiše u polje za prikaz. Slike (Slika 2.27b) se prikazuju unutar polja za prikaz.



(a)



(b)

Slika 2.27: Pregledač tekstualnih fajlova i slika

Zadatak 2.2.19: *Kriptoaritmetika u tabeli*

Implementirati aplikaciju koja pronalazi sva rešenja jednačine:

$$SEE + YOU = SOON.$$

Različitim slovima odgovaraju različite cifre, a brojevi SEE, YOU i SOON ne smeju početi cifrom 0.

Grafički interfejs aplikacije treba da izgleda kao na Slici 2.28. Klikom na dugme **Pronadji sva resenja**, aplikacija treba da pokrene zadnji deo koji rešava kriptoaritmetički problem. Rešenja prikazati u tabeli tako da kolone sadrže vrednosti slova S, E, Y, O, U, N i odgovarajuću numeričku jednačinu. Interfejs prikazuje i ukupan broj rešenja ispod polja za prikaz rezultata.

Kriptoaritmetika: SEE + YOU = SOON

Pronadji sva resenja

S	E	Y	O	U	N	Jednacina
1	9	8	0	3	2	199 + 803 = 1002
1	9	8	0	4	3	199 + 804 = 1003
1	9	8	0	5	4	199 + 805 = 1004
1	9	8	0	6	5	199 + 806 = 1005
1	9	8	0	7	6	199 + 807 = 1006

Broj resenja: 5

Slika 2.28: Aplikacija *Kriptoaritmetika u tabeli*

Zadatak 2.2.20: Optimizator ranca

Implementirati aplikaciju koja rešava ograničeni problem ranca. Grafički interfejs aplikacije treba da izgleda kao na Slici 2.29. Korisnik unosi kapacitet ranca i u tabeli uređuje predmete. Za svaki predmet zadaju se naziv, broj mesta koji zauzima, vrednost i najveći dozvoljeni broj primeraka.

Zadnji deo aplikacije treba da odredi koliko primeraka svakog predmeta treba poneti tako da ukupno zauzeće ne premaši kapacitet ranca, a ukupna vrednost bude maksimalna. Rezultat prikazati u dodatnoj koloni tabele i u labeli koja ispisuje ukupnu vrednost i iskorišćen kapacitet.

Optimizator ranca

Kapacitet: 

Naziv	Mesta	Vrednost	Najviše	U rancu
Viski	4	15	5	1
Parfem	3	10	5	1
Casa	2	7	5	1
Knjiga	1	2	10	0

Vrednost: 32, zauzeto mesta: 9

Slika 2.29: Aplikacija *Optimizator ranca*

Zadatak 2.2.21: Vizuelizacija problema N dama

Implementirati aplikaciju koja rešava problem N dama za tablu dimenzije $N \times N$, gde korisnik bira vrednost N iz intervala $[4, 10]$. Dve dame se napadaju ako se nalaze u istoj vrsti, istoj koloni ili istoj dijagonali.

Grafički interfejs aplikacije treba da izgleda kao na Slici 2.30. Zadnji deo aplikacije treba da pronade sve rasporede dama tako da se nikoje dve dame ne napadaju. Interfejs treba da sadrži tabelu koja prikazuje jedan raspored, dugmad za prelazak na prethodno i sledeće rešenje i labelu koja prikazuje redni broj trenutno prikazanog rešenja. Interfejs prikazuje i ukupan broj rešenja ispod polja za prikaz rezultata.

Vizuelizacija problema N dama

Dimenzija: 

D							
				D			
							D
					D		
		D					
						D	
	D						
			D				

Resenje 1/92

Slika 2.30: Aplikacija za vizuelizaciju problema N dama

2.3 Ispitni zadaci

Zadatak 2.3.1: *MATF kalkulator*

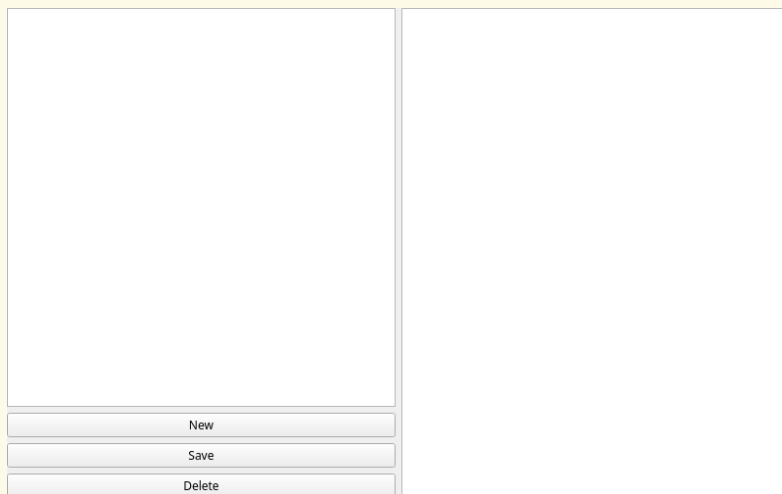
Implementirati aplikaciju *Matf Calculator* sa rasporedom komponenti i izgledom kao na Slici 2.31. *Matf Kalkulator* podržava osnovne aritmetičke operacije sabiranja, oduzimanja, množenja i deljenja. Implementirati brisanje unetih brojeva (*back* dugme) kao i resetovanje rada kalkulatora klikom na odgovarajuće dugme.



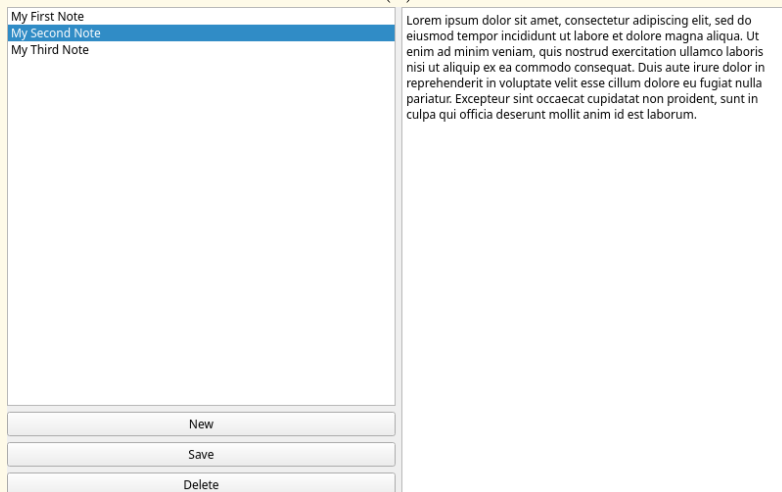
Slika 2.31: Aplikacija *MATF Calculator*

Zadatak 2.3.2: Beleške

Implementirati aplikaciju za čuvanje beleški sa izgledom kao na Slici 2.32. Aplikacija čuva beleške u memoriji. Dugme **Add** dodaje novu belešku u spisak beleški, sa imenom koje se unosi u iskaćući dijalog za unos teksta. Dugme **Save** ažurira tekst odabrane beleške (u protivnom, izmene trenutne beleške se gube nakon odabira druge beleške iz spiska). Dugme **Delete** briše odabranu belešku iz spiska.



(a)

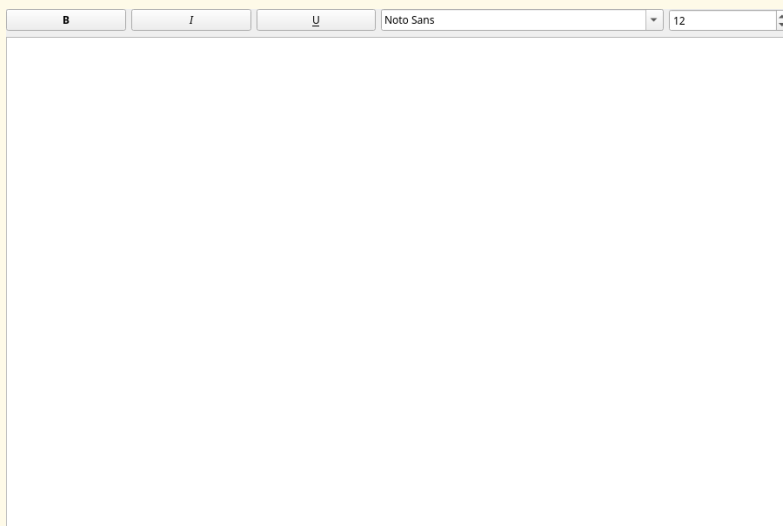


(b)

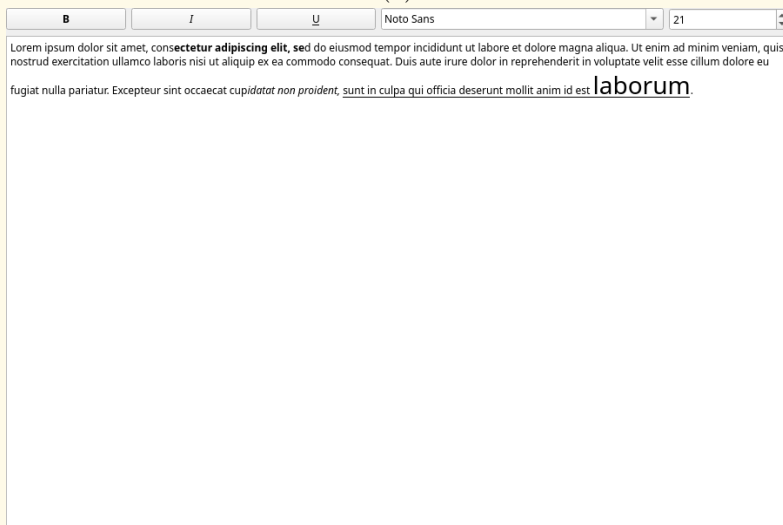
Slika 2.32: Aplikacija za čuvanje beleški

Zadatak 2.3.3: Formatiranje teksta

Implementirati aplikaciju za formatiranje teksta sa izgledom kao na Slici 2.33. Omogućiti podebljanje, italicizovanje, podvlačenje i promenu fonta odabranog raspona teksta.



(a)



(b)

Slika 2.33: Aplikacija za formatiranje teksta

Zadatak 2.3.4: Prijava

Implementirati aplikaciju za prijavu korisnika sa izgledom kao na Slici 2.34. Pretpostaviti da je ispravno korisničko ime `admin` i lozinka `1234`. Prilikom pokušaja prijave, u zavisnosti od unetih podataka, ispisati odgovarajući tekst kao u primeru ispod.

Login

Login

(a)

Login

Login

Invalid username or password

(b)

Login

Login

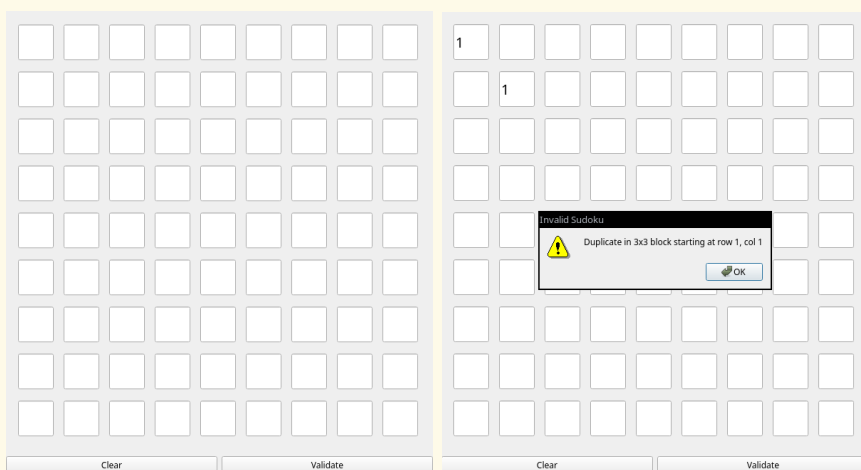
Login successful

(c)

Slika 2.34: Aplikacija za prijavu

Zadatak 2.3.5: Igra „Sudoku”

Implementirati aplikaciju za igru *Sudoku* sa izgledom kao na Slici 2.35. Cilj igre *Sudoku* je popuniti kvadratnu mrežu 9×9 brojevima od 1 do 9 tako da svaki red, svaka kolona i svaka 3×3 pod-mreža (blok) sadrže sve brojeve od 1 do 9, bez ponavljanja. Prilikom pokretanja aplikacije, sva polja su prazna. Igra je završena kada je mreža potpuno popunjena i kada su sva pravila ispoštovana. Dugme **Clear** čisti unete brojeve iz svih ćelija, dok dugme **Validate** proverava da li uneti brojevi zadovoljavaju *Sudoku* pravila, i vraća odgovarajuću poruku u iskaćućem prozoru.



(a)

(b)

Slika 2.35: Igra *Sudoku*

Zadatak 2.3.6: MATF Generator brojeva

U Septembru 2003. godine razvojni tim Matematičkog fakulteta razvio je paket aplikacija koji je podržavao intenzivna numerička izračunavanja u programskim jezicima C i Fortran. Najprodavanija aplikacija računala je (najbrže do tada!) Fibonačijeve i Markus-Lukasove brojeve.

Fibonačijevi brojevi se računaju po formuli:

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}$$

Markus-Lukasovi brojevi se računaju po formuli ($c_0, c_1, a, b \in \mathbb{R}$, $m, l \in \mathbb{N}$):

$$L_0 = c_0, L_1 = c_1, L_n = a^m L_{n-1} + b^l L_{n-2}$$

Napisati program koji računa n -ti Fibonačijev ili Markus-Lukasov broj u zavisnosti od izabrane opcije, po uzoru na Sliku 2.36. U slučaju računanja Markus-Lukasovog broja, potrebno je učitati i odgovarajući konfiguracioni JSON fajl.

Grafički interfejs se sastoji od:

- labela sa nazivom programa,
- radio dugmadi pomoću kojih se selektuje algoritam,
- dela za učitavanje parametra n i konfiguracionog fajla u slučaju odabira Kvazi-Lukasovih brojeva,
- dugmeta koje pokreće računanje i
- polja za ispis rezultata

Primer 1

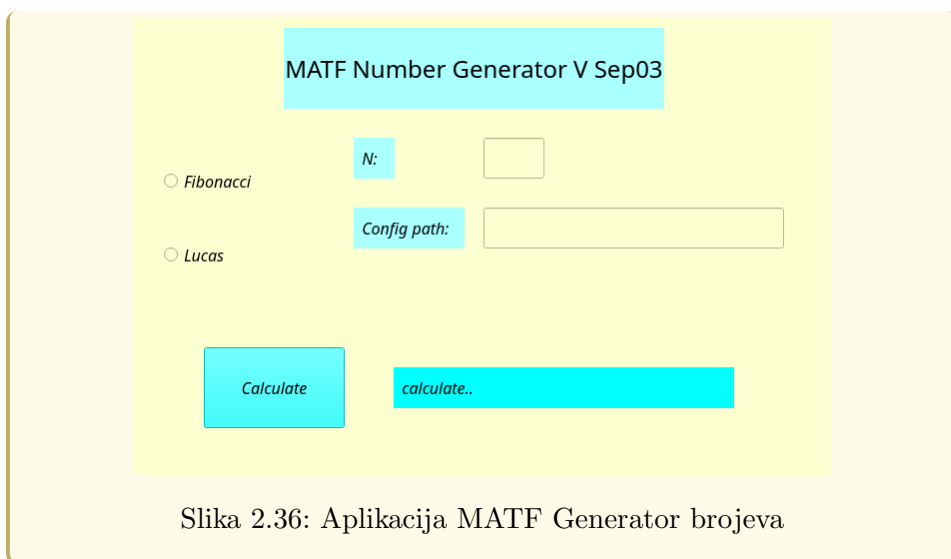
```
UŁAZNA DATOTEKA config_lucas.json:
{"c0":0.63, "c1":-5,60, "a":1.23, "b":1.88, "m":2, "l":u1}
```

Primer 2

```
POZIV: python 1.py F 50
IZLAZ: 7778742049
```

Primer 3

```
POZIV: python 1.py L 7
IZLAZ: 32672.409281
```



Zadatak 2.3.7: Kriptoaritmetika u tabeli

Implementirati aplikaciju koja pronalazi sva rešenja jednačine:

$$MEN * AND = WOMEN$$

Različitim slovima odgovaraju različite cifre, a brojevi MEN, AND i WOMEN ne smeju početi cifrom 0.

Grafički interfejs aplikacije treba da izgleda kao na Slici 2.37. Klikom na dugme **Pronadji sva resenja**, aplikacija treba da pokrene zadnji deo koji rešava kriptoaritmetički problem. Rešenja prikazati u tabeli tako da kolone sadrže vrednosti slova M, E, N, A, D, W, O i odgovarajuću numeričku jednačinu. Interfejs prikazuje i ukupan broj rešenja ispod polja za prikaz rezultata.

Kriptoaritmetika: MEN * AND = WOMEN

Pronadji sva resenja

M	E	N	A	D	W	O	Jednacina
1	2	5	7	3	9	4	125 + 753 = 94125
2	6	0	3	1	7	8	260 + 301 = 78260
2	9	0	3	1	8	7	290 + 301 = 87290
3	4	0	2	1	6	8	340 + 201 = 68340
3	8	0	2	1	7	6	380 + 201 = 76380
3	9	0	2	1	7	8	390 + 201 = 78390
4	3	0	2	1	8	6	430 + 201 = 86430
4	7	2	1	6	5	9	472 + 126 = 59472
4	8	0	2	1	9	6	480 + 201 = 96480

Broj resenja: 11

Slika 2.37: Aplikacija *Kriptoaritmetika u tabeli*

Zadatak 2.3.8: Razmena novca

Implementirati aplikaciju koja pronalazi sve načine da se zadati iznos predstavi novčićima od 1, 2, 5, 10 i 20 dinara. Korisnik unosi iznos pomoću numeričkog polja, a klikom na dugme **Pronadji kombinacije** aplikacija prikazuje sve kombinacije u tabeli.

Grafički interfejs aplikacije treba da izgleda kao na Slici 2.38 i treba da prikaže broj pronađenih rešenja i po jedan red tabele za svako rešenje.

Razmena novca ogranicenjima

Iznos:

1 din	2 din	5 din	10 din	20 din	Ukupno
0	0	0	1	2	50
0	0	0	3	1	50
0	0	0	5	0	50
0	0	2	0	2	50
0	0	2	2	1	50
0	0	2	4	0	50
0	0	4	1	1	50
0	0	4	3	0	50
0	0	6	0	1	50
0	0	6	0	0	50

Broj resenja: 450

Slika 2.38: Aplikacija *Razmena novca*

Zadatak 2.3.9: Magična sekvenca

Implementirati aplikaciju koja za unetu dužinu n pronalazi sve magične sekvence dužine n . Sekvenca $S = (x_0, x_1, \dots, x_{n-1})$ je magična ako se broj i pojavljuje tačno x_i puta u sekvenci, za svako $i \in \{0, 1, \dots, n-1\}$.

Grafički interfejs aplikacije treba da izgleda kao na Slici 2.39. Korišnik unosi broj n pomoću komponente za numerički unos. Klikom na dugme **Pronadji**, zadnji deo aplikacije rešava problem traženja magičnih sekvenci i vraća sva rešenja. Interfejs treba da prikaže rešenja u polju za prikaz i poruku ukoliko magična sekvenca date dužine ne postoji. Interfejs prikazuje i ukupan broj rešenja ispod polja za prikaz rezultata.

Generator magicnih sekvenci

Duzina:

[2, 1, 2, 0, 0]

Broj resenja: 1

Slika 2.39: Aplikacija *Magična sekvenca*

Zadatak 2.3.10: *Sudoku* rešavač

Implementirati aplikaciju za rešavanje *Sudoku* zagonetke. Grafički interfejs aplikacije treba da izgleda kao na Slici 2.40. Interfejs treba da sadrži tabelu dimenzija 9×9 u koju korisnik unosi početnu tablu. Prazna polja se mogu ostaviti nepopunjena. Klikom na dugme **Resi**, aplikacija popunjava tabelu jednim ispravnim rešenjem. Interfejs treba da omogući učitavanje unapred zadatog primera i brisanje cele table.

Zadnji deo aplikacije treba da proveri da li početna tabla krši *Sudoku* ograničenja i da pronađe rešenje u kome svaki red, svaka kolona i svaki blok dimenzija 3×3 sadrže cifre od 1 do 9 bez ponavljanja.

Sudoku resavac ogranicenjima

6	3	8	7					
	2	7				5		
5	4		6		9	2		
	1		5	9		3		
		9		7		4		
		6		2	4		1	
		5	3		1		2	7
		2				8	3	
					7	9	4	5

Učitaj primer

Resi

Ocisti

Ucitan je primer.

Slika 2.40: Aplikacija *Sudoku* rešavač

Zadatak 2.3.11: Plan obuka zaposlenih

Implementirati aplikaciju koja rešava optimizacioni problem planiranja obuka. Kompanija može poslati radnike na kurs *Elixir* ili *Dart*. Kurs *Elixir* košta 100 evra po osobi, daje 150 projekat-sati mesečno i donosi 5 evra po satu. Kurs *Dart* košta 105 evra po osobi, daje 170 projekat-sati mesečno i donosi 6 evra po satu.

Grafički interfejs aplikacije treba da izgleda kao na Slici 2.41. Korisnik unosi budžet, najveći dozvoljeni broj projekat-sati i ukupan broj radnika koji se obučava. Zadnji deo aplikacije treba da pronađe broj radnika za svaki kurs tako da neto dobit nakon troška obuke bude maksimalna. Interfejs treba da prikaže broj radnika po kursu, neto dobit, potrošen budžet i iskorišćene projekat-sate.

Plan obuka zaposlenih

Budžet:

26000

Projektni sati:

51200

Broj radnika:

250

Izracunaj plan

Kurs	Cena	Sati	Dobit/sat	Broj radnika
Elixir	100	150	5	50
Dart	105	170	6	200

Dobit: 215500, trosak: 26000, projektni sati: 41500

Slika 2.41: Aplikacija *Plan obuka zaposlenih*

Zadatak 2.3.12: Magični kvadrat 3×3

Implementirati aplikaciju koja pronalazi sve magične kvadrate dimenzija 3×3 . Potrebno je rasporediti brojeve od 1 do 9 tako da su svi različiti i da zbir svake vrste, svake kolone i obe dijagonale bude 15.

Grafički interfejs aplikacije treba da izgleda kao na Slici 2.42. Zadnji deo aplikacije treba da pronađe sva rešenja. Interfejs treba da prikaže jedno rešenje u tabeli dimenzija 3×3 i da omogući prelazak na prethodno i sledeće rešenje.

Magični kvadrat 3x3

2	7	6
9	5	1
4	3	8

PronadjiPrethodnoSledece

Resenje 1/8

Slika 2.42: Aplikacija *Magični kvadrat 3×3*

Zadatak 2.3.13: *Bojenje mape*

Implementirati aplikaciju koja rešava problem bojenja mape. Mapa ima oblasti Sever, Zapad, Centar, Istok, Jug i Obala. Susjedne oblasti ne smeju imati istu boju. Parovi susednih oblasti su:

Sever-Zapad, Sever-Centar, Sever-Istok, Zapad-Centar, Zapad-Jug, Centar-Istok, Centar-Jug, Centar-Obala, Istok-Obala, Jug-Obala.

Grafički interfejs aplikacije treba da izgleda kao na Slici 2.43. Korisnik bira broj dostupnih boja. Zadnji deo aplikacije treba da pronade sva validna bojenja, a interfejs treba da prikaže svako bojenje u tabeli i ukupan broj pronađenih bojenja.

Bojenje mape ogracenjima

Broj boja:

Sever	Zapad	Centar	Istok	Jug	Obala
Plava	Zelena	Crvena	Zelena	Plava	Zuta
Plava	Zelena	Crvena	Zelena	Zuta	Plava
Plava	Zelena	Crvena	Zuta	Plava	Zelena
Plava	Zelena	Crvena	Zuta	Zuta	Plava
Plava	Zelena	Crvena	Zuta	Zuta	Zelena
Plava	Zuta	Crvena	Zelena	Plava	Zuta
Plava	Zuta	Crvena	Zelena	Zelena	Plava
Plava	Zuta	Crvena	Zelena	Zelena	Zuta
Plava	Zuta	Crvena	Zuta	Plava	Zelena

Broj bojenja: 120

Slika 2.43: Aplikacija *Bojenje mape*

Zadatak 2.3.14: Raspored časova

Implementirati aplikaciju koja pravi raspored časova koristeći ograničenja. Grafički interfejs aplikacije treba da izgleda kao na Slici 2.44. Korisnik u tabelu unosi časove, pri čemu svaki red sadrži naziv predmeta, nastavnika i grupu. Aplikacija raspoređuje časove u termine:

Pon 09:00, Pon 11:00, Uto 09:00, Uto 11:00

i sale:

A1, A2

Zadnji deo aplikacije treba da pronađe raspored u kome se u istom terminu ne koriste ista sala, isti nastavnik ili ista grupa. Interfejs treba da omogući učitavanje primera, pokretanje rešavanja i prikaz rasporeda u tabeli sa kolonama za predmet, nastavnika, grupu, termin i salu.

Raspored casova ogranicenjima

Ulazni casovi

Predmet	Nastavnik	Grupa
Matematika	Ana	G1
Programiranje	Boris	G1
Fizika	Ana	G2
Baze	Ceca	G1
Algoritmi	Boris	G2

Ucitaj primer

Napravi raspored

Raspored

Predmet	Nastavnik	Grupa	Termin	Sala
Matematika	Ana	G1	Pon 09:00	A1
Programiranje	Boris	G1	Pon 11:00	A1
Fizika	Ana	G2	Pon 11:00	A2
Baze	Ceca	G1	Uto 09:00	A1
Algoritmi	Boris	G2	Pon 09:00	A2

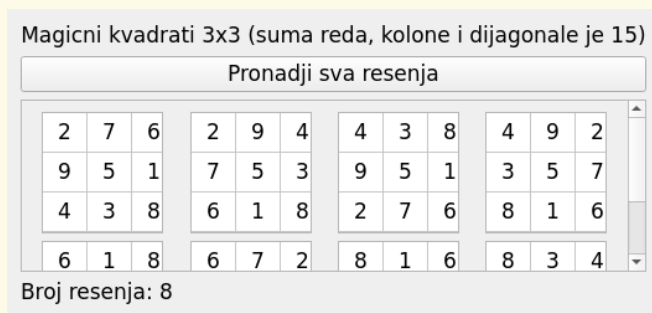
Raspored je pronadjen.

Slika 2.44: Aplikacija *Raspored časova*

Zadatak 2.3.15

Implementirati aplikaciju koja pronalazi sve magične kvadrate dimenzija 3×3 . Magičan kvadrat je raspored brojeva od 1 do 9, pri čemu se svaki broj koristi tačno jednom, takav da je zbir brojeva u svakoj vrsti, svakoj koloni i obema dijagonalama jednak 15.

1. Napisati funkciju `magicni_kvadrati` koja modeluje opisani problem kao problem programiranja ograničenja i vraća sva njegova rešenja. Svaki magični kvadrat predstaviti kao matricu dimenzija 3×3 , odnosno listu od tri vrste, pri čemu je svaka vrsta lista od tri broja.
2. Implementirati grafički deo aplikacije (biblioteka `PyQt5`). Interfejs treba da sadrži naslov i dugme za pokretanje rešavanja. Nakon pritiska na dugme, prikaz aplikacije treba da bude kao na Slici 2.45: za svako rešenje se prikazuje zaseban kvadrat dimenzija 3×3 sa vrednostima polja. Pošto pronađenih rešenja može biti više nego što staje u prozor, kvadrati se prikazuju u oblasti sa klizačem za vertikalno pomeranje (kao na slici). Ispod kvadrata se prikazuje labela sa ukupnim brojem pronađenih rešenja.



Slika 2.45: Aplikacija *Magični kvadrati 3x3*

Zadatak 2.3.16: Kombinacije novčića

Implementirati aplikaciju koja pronalazi sve načine da se iznos od 10 dinara isplati novčićima u apoenima od 1, 2, 5 i 10 dinara. Dva načina se smatraju različitim ako se razlikuju po broju upotrebljenih novčića bar jednog apoena; redosled novčića nije bitan, a pojedini apoeni ne moraju biti upotrebljeni.

1. Napisati funkciju `kombinacije_novcica` koja za zadati iznos modeluje opisani problem kao problem programiranja ograničenja i vraća sva njegova rešenja. Funkcija se za ovu zagonetku poziva sa iznosom 10.
2. Implementirati grafički deo aplikacije koji sadrži naslov i dugme za pokretanje rešavanja. Nakon pritiska na dugme, prikaz aplikacije treba da bude kao na Slici 2.46: u tabeli se prikazuju pronađena rešenja, pri čemu svaki red odgovara jednoj kombinaciji. Kolone 1 din, 2 din, 5 din i 10 din sadrže broj novčića odgovarajućeg apoena, dok kolona Ukupno sadrži ukupan broj upotrebljenih novčića. Pošto pronađenih rešenja može biti više nego što staje u prozor, tabela ima klizač za vertikalno pomeranje (kao na slici). Ispod tabele se prikazuje labela sa ukupnim brojem pronađenih rešenja.

Isplata iznosa od 10 dinara novcicima 1, 2, 5 i 10

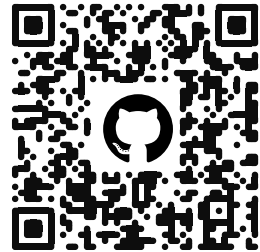
Pronadji sva resenja				
1 din	2 din	5 din	10 din	Ukupno
0	0	0	1	1
0	0	2	0	2
0	5	0	0	5
1	2	1	0	4

Broj resenja: 11

Slika 2.46: Aplikacija *Kombinacije novčića*

Glava 3

Funkcionalno programiranje



Uvod

Funkcionalno programiranje predstavlja proces programiranja pomoću primene i kompozicije funkcija. U funkcionalnom programiranju, funkcije su *građani prvog reda*, što znači da se mogu prosledivati drugim funkcijama ili vratiti kao povratna vrednost drugih funkcija, i tako gradeći funkcije *višeg reda*. Većina modernih programskih jezika podržava koncepte funkcionalnog programiranja — funkcije višeg reda kao što su `map` i `filter`, anonimne (*lambda*) funkcije, algebarske tipove, funktore, monade, i sl.

Funkcionalno programiranje garantuje svojstvo *transparentnosti referenci* — funkcije za iste ulazne argumente moraju uvek da proizvedu iste rezultate. Drugim rečima, funkcija ne može imati nuspojave (sporedne efekte). Transparentnost referenci zabranjuje korišćenje promenljivih — sve vrednosti su konstante. Zbog toga, programi pisani u funkcionalnom stilu su kratki i koncizni, često veoma razumljivi, skalabilni i pogodni za paralelizaciju.

Koncept parcijalne primene funkcije ili *karijevanja* (eng. *currying*), prisutan u većini funkcionalnih programskih jezika, omogućava primenu funkcije na argumente redom, pri čemu svaka primena vraća novu funkciju koja prihvata naredni argument. Na primer, funkcija `saberi` arnosti dva može da se posmatra kao funkcija jednog argumenta koja vraća novu funkciju arnosti jedan.

Konvencije zapisa

U nastavku ćemo koristiti konvenciju programskog jezika *Haskell*^a:

1. Imena funkcija počinju malim slovom i mogu u sebi sadržati i specijalne simbole (npr. `f'` je validno ime funkcije).
2. Imena tipova i klasa tipova počinju velikim slovom (npr. `Int`).
3. Imena funkcija i tipova se mogu sastojati samo od specijalnih simbola (npr. funkcija `+` ili tip `->`), u kom slučaju se najčešće koriste kao infiksni operatori. Svako referisanje na takve funkcije i tipove se onda mora okružiti zagradama (npr. `(+)`).
4. Parametri funkcija ili tipski parametri tipova u slučaju generičkih tipova se navode odmah nakon imena funkcije odnosno tipa (npr. `saberi a b` ili `List Int`).
5. Tip `(->)` `a b`, najčešće korišćen u infiksnoj formi (sa desnom asocijativnošću) `a -> b`, predstavlja funkciju koja kao argument prima `a` i kao rezultat vraća vrednost tipa `b`.
6. Tip izraza se navodi nakon simbola `::`, npr:
`(v + 1) :: Int` označava da izraz `v + 1` ima tip `Int`
7. Tip funkcije više argumenata se, na osnovu parcijalne primene definiše kao:
`f :: a -> (b -> (c -> ...))`
 Pošto je operator `(->)` desno-asocijativan, često ćemo izostaviti zagrade:
`f :: a -> b -> c -> ...`
 Zagrade su nekad neophodne, na primer funkcija:
`f1 :: (a -> b) -> c`
 je funkcija koja prima jedan argument (funkciju `a -> b`) i vraća vrednost tipa `(c)`. Sa druge strane, funkcija:
`f2 :: a -> b -> c`
 prima jedan argument (`a`) i vraća novu funkciju jednog argumenta (`b -> c`). Dakle, možemo funkciju `f2` posmatrati kao funkciju dva argumenta.
8. Tipski sistem korišćen u zadacima odgovara tipskom sistemu programskog jezika *Haskell*.

^aFunkcionalni programski jezik *Haskell* je dobio ime po Haskelu Kariju (eng. *Haskell Curry*), po kome je i tehnika *karijevanja* (eng. *currying*) dobila ime.

3.1 Elementarni koncepti funkcionalne paradigme

Zadatak 3.1.1

Napisati funkciju `sumaPrvihN n`, čiji je tip:

`sumaPrvihN :: Int -> Int`

koja prima pozitivan ceo broj `n` i vraća sumu prvih `n` prirodnih brojeva.

Primer 1

```
Poziv: sumaPrvihN 10
Izlaz: 45
```

Primer 2

```
Poziv: sumaPrvihN 1000
Izlaz: 499500
```

Primer 3

```
Poziv: sumaPrvihN 0
Izlaz: 0
```

Zadatak 3.1.2

Napisati funkciju `proizvodPrvih n`, čiji je tip:

`proizvodPrvih :: Integer -> Integera`

koja prima pozitivan ceo broj `n` i vraća $n!$ (faktoriyel broja `n`).

Primer 1

```
Poziv: proizvodPrvih 5
Izlaz: 120
```

Primer 2

```
Poziv: proizvodPrvih 23
Izlaz: 25852016738884976640000
```

^aTip `Integer` predstavlja ceo broj nedefinisane širine, dok je tip `Int` ograničen.

Zadatak 3.1.3

Napisati funkciju `fib n`, čiji je tip:

`fib :: Integer -> Integer`

koja prima pozitivan ceo broj `n` i računa `n`-ti element Fibonačijevog niza.

Primer 1

```
Poziv: fib 5
Izlaz: 5
```

Primer 2

```
Poziv: fib 12
Izlaz: 144
```

Primer 3

```
Poziv: fib 0
Izlaz: 0
```

Zadatak 3.1.4

Napisati funkciju `nzd a b`, čiji je tip:

`nzd :: Int -> Int -> Int`

koja prima pozitivne cele brojeve `a` i `b` i računa najveći zajednički delilac brojeva `a` i `b` (koristiti Euklidov algoritam).

Primer 1

Poziv: `nzd 12 15`
Izlaz: `3`

Primer 2

Poziv: `nzd 12 13`
Izlaz: `1`

Primer 3

Poziv: `nzd 132 444`
Izlaz: `12`

Zadatak 3.1.5

Napisati predikat `prost n`, čiji je tip:

`prost :: Integer -> Bool`

koji testira da li je pozitivan ceo broj `n` prost.

Primer 1

Poziv: `prost 5`
Izlaz: `True`

Primer 2

Poziv: `prost 2`
Izlaz: `True`

Primer 3

Poziv: `prost 1`
Izlaz: `False`

Primer 4

Poziv: `prost 673`
Izlaz: `True`

Primer 5

Poziv: `prost 20863`
Izlaz: `False`

Primer 6

Poziv: `prost 100000007`
Izlaz: `True`

Zadatak 3.1.6

Napisati funkciju `tipJednacine a b c`, čiji je tip:

`tipJednacine :: Int -> Int -> Int -> String`

koja vraća tip kvadratne jednačine $ax^2 + bx + c = 0$. Tip može biti: Degenerisana, Jedno resenje, Dva resenja, ili Bez resenja. Koeficijenti kvadratne jednačine su celi brojevi.

Primer 1

Poziv: `tipJednacine 1 2 1`
Izlaz: `Jedno resenje`

Primer 2

Poziv: `tipJednacine (-1) 8 5`
Izlaz: `Dva resenja`

Zadatak 3.1.7

Napisati funkciju `izDekadne x osnova`, čiji je tip:

`izDekadne :: Int -> Int -> Int`

koja prebacuje ceo broj `x` iz dekadne u osnovu `osnova`;

i funkciju `uDekadnu x osnova`, čiji je tip:

`uDekadnu :: Int -> Int -> Int`

koja prebacuje ceo broj `x` iz osnove `osnova` u dekadnu osnovu.

Pretpostaviti da je $1 < \text{osnova} < 10$.

Primer 1

Poziv: `izDekadne 101 2`
Izlaz: `1100101`

Primer 2

Poziv: `uDekadnu 765 8`
Izlaz: `501`

Zadatak 3.1.8

Napisati funkciju `ceoDeo x`, čiji je tip:

`ceoDeo :: Int -> Int`

koja računa ceo deo korena pozitivnog celog broja `x` (bez korišćenja ugrađenih funkcija za koren i/ili stepen).

Primer 1

Poziv: `ceoDeo 15`
Izlaz: `3`

Primer 2

Poziv: `ceoDeo 100`
Izlaz: `10`

Zadatak 3.1.9

Napisati funkciju `parMax p`, čiji je tip:

`parMax :: (Double, Double) -> Double`

koja određuje veći element iz para realnih brojeva `p`.

Primer 1

Poziv: `parMax`
 `(3.1, 4.2)`
Izlaz: `4.2`

Primer 2

Poziv: `parMax`
 `(-4.1, -5.1)`
Izlaz: `-4.1`

Primer 3

Poziv: `parMax`
 `(-4.1, 4.1)`
Izlaz: `4.1`

Zadatak 3.1.10

Napisati funkciju `sumaKvadrata n`, čiji je tip:

`sumaKvadrata :: Int -> Int`

koja prima pozitivan ceo broj `n` i računa sumu kvadrata prvih `n` prirodnih brojeva (bez korišćenja formule).

Primer 1

```
Poziv: sumaKvadrata 5  
Izlaz: 55
```

Primer 2

```
Poziv: sumaKvadrata 10  
Izlaz: 385
```

Zadatak 3.1.11

Napisati funkciju `delioci n`, čiji je tip:

`delioci :: Int -> [Int]`

koja pravi listu svih pravih delioca pozitivnog celog broja `n`.

Primer 1

```
Poziv: delioci 12  
Izlaz: [2,3,4,6]
```

Primer 2

```
Poziv: delioci 13  
Izlaz: []
```

Zadatak 3.1.12

Napisati funkciju `brojDelilaca n`, čiji je tip:

`brojDelilaca :: Int -> Int`

koja prima pozitivan ceo broj `n` i vraća broj pravih delilaca broja `n`.

Primer 1

```
Poziv: brojDelilaca 5  
Izlaz: 0
```

Primer 2

```
Poziv: brojDelilaca 12  
Izlaz: 4
```

3.2 Liste i funkcije višeg reda

Zadatak 3.2.1

Napisati funkciju `lista a b`, čiji je tip:

`lista :: Int -> Int -> [Int]`

koja pravi listu celih brojeva iz segmenta $[a, b]$. U slučaju da granice segmenta nisu ispravne, rezultat je prazna lista.

Primer 1

```
Poziv: lista 1 10
Izlaz: [1,2,3,4,5,6,7,8,9,10]
```

Primer 2

```
Poziv: lista 9 4
Izlaz: []
```

Zadatak 3.2.2

Napisati funkciju `parni n`, čiji je tip:

`parni :: Int -> [Int]`

koja prima pozitivan ceo broj n i pravi listu prvih n parnih prirodnih brojeva.

Primer 1

```
Poziv: parni 3
Izlaz: [2,4,6]
```

Primer 2

```
Poziv: parni 8
Izlaz: [2,4,6,8,10,12,14,16]
```

Zadatak 3.2.3

Napisati funkciju `zavisnoY a b`, čiji je tip:

`zavisnoY :: Int -> Int -> [(Int, Int)]`

koja generiše listu parova celih brojeva (x, y) , za koje x pripada segmentu $[a, b]$, a y pripada segmentu $[x, b]$.

Primer 1

```
Poziv: zavisnoY 1 3
Izlaz: [(1,1),(1,2),(1,3),
        (2,2),(2,3),(3,3)]
```

Primer 2

```
Poziv: zavisnoY 0 2
Izlaz: [(0,0),(0,1),(0,2),
        (1,1),(1,2),(2,2)]
```

Zadatak 3.2.4

Definisati funkciju `prefixN 1`, čiji je tip:

`prefixN :: [Int] -> [Int]`

koja izdvaja najduži prefiks negativnih elemenata celobrojne liste 1.

Primer 1

```
Poziv: prefixN [-2,-1,0,-2]
Izlaz: [-2,-1]
```

Primer 2

```
Poziv: prefixN [1,2,3,4,5]
Izlaz: []
```

Zadatak 3.2.5

Napisati funkciju `nadovezi n 11 12`, čiji je tip:

`nadovezi :: Int -> [a] -> [a] -> [a]`

koja prima pozitivan ceo broj `n` i `n` puta nadovezuje listu 12 na listu 11.

Primer 1

```
Poziv: nadovezi 3 [1] [2]
Izlaz: [1,2,2,2]
```

Primer 2

```
Poziv: nadovezi 2 [] [1,2,3]
Izlaz: [1,2,3,1,2,3]
```

Zadatak 3.2.6

Napisati funkciju `harm n`, čiji je tip:

`harm :: Int -> [Double]`

koja prima pozitivan ceo broj `n` i pravi listu prvih `n` elemenata harmonijskog reda. Harmonijski red se dobija po formuli:

$$\sum_{n=1}^{\infty} \frac{1}{n} = 1 + \frac{1}{2} + \frac{1}{3} + \frac{1}{4} + \dots$$

Primer 1

```
Poziv: harm 2
Izlaz: [1.0,0.5]
```

Primer 2

```
Poziv: harm 7
Izlaz: [1.0,0.5,0.33333333,0.25,0.2,0.16666666,0.14285714]
```


Zadatak 3.2.7

Napisati funkciju `fibLista n`, čiji je tip:

`fibLista :: Int -> [Int]`

koja prima pozitivan ceo broj n i pravi listu prvih n elemenata Fibonačijevog niza.

Primer 1

```
Poziv: fibLista 14
Izlaz: [1,1,2,3,5,8,13,21,34,55,89,144,233,377]
```

Zadatak 3.2.8

Napisati funkciju `savrseni n`, čiji je tip:

`savrseni :: Int -> [Int]`

koja pravi listu savršenih brojeva manjih od n . Broj n je savršen ukoliko je jednak sumi svojih faktora (delilaca) manjih od n .

Primer 1

```
Poziv: savrseni 50
Izlaz: [6,28]
```

Primer 2

```
Poziv: savrseni 1000
Izlaz: [6,28,496]
```

Zadatak 3.2.9

Napisati funkciju `magicni l`, čiji je tip:

`magicni :: [Int] -> [(Int, Int)]`

koja svaki prirodan broj iz liste l uparuje sa njegovim magičnim parnjakom. Magičan parnjak broja n odgovara proizvodu cifara broja n .

Primer 1

```
Poziv: magicni [27,4]
Izlaz: [(27,14),(4,4)]
```

Primer 2

```
Poziv: magicni [10,154]
Izlaz: [(10,0),(154,20)]
```

Primer 3

```
Poziv: magicni [100,99,11,22,112233]
Izlaz: [(100,0),(99,81),(11,1),(22,4),(112233,36)]
```

Zadatak 3.2.10

Napisati funkciju `zbirPar n`, čiji je tip:

`zbirPar :: Int -> [(Int, Int)]`

koja pravi listu parova (a, b) takvih da su a i b prirodni brojevi čiji je zbir jednak n .

Primer 1

```
Poziv: zbirPar 10
Izlaz: [(1,9),(2,8),(3,7),(4,6),(5,5),(6,4),(7,3),(8,2),(9,1)]
```

Zadatak 3.2.11

Napisati funkciju `poslednji l`, čiji je tip:

`poslednji :: [a] -> a`

koja određuje poslednji element neprazne liste l .

Primer 1

```
Poziv: poslednji ["pon","ned"]
Izlaz: "ned"
```

Primer 2

```
Poziv: poslednji [5,4,3,2,1]
Izlaz: 1
```

Zadatak 3.2.12

Nakon još jedne košarkaške sezone, potrebno je sumirati rezultate svih igrača. Svaki igrač ima jedinstveni redni broj, pod kojim se u bazi, u listi prezimena, čuva njegovo prezime, a u listi pogodaka, ostvaren broj poena u sezoni. Uparivanjem odgovarajućih podataka, napraviti za svakog igrača jedinstveni par oblika `(prezime, poeni)`.

Napisati funkciju `parOdListi lista1 lista2`, čiji je tip:

`parOdListi :: [String] -> [Int] -> [(String, Int)]`

koja pravi listu parova od dve liste, liste prezimena i liste pogodaka, tako da prvi element svakog para bude iz prve liste, a drugi element svakog para bude iz druge liste.

Primer 1

```
Poziv: parOdListi ["Mikic","Peric","Jovic"] [100,76,96]
Izlaz: [("Mikic",100),("Peric",76),("Jovic",96)]
```

Zadatak 3.2.13

U toku su prijave za plesno takmičenje parova. Pored nagrade za najbolji par, dodeljuju se i pojedinačne nagrade za najboljeg ženskog i muškog takmičara. Da bi žiri jednostavnije beležio poene i odredio nagrade, potrebno je da im se dostave, pored liste parova koji se takmiče, i liste samo muških, tj. samo ženskih takmičara, odvojeno. Napisati funkciju `listaUPar` lista, čiji je tip:

`listaUPar :: [(String, String)] -> ([String], [String])`

koja pretvara listu parova u par dve liste, tako da prva lista sadrži prve elemente svih parova, a druga druge elemente svih parova pod pretpostavkom da je prvi u paru uvek ženska osoba, a drugi muška.

Primer 1

```
Poziv: listaUPar
      [("Ivana","Milan"), ("Ana","Jovan"), ("Anica","Petar")]
Izlaz: (["Ivana","Ana","Anica"], ["Milan","Jovan","Petar"])
```

Zadatak 3.2.14

Definisati funkciju `obrni 1`, čiji je tip:

`obrni :: [a] -> [a]`

za obrtanje liste 1.

Primer 1

```
Poziv: obrni [1,2,3]
Izlaz: [3,2,1]
```

Primer 2

```
Poziv: obrni "abcde"
Izlaz: "edcba"
```

Zadatak 3.2.15

Definisati funkciju `svi p 1`, čiji je tip:

`obrni :: (a -> Bool) -> [a] -> Bool`

koja proverava da li svi elementi liste 1 zadovoljavaju dati predikat p.

Primer 1

```
Poziv: svi even [2,4,6]
Izlaz: True
```

Primer 2

```
Poziv: svi (>3) [1,2,3,4,5]
Izlaz: False
```

Zadatak 3.2.16

Data je lista 1 koja sadrži liste ocena raznih učenika osnovne škole.

Definisati funkciju `prosekOdlicni 1`, čiji je tip:

`prosekOdlicni :: [Int] -> Double`

koja računa prosek svih odličnih učenika čije su ocene date u listi 1.

Primer 1

```
Poziv: prosekOdlicni [[1,2,3],[5,5,5],[4,5,5]]
Izlaz: 4.833333
```

Zadatak 3.2.17

Napisati funkciju `spoji 1`, čiji je tip:

`spoji :: [[a]] -> [a]`

koja spaja listu listi istog tipa 1 u jednu listu.

Primer 1

```
Poziv: spoji [["a"],["b"],["c"]]
Izlaz: ["a","b","c"]
```

Primer 2

```
Poziv: spoji [[],[1],[],[1,2,3]]
Izlaz: [1,1,2,3]
```

Zadatak 3.2.18

Definisati funkciju `prefiksi 1`, čiji je tip:

`prefiksi :: [a] -> [[a]]`

koja pravi listu svih prefiksa liste 1.

Primer 1

```
Poziv: prefiksi [1,2,3,4,5]
Izlaz: [[],[1],[1,2],[1,2,3],[1,2,3,4],[1,2,3,4,5]]
```

Primer 2

```
Poziv: prefiksi "abcdef"
Izlaz: ["","a","ab","abc","abcd","abcde","abcdef"]
```

Zadatak 3.2.19

Napisati funkciju `sufiksi l`, čiji je tip:

`sufiksi :: [a] -> [[a]]`

koja pravi listu svih sufixa liste `l`.

Primer 1

```
Poziv:  sufiksi [1,2,3]
Izlaz:  [[1,2,3],[2,3],[3],[]]
```

Primer 2

```
Poziv:  sufiksi []
Izlaz:  [[]]
```

Zadatak 3.2.20

Napisati funkciju `izbaci k l`, čiji je tip:

`izbaci :: Int -> [a] -> [a]`

koja izbacuje `k`-ti element iz liste `l`. U slučaju da je zadata neispravna pozicija u listi, funkcija vraća nepromenjenu listu.

Primer 1

```
Poziv:  izbaci 2 [1,2,3,4,5]
Izlaz:  [1,2,4,5]
```

Primer 2

```
Poziv:  izbaci (-2) [1,2,3]
Izlaz:  [1,2,3]
```

Zadatak 3.2.21

Napisati funkciju `ubaci k e l`, čiji je tip:

`brisiPonavljanja :: Int -> a -> [a] -> [a]`

koja ubacuje u listu `l` na poziciju `k` element `e`. U slučaju da je zadata neispravna pozicija u listi, dodati element `e` na kraj liste.

Primer 1

```
Poziv:  ubaci 2 5 [1,2,3]
Izlaz:  [1,2,5,3]
```

Primer 2

```
Poziv:  ubaci 10 5 [1,2,3,4,5]
Izlaz:  [1,2,3,4,5,5]
```

Zadatak 3.2.22

Milan obožava da sakuplja sličice fudbalera. Da bi jednostavnije pratio koje mu sličice iz kolekcije još uvek nedostaju, čuva ih rastuće sortirane po rednom broju. Planirao je da dopuni svoju kolekciju na narednoj razmeni sličica, pa za tu priliku želi da iz svoje kolekcije izbaci sve nepotrebne duplikate koje će poneti na razmenu.

Napisati funkciju `brisiPonavljanja` 1, čiji je tip:

`brisiPonavljanja :: [a] -> [a]`

koja briše sva uzastopna ponavljanja elemenata u listi 1.

Primer 1

Poziv: `brisiPonavljanja`
`[10,10,10,11]`
Izlaz: `[4,5,2,11]`

Primer 2

Poziv: `brisiPonavljanja`
`[2,10,11,11,11,2,2]`
Izlaz: `[2,10,11,2]`

Zadatak 3.2.23

Napisati funkciju `ukloniDuplikate` 1, čiji je tip:

`ukloniDuplikate :: [a] -> [a]`

koja uklanja sve duplikate iz liste 1.

Primer 1

Poziv: `ukloniDuplikate`
`[1,2,1,3]`
Izlaz: `[1,2,3]`

Primer 2

Poziv: `ukloniDuplikate`
`"abaccddb"`
Izlaz: `"abcd"`

Zadatak 3.2.24

Suzana uči brojanje i razlikovanje boja koristeći kutiju punu jednobojnih kuglica. Ona prvo žmureći iz kutije izvuče određeni broj kuglica i poređa ih u niz u redosledu izvlačenja. Zatim izabere proizvoljnu boju i odredi na kojoj se sve poziciji u nizu izvučenih kuglica nalazi kuglica željene boje.

Napisati funkciju `pozicije` k 1, čiji je tip:

`pozicije :: String -> [String] -> [Int]`

koja vraća listu pozicija kuglice k u listi 1.

Primer 1

Poziv: `pozicije "bela" ["plava", "bela", "bela", "ljubicasta", "bela"]`
Izlaz: `[1,2,4]`

Zadatak 3.2.25

Kasirka Mica mora da ručno kuca artikle na kasi jer se pokvario skener barkodova. Pomozite Mici da taj posao obavi što brže grupisanjem artikala iste vrste na pokretnoj traci.

Napisati funkciju `grupisi l`, čiji je tip:

`grupisi :: [String] -> [String]`

koja grupiše sva uzastopna ponavljanja nekog elementa liste `l` u podlistu tako da rezultat bude lista listi.

Primer 1

```
Poziv: grupisi ["sir","jogurt","jogurt","hleb","hleb","mleko"]
Izlaz: [["sir"],["jogurt","jogurt"],["hleb","hleb"],["mleko"]]
```

Primer 2

```
Poziv: grupisi ["jabuke","hleb","sir","jabuke"]
Izlaz: [["jabuke"],["hleb"],["sir"],["jabuke"]]
```

Zadatak 3.2.26

Napisati funkciju `broj lista`, čiji je tip:

`broj :: [Int] -> Int`

koja vraća broj određen ciframa koje se nalaze u listi čitajući ih sa početka ka kraju liste i funkciju `broj' lista`, čiji je tip:

`broj' :: [Int] -> Int`

koja vraća broj određen ciframa koje se nalaze u listi čitajući ih sa kraja ka početku liste.

Primer 1

```
Poziv: broj [1,0,1]
Izlaz: 101
```

Primer 2

```
Poziv: broj [0,5,2]
Izlaz: 250
```

Primer 3

```
Poziv: broj [0,5,2,0]
Izlaz: 250
```

Primer 4

```
Poziv: broj [1,2,3,4,5]
Izlaz: 54321
```

Zadatak 3.2.27

Definisati funkciju `qsort` 1, čiji je tip:

`qsort :: Ord a => [a] -> [a]`

sortira listu 1 u rastućem poretку koristeći algoritam *quick sort*. Za pivot uzeti prvi element liste.

Primer 1

Poziv: `qsort [1,2,1,3]`
Izlaz: `[1,1,2,3]`

Primer 2

Poziv: `qsort "abacddb"`
Izlaz: `"aabbccdd"`

Zadatak 3.2.28

Definisati funkciju varijacije 1 k, čiji je tip:

`varijacije :: [a] -> Int -> [[a]]`

koja generiše listu koja sadrži sve varijacije sa ponavljanjem elemenata date liste 1 dužine k.

Primer 1

Poziv: `varijacije [1,2,3] 2`
Izlaz: `[[1,1],[1,2],[1,3],[2,1],[2,2],[2,3],[3,1],[3,2],[3,3]]`

Zadatak 3.2.29

Formacija igre kolo je polukrug sa istaknutom ulogom prvog i poslednjeg igrača.

Napisati funkciju `ucesljaj mIgraci zIgraci`, čiji je tip:

`ucesljaj :: [String] -> [String] -> [String]`

koja pravi jednu formaciju za kolo naizmeničnim učešljavanjem igrača iz date grupe muških (`mIgraci`) i ženskih (`zIgraci`) igrača, redom.

Primer 1

Poziv: `ucesljaj ["Petar","Aleksa","Filip"]`
`["Milica","Jovana","Anica"]`
Izlaz: `["Petar","Milica","Aleksa","Jovana","Filip","Anica"]`

Zadatak 3.2.30

Aleksa i Luka žele da komuniciraju preko šifrovanih poruka koje predstavljaju listom niski. Aleksa šifrjuje željene podatke na sledeći način: ukoliko se niska koju šalje sastoji samo od cifara, na njen početak i kraj dodaje karakter C, ako se sastoji samo od malih slova, na njen početak i kraj dodaje karakter S, a inače na njen početak i kraj dodaje karakter O. Definisati sledeće funkcije koje pomažu Aleksi da pošalje Luki šifrovanu poruku:

1. broj s, čiji je tip:

```
broj :: String -> Bool
```

koja za datu nisku s proverava da li su svi njeni karakteri cifre;

Primer 1

```
Poziv: broj "123"
Izlaz: True
```

Primer 2

```
Poziv: broj ['c','a','o']
Izlaz: False
```

2. mala s, čiji je tip:

```
mala :: String -> Bool
```

koja za datu nisku s proverava da li su svi njeni karakteri mala slova;

Primer 3

```
Poziv: mala "123"
Izlaz: False
```

Primer 4

```
Poziv: mala ['c','a','o']
Izlaz: True
```

3. sifruj ls, čiji je tip:

```
sifruj :: [String] -> [String]
```

koja datu listu niski ls transformiše na sledeći način: ukoliko se niska koju šalje sastoji samo od cifara, na njen početak i kraj dodaje karakter C, ako se sastoji samo od malih slova, na njen početak i kraj dodaje karakter M, a inače na njen početak i kraj dodaje karakter O.

Primer 5

```
Poziv: sifruj ["11","mj","dAn","test"]
Izlaz: ["C11C","MmjM","OdAnO","MtestM"]
```

Zadatak 3.2.31

Za razliku od Alekse i Luke, Ana i Milica imaju problem dešifrovanja podataka. Da bi Ana dešifrovala podatke koje joj Milica pošalje potrebno je da svaku nisku iz dobijene liste transformiše na sledeći način: ukoliko dobijena niska počinje cifrom, sa njenog početka izbaciti onoliko karaktera koliko ta niska ima cifara, a ukoliko dobijena niska počinje malim slovom, sa njenog početka izbaciti onoliko karaktera koliko ta niska ima malih slova. Pretpostaviti da će ovakvim dešifrovanjem Ana uvek dobiti ispravne izvorne podatke. Definisati sledeće funkcije koje pomažu Ani da dešifruje Miličine poruke:

1. `cifre s`, čiji je tip:

`cifre :: String -> Int`

koja za datu nisku `s` vraća broj karaktera niske koji su cifre;

Primer 1

```
Poziv: cifre "11Maj"
Izlaz: 2
```

Primer 2

```
Poziv: cifre ['m','a','j']
Izlaz: 0
```

2. `mala s`, čiji je tip:

`mala :: String -> Int`

koja za datu nisku `s` vraća broj karaktera niske koji su mala slova;

Primer 3

```
Poziv: mala "petak"
Izlaz: 5
```

Primer 4

```
Poziv: mala ['C','a','o']
Izlaz: 2
```

3. `desifruj ls`, čiji je tip:

`desifruj :: [String] -> [String]`

koja datu listu niski `ls` transformiše na sledeći način: ukoliko niska počinje cifrom, sa njenog početka izbacuje onoliko karaktera koliko ta niska ima cifara, a ukoliko dobijena niska počinje malim slovom, sa njenog početka izbacuje onoliko karaktera koliko ta niska ima malih slova.

Primer 5

```
Poziv: desifruj ["aaa11","iminamaj2017","101petak"]
Izlaz: ["11","maj2017","petak"]
```

3.3 Algebarski tipovi, klase i svojstva

Zadatak 3.3.1

Definisati tip podataka **Oblik** koji može biti **Kvadrat** (karakterisan stranicom **a**), **Pravougaonik** (karakterisan dvema stranicama **a** i **b**), **Krug** (karakterisan poluprečnikom **r**) ili **Trougao** (karakterisan sa tri stranice **a**, **b** i **c**). Dužine su realni brojevi.

Primer 1

```
POZIV: Kvadrat 3.0
IZLAZ: Kvadrat {a = 3.0}
```

Primer 2

```
POZIV: Pravougaonik 3.0 4.0
IZLAZ: Pravougaonik {a = 3.0,
b = 4.0}
```

Primer 3

```
POZIV: Krug 3.0
IZLAZ: Krug {r = 3.0}
```

Primer 4

```
POZIV: Trougao 1.0 2.0 3.0
IZLAZ: Trougao {a = 1.0,
b = 2.0, c = 3.0}
```

Zadatak 3.3.2

Definisati tip podataka **Zivotinja** koji može biti **Pas**, **Macka** ili **Papagaj**. Zatim definisati tip **Ljubimac** karakterisan imenom, godinama i tipom životinje.

Primer 1

```
POZIV: Ljubimac Pas "Lesi" 1
IZLAZ: Ljubimac {tip = Pas, ime = "Lesi", godine = 1}
```

Primer 2

```
POZIV: Ljubimac Macka "Kiti" 5
IZLAZ: Ljubimac {tip = Macka, ime = "Kiti", godine = 5}
```

Primer 3

```
POZIV: Ljubimac Papagaj "Lu" 3
IZLAZ: Ljubimac {tip = Papagaj, ime = "Lu", godine = 3}
```

Zadatak 3.3.3

Definisati tip podataka `Prav` karakterisan dvema realnim stranicama `a` i `b`. Postarati se da se pravougaonici mogu porediti po jednakosti i prikazati u formatu `[a x b]`.

Primer 1

```
Poziv: (Prav 3.0 4.0) == (Prav 3.0 4.0)
Izlaz: True
```

Primer 2

```
Poziv: (Prav 3.0 4.0) == (Prav 3.0 4.1)
Izlaz: False
```

Primer 3

```
Poziv: Prav 3.0 4.0
Izlaz: [3.0 x 4.0]
```

Zadatak 3.3.4

Napisati funkcije `glava` i `rep` koje bezbedno vraćaju glavu i rep liste, koristeći tip koji enkapsulira opcionu vrednost.

Primer 1

```
Poziv: glava [1,2,3]
Izlaz: Just 1
```

Primer 2

```
Poziv: glava []
Izlaz: Nothing
```

Primer 3

```
Poziv: glava ["a","b","c"]
Izlaz: Just "a"
```

Primer 4

```
Poziv: glava [(1,2)]
Izlaz: Just (1,2)
```

Primer 5

```
Poziv: rep [1,2,3]
Izlaz: Just [2,3]
```

Primer 6

```
Poziv: rep []
Izlaz: Nothing
```

Primer 7

```
Poziv: rep ["a","b","c"]
Izlaz: Just ["b","c"]
```

Primer 8

```
Poziv: rep [(1,2)]
Izlaz: Just []
```

Zadatak 3.3.5

Napisati funkcije **glava** i **rep** koje bezbedno vraćaju glavu i rep liste, koristeći tip koji reprezentuje dve mogućnosti — vrednost glave liste, ili poruku o grešci (u primeru ispod leva vrednost označava grešku, dok desna označava validnu vrednost).

Primer 1

```
Poziv: glava [1,2,3]
Izlaz: Right 1
```

Primer 2

```
Poziv: glava []
Izlaz: Left "Prazna lista"
```

Primer 3

```
Poziv: glava ["a","b","c"]
Izlaz: Right "a"
```

Primer 4

```
Poziv: glava [(1,2)]
Izlaz: Right (1,2)
```

Primer 5

```
Poziv: rep [1,2,3]
Izlaz: Right [2,3]
```

Primer 6

```
Poziv: rep []
Izlaz: Left "Prazna lista"
```

Primer 7

```
Poziv: rep ["a","b","c"]
Izlaz: Right ["b","c"]
```

Primer 8

```
Poziv: rep [(1,2)]
Izlaz: Right []
```

Zadatak 3.3.6

Napisati biblioteku za rad sa tačkama u ravni. Definirati tipove **Tacka** (alias za par, gde su koordinate realni brojevi) i **Putanja** (alias za listu tačaka). Implementirati funkcije:

1. **tacka x y**, koja konstruiše tačku sa datim koordinatama **x** i **y**,
2. **putanja ts**, koja konstruiše putanju od date liste tačaka **ts**,
3. **duzinaPutanje p**, koja vraća dužinu putanje **p**,
4. **translirajTacku t dx dy**, koja translira tacku **t** za vektor **(dx, dy)**,
5. **rastojanje t1 t2**, koja vraća rastojanje između tačaka **t1** i **t2**,
6. **uKrugu r ts**, koja iz liste tačaka **ts** vraća one koje se nalaze u krugu poluprečnika **r** od tačke **(0,0)**,
7. **translirajPutanju p dx dz**, koja translira sve tačke putanje **p** za vektor **(dx, dy)**,
8. **nadovezi t p**, koja nadovezuje tačku **t** na kraj putanje **p**,
9. **spojiPutanje p1 p2**, koja spaja putanje **p1** i **p2** i vraća novu putanju,
10. **centroid ts**, koja vraća centroid liste tačaka **ts**,
11. **kvadrantTacke t**, koja vraća kvadrant tačke **t** (broj u intervalu **[1, 4]**, 0 ako je u pitanju koordinatni početak),
12. **kvadrantPutanje p**, koja vraća kvadrant u kom se nalazi putanja **p** ako je ona u celosti sadržana u nekom od njih, 0 inače,
13. **tackeUKvadrantu kv ts**, koja iz liste tačaka **ts** vraća samo one koje se nalaze u kvadrantu **kv**,
14. **tackeVanKvadranta kv ts**, koja iz liste tačaka **ts** vraća one koje se nalaze van kvadranta **kv**, i
15. **maksimumi ts**, koja vraća uređeni par koji predstavlja maksimume **x** i **y** koordinata tačaka liste **ts**.

Primer 1

```
POZIV: tacka 1.0 2.0  
IZLAZ: (1.0, 2.0)
```

Primer 2

```
POZIV: tacka (-1.0) 2.0  
IZLAZ: (-1.0, 2.0)
```

Primer 3

```
POZIV: putanja [tacka 0 0, tacka 1 1]  
IZLAZ: [(0.0,0.0),(1.0,1.0)]
```

Primer 4

```
POZIV: duzinaPutanje (putanja [tacka 0 0, tacka 3 4])  
IZLAZ: 5.0
```

Primer 5

```
POZIV: translirajTacku (tacka 1 2) 3 4  
IZLAZ: (4.0,6.0)
```

Primer 6

```
POZIV: rastojanje (tacka 0 0) (tacka 3 4)  
IZLAZ: 5.0
```

Primer 7

```
POZIV: uKrug 2.5 [tacka 1 1, tacka 3 3, tacka 2 0]  
IZLAZ: [(1.0,1.0),(2.0,0.0)]
```

Primer 8

```
POZIV: translirajPutanju (putanja [tacka 0 0, tacka 1 1]) 1 2  
IZLAZ: [(1.0,2.0),(2.0,3.0)]
```

Primer 9

```
POZIV: nadovezi (tacka 2 2) (putanja [tacka 0 0, tacka 1 1])  
IZLAZ: [(0.0,0.0),(1.0,1.0),(2.0,2.0)]
```

Primer 10

```
Poziv: spojiPutanje (putanja [tacka 0 0])  
      (putanja [tacka 1 1, tacka 2 2])  
Izlaz: [(0.0,0.0),(1.0,1.0),(2.0,2.0)]
```

Primer 11

```
Poziv: centroid [tacka 0 0, tacka 2 2]  
Izlaz: (1.0,1.0)
```

Primer 12

```
Poziv: kvadrantTacke  
      (tacka 1 1)  
Izlaz: 1
```

Primer 13

```
Poziv: kvadrantTacke  
      (tacka -1 1)  
Izlaz: 2
```

Primer 14

```
Poziv: kvadrantPutanje (putanja [tacka 1 1, tacka 2 2])  
Izlaz: 1
```

Primer 15

```
Poziv: kvadrantPutanje (putanja [tacka 1 1, tacka -1 1])  
Izlaz: 0
```

Primer 16

```
Poziv: tackeUKvadrantu 1 [tacka 1 1, tacka -1 1, tacka 2 2]  
Izlaz: [(1.0,1.0),(2.0,2.0)]
```

Primer 17

```
Poziv: tackeVanKvadranta 1 [tacka 1 1, tacka -1 1, tacka 2 2]  
Izlaz: [(-1.0,1.0)]
```

Primer 18

```
Poziv: maksimumi [tacka 1 5, tacka 3 2, tacka 2 7]  
Izlaz: (3.0,7.0)
```


Zadatak 3.3.7

Napisati biblioteku za pomoć asistentima u arhiviranju i održavanju rezultata ispita. Definirati tipove **StepenStudija** (osnovne, master, doktorske), **Student** (karakterisan brojem indeksa, imenom, prezimenom i stepenom studija) i **Rezultat** (svaki student ima opcioni rezultat predstavljen brojem poena). Obezbediti da se student može ispisati na standardni izlaz i porediti po jednakosti sa ostalim studentima po broju indeksa. Implementirati funkcije:

1. **rezultati studenti**, koja konstruiše listu rezultata za date studente, gde je svaki rezultat trenutno prazan,
2. **poeni student rezultati**, koja vraća broj poena datog studenta iz liste rezultata ili poruku ako se student ne nalazi u istoj,
3. **ponisti student rezultati**, koja poništava poene za datog studenta iz liste rezultata,
4. **ponistiSve rezultati**, koja poništava poene za sve studente iz liste rezultata,
5. **studije stepenStudija rezultati**, koja vraća samo one rezultate za studente sa datog stepena studija,
6. **polozeni rezultati**, koja vraća samo one rezultate gde je student položio ispit,
7. **upisi student poeni rezultati**, koja upisuje novi rezultat za datog studenta u listu rezultata,
8. **najboljih n rezultati**, koja vraća *n* najboljih rezultata (samo broj poena) iz liste rezultata, sortiranih opadajuće.

Primer 1

```
POZIV: rezultati [Student 1 "Pera" "Peric" Osnovne,  
Student 2 "Mika" "Mikic" Master]  
IZLAZ: [Rezultat (Student 1 "Pera" "Peric" Osnovne) Nothing,  
Rezultat (Student 2 "Mika" "Mikic" Master) Nothing]
```

Primer 2

```
POZIV: upisi (Student 1 "Pera" "Peric" Osnovne) 85  
(rezultati [Student 1 "Pera" "Peric" Osnovne])  
IZLAZ: [Rezultat (Student 1 "Pera" "Peric" Osnovne) (Just 85)]
```

Primer 3

```
Poziv: poeni (Student 1 "Pera" "Peric" Osnovne)
      [Rezultat (Student 1 "Pera" "Peric" Osnovne) (Just 90)]
Izlaz: Right 90
```

Primer 4

```
Poziv: poeni (Student 2 "Mika" "Mikic" Master) []
Izlaz: Left "Student nije pronadjen"
```

Primer 5

```
Poziv: ponisti (Student 1 "Pera" "Peric" Osnovne)
      [Rezultat (Student 1 "Pera" "Peric" Osnovne) (Just 80)]
Izlaz: [Rezultat (Student 1 "Pera" "Peric" Osnovne) Nothing]
```

Primer 6

```
Poziv: ponistiSve
      [Rezultat (Student 1 "Pera" "Peric" Osnovne) (Just 80),
       Rezultat (Student 2 "Mika" "Mikic" Master) (Just 70)]
Izlaz: [Rezultat (Student 1 "Pera" "Peric" Osnovne) Nothing,
       Rezultat (Student 2 "Mika" "Mikic" Master) Nothing]
```

Primer 7

```
Poziv: studije Osnovne
      [Rezultat (Student 1 "Pera" "Peric" Osnovne) (Just 80),
       Rezultat (Student 2 "Mika" "Mikic" Master) (Just 70)]
Izlaz: [Rezultat (Student 1 "Pera" "Peric" Osnovne) (Just 80)]
```

Primer 8

```
Poziv: polozeni
      [Rezultat (Student 1 "Pera" "Peric" Osnovne) (Just 80),
       Rezultat (Student 2 "Mika" "Mikic" Master) (Just 45)]
Izlaz: [Rezultat (Student 1 "Pera" "Peric" Osnovne) (Just 80)]
```

Primer 9

```
Poziv: najboljih 2
      [Rezultat (Student 1 "Pera" "Peric" Osnovne) (Just 80),
       Rezultat (Student 2 "Mika" "Mikic" Master) (Just 90),
       Rezultat (Student 3 "Zika" "Zikic" Doktorske) (Just 85)]
Izlaz: [90,85]
```

Primer 10

```
Poziv: najboljih 2
      [Rezultat (Student 1 "Pera" "Peric" Osnovne) Nothing,
       Rezultat (Student 2 "Zika" "Zikic" Osnovne) (Just 45),
       Rezultat (Student 3 "Mika" "Mikic" Osnovne) Nothing]
Izlaz: [45]
```

Zadatak 3.3.8

Definisati tipove podataka *Krug*, *Prav* i *Kvadrat*. Tip *Krug* sadrži poluprečnik, *Prav* širinu i visinu, dok *Kvadrat* sadrži dužinu stranice. Zatim definisati klasu *Oblik* koja opisuje geometrijske oblike, i sastoji se od funkcija:

```
povrsina :: a -> Double i
obim     :: a -> Double.
```

Instancirati klasu *Oblik* za tipove *Krug*, *Prav* i *Kvadrat*.

Primer 1

```
Poziv: povrsina (Krug 1)
Izlaz: 3.141592653589793
```

Primer 2

```
Poziv: obim (Krug 1)
Izlaz: 6.283185307179586
```

Primer 3

```
Poziv: povrsina (Prav 2 3)
Izlaz: 6.0
```

Primer 4

```
Poziv: obim (Prav 2 3)
Izlaz: 10.0
```

Primer 5

```
Poziv: povrsina (Kvadrat 4)
Izlaz: 16.0
```

Primer 6

```
Poziv: obim (Kvadrat 4)
Izlaz: 16.0
```

Zadatak 3.3.9

Definisati tip podataka `UList a` koji implementira jednostruko povezanu listu elemenata tipa `a`. Definisati konstruktore `Empty` (za konstrukciju prazne liste) i `Cons x xs` za nadovezivanje elementa `x` (tipa `a`) na listu `xs` (tipa `UList a`).

Instancirati klase `Show` (sa formatom kao u test primerima ispod), `Eq`, i `Foldable` za tip `UList`.

Primer 1

```
POZIV: show (Cons 1 (Cons 2 Empty))
IZLAZ: "<[1,2]>"
```

Primer 2

```
POZIV: show Empty
IZLAZ: "<[]>"
```

Primer 3

```
POZIV: Cons 1 (Cons 2 (Cons 3 Empty)) ==
      Cons 1 (Cons 2 (Cons 3 Empty))
IZLAZ: True
```

Primer 4

```
POZIV: Cons 1 (Cons 2 Empty) == Cons 2 (Cons 1 Empty)
IZLAZ: False
```

Primer 5

```
POZIV: foldr (+) 0 (Cons 1 (Cons 2 (Cons 3 Empty)))
IZLAZ: 6
```

Primer 6

```
POZIV: foldr (:) [] (Cons 1 (Cons 2 (Cons 3 Empty)))
IZLAZ: [1,2,3]
```

Zadatak 3.3.10

Definisati tip podataka `OList a` koji implementira jednostruko povezanu listu elemenata tipa `a`. Definisati konstruktore `End` (za konstrukciju prazne liste) i `(:/:) x xs` za nadovezivanje elementa `x` (tipa `a`) na listu `xs` (tipa `OList a`). Konstruktor `(:/:)` implementirati kao infiksni binarni operator.

Instancirati klase `Show` (sa formatom kao u test primerima ispod), `Eq`, i `Foldable` za tip `OList`.

Definisati infiksni operator `(+ / +)`, čiji je tip:

`OList a -> OList a -> OList a`,
koji spaja dve liste tipa `OList a`.

Primer 1

```
Poziv: show (1 :/: 2 :/: 3 :/: End)
Izlaz: "[1,2,3]:"
```

Primer 2

```
Poziv: show Empty
Izlaz: "[]:"
```

Primer 3

```
Poziv: (1 :/: 2 :/: 3 :/: End) == (1 :/: 2 :/: 3 :/: End)
Izlaz: True
```

Primer 4

```
Poziv: (1 :/: 2 :/: 3 :/: End) == (1 :/: 3 :/: 2 :/: End)
Izlaz: False
```

Primer 5

```
Poziv: foldr (+) 0 (1 :/: 3 :/: 2 :/: End)
Izlaz: 6
```

Primer 6

```
Poziv: foldr (:) [] (1 :/: 3 :/: 2 :/: End)
Izlaz: [1,2,3]
```

Primer 7

```
Poziv: (1 :/: 3 :/: 2 :/: End) +/+ (4 :/: 5 :/: 6 :/: End)
Izlaz: :[1,2,3,4,5,6]:
```

Primer 8

```
Poziv: End +/+ (1 :/: End)
Izlaz: :[1]:
```

3.4 Ispitni zadaci

Zadatak 3.4.1

Napisati biblioteku funkcija za rad sa bankarskim računima. Račun posmatrati kao par (broj računa, iznos), tipa (String, Int). Implementirati funkcije:

1. **otvori** b br, čiji je tip:
`otvori :: [(String, Int)] -> String -> [(String, Int)]`
koja otvara račun u banci dodavanjem računa sa brojem **br** i iznosom 0 **na početak** liste računa **b**, i vraća potencijalno modifikovanu listu računa (račun se ne otvara ukoliko drugi račun sa istim brojem već postoji u banci);
2. **zatvori** b br, čiji je tip:
`zatvori :: [(String, Int)] -> String -> [(String, Int)]`
koja zatvara račun u banci uklanjanjem računa sa brojem **br** iz liste računa **b**, i vraća potencijalno modifikovanu listu računa;
3. **uplati** b br iznos, čiji je tip:
`uplati :: [(String, Int)] -> String -> Int -> [(String, Int)]`
koja uplaćuje **iznos** na račun **br** iz liste računa **b** i vraća potencijalno modifikovanu listu računa.

Primer 1

```
Poziv: otvori [("acc-007", 5), ("acc-005", 6)] "acc-003"  
Izlaz: [("acc-003", 0), ("acc-007", 5), ("acc-005", 6)]
```

Primer 2

```
Poziv: uplati [("acc-007", 5), ("acc-005", 6)] "acc-007" 5  
Izlaz: [("acc-007", 10), ("acc-005", 6)]
```

Primer 3

```
Poziv: zatvori [("acc-007", 5), ("acc-005", 6)] "acc-007"  
Izlaz: [("acc-005", 6)]
```

Zadatak 3.4.2

Napraviti biblioteku funkcija za rad sa bankarskim transakcijama. Svaka transakcija se sastoji od identifikatora, iznosa i brojeva računa pošiljaoca i primaoca. U okviru biblioteke je neophodno isporučiti tip podataka koji predstavlja transakciju, kolekciju izvršenih transakcija, kao i funkcije za listanje, izvršavanje i poništavanje transakcija. Implementirati (bez rekurzije, koristeći funkcije višeg reda):

1. prikazivi tip podataka `Transakcija` (podrazumevano instancira `Show i Eq`) sa konstruktorom `MkTransakcija` i poljima: `ident` (identifikator, ceo broj, jedinstven za svaku transakciju), `iznos` (ceo broj), `posiljalac` (niska koja predstavlja broj računa pošiljaoca) i `primalac` (niska koja predstavlja broj računa primaoca);
2. tip podataka `AktivneTransakcije` koji predstavlja alias za kolekciju transakcija;
3. funkciju `izlistaj ts br`, čiji je tip:
`izlistaj :: AktivneTransakcije -> String -> [Transakcija]`
 koja vraća sve transakcije iz aktivnih transakcija (`ts`) u kojima figuriše dati broj računa (`br`), bilo kao pošiljalac ili primalac;
4. funkciju `dodaj ts t`, čiji je tip:
`dodaj :: AktivneTransakcije -> Transakcija -> AktivneTransakcije`
 koja dodaje transakciju (`t`) **na početak** spiska aktivnih transakcija (`ts`) i vraća modifikovani spisak transakcija;
5. funkciju `ukloni ts ident`, čiji je tip:
`ukloni :: AktivneTransakcije -> Int -> AktivneTransakcije`
 koja uklanja transakciju sa datim identifikatorom (`ident`) iz spiska aktivnih transakcija (`ts`) i vraća modifikovani spisak transakcija;
6. funkciju `ukupno ts`, čiji je tip:
`ukupno :: AktivneTransakcije -> Int`
 koja računa ukupni iznos svih transakcija.

Primer 1

```
Poziv: izlistaj [(MkTransakcija 1 100 "pera" žika"),  
                (MkTransakcija 2 150 "mara" žika")] "mara"  
Izlaz: [(MkTransakcija 2 150 "mara" žika")]
```

Primer 2

```
Poziv: dodaj [(MkTransakcija 1 100 "pera" žika"),  
              (MkTransakcija 2 150 "mara" žika")]  
Izlaz: [(MkTransakcija 2 150 "mara" žika"),  
        (MkTransakcija 1 100 "pera" žika")]
```

Primer 3

```
Poziv: ukloni [(MkTransakcija 1 100 "pera" žika"),  
               (MkTransakcija 2 150 "mara" žika")] 1  
Izlaz: [MkTransakcija 2 150 "mara" žika"]
```

Primer 4

```
Poziv: ukupno [(MkTransakcija 1 100 "pera" žika"),  
               (MkTransakcija 2 150 "mara" žika")]  
Izlaz: 250
```

Zadatak 3.4.3

Napisati biblioteku dodatnih funkcija za rad sa listama. Implementirati funkcije:

1. funkciju `maxl lst`, čiji je tip:
`maxl :: Ord a => [a] -> a`
 koja vraća maksimalni element neprazne liste `lst`;
2. funkciju `ind x lst`, čiji je tip:
`ind :: Eq a => [a] -> a -> Int`
 koja vraća indeks elementa `x` u listi `lst` (indeks se broji od 1), a inače `-1` ukoliko se `x` ne nalazi u listi `lst`;
3. funkciju `presek lst1 lst2`, čiji je tip:
`presek :: Eq a => [a] -> [a] -> [a]`
 koja vraća presek dve liste (listu elemenata koji se nalaze u obe liste), pritom u listama nema ponavljanja elemenata;
4. funkciju `umetni lst x`, čiji je tip:
`umetni :: [a] -> a -> [a]`
 koja umeće element `x` između svaka dva elementa liste `lst`.

Primer 1

```
Poziv: maxl [2,4,5,4]
Izlaz: 5
```

Primer 2

```
Poziv: maxl "abcd"
Izlaz: 'd'
```

Primer 3

```
Poziv: ind 3 [1..10]
Izlaz: 3
```

Primer 4

```
Poziv: ind 100 [1..10]
Izlaz: -1
```

Primer 5

```
Poziv: presek [1..10] [8..15]
Izlaz: [8,9,10]
```

Primer 6

```
Poziv: presek [1..5] [6..10]
Izlaz: []
```

Primer 7

```
Poziv: umetni [1..5] 3
Izlaz: [1,3,2,3,3,3,4,3,5]
```

Primer 8

```
Poziv: umetni [1] 3
Izlaz: [1]
```

Zadatak 3.4.4

Napisati biblioteku dodatnih funkcija za rad sa stekom. Stek može sadržati proizvoljno mnogo elemenata određenog tipa, i potrebno je pružiti sledeće funkcionalnosti:

1. alias **Stek** **a** za strukturu podataka koji svoje elemente čuva unutar liste, gde je glava liste vrh steka;
2. funkciju **push s x**, čiji je tip:
`push :: Stek a -> a -> Stek a`
 koja ubacuje element **x** na vrh steka **s**;
3. funkciju **top s**, čiji je tip:
`top :: Stek a -> Maybe a`
 koja vraća vrh steka **s** ako stek nije prazan, a inače praznu vrednost;
4. funkciju **pop s**, čiji je tip:
`pop :: Stek a -> (Maybe a, Stek a)`
 koja vraća par (vrh steka **s**, stek **s** bez vrha), dok ukoliko je stek prazan vraća se prazna vrednost za vrh steka;
5. funkciju **stMap s f**, čiji je tip:
`stMap :: Stek a -> (a -> b) -> Stek b`
 koja primenjuje funkciju **f** na sve elemente steka **s**;
6. funkciju **proredi**, čiji je tip:
`proredi :: Stek a -> Stek a`
 koja uklanja svaki element sa neparnim indeksom (indeksi elemenata liste se broje od 0).

Primer 1

```
Poziv: push 4 [1,2,3]
Izlaz: [4,1,2,3]
```

Primer 2

```
Poziv: top [1,2,3]
Izlaz: Just 1
```

Primer 3

```
Poziv: top []
Izlaz: Nothing
```

Primer 4

```
Poziv: pop [1,2,3]
Izlaz: (Just 1, [2,3])
```

Primer 5

```
Poziv: pop []
Izlaz: (Nothing, [])
```

Primer 6

```
Poziv: stMap [1] (+1)
Izlaz: [2]
```

Primer 7

```
Poziv: stMap [1,2,3] show
Izlaz: ["1","2","3"]
```

Primer 8

```
Poziv: proredi [1,2,3]
Izlaz: [1,3]
```

Zadatak 3.4.5

Napisati biblioteku za pojednostavljenu serijalizaciju^a osnovnih struktura podataka u format JSON. Implementirati funkcije:

1. `tupJ t`, čiji je tip:
`tupJ :: (Show a, Show b) => (a,b) -> String`
 koja vraća JSON reprezentaciju para `t` (pretpostaviti da serijalizacija elementa `e` para `t` odgovara pozivu: `show e`);
2. `lstJ lst`, čiji je tip:
`lstJ :: Show a => [a] -> String`
 koja vraća JSON reprezentaciju liste `lst` (pretpostaviti da serijalizacija elementa `e` liste `lst` odgovara pozivu: `show e`);
3. `flstJ p lst`, čiji je tip:
`flstJ :: Show a => ([a] -> Bool) -> [[a]] -> [String]`
 koja vraća listu serijalizovanih listi liste `lst` za koje važi predikat `p`;
4. `zipJ lst`, čiji je tip:
`zipJ :: Show a => [a] -> [(a, String)]`
 koja serijalizuje svaki element `e` liste `lst` u nisku `json` i vraća listu parova `(e, json)` (pretpostaviti da serijalizacija elementa `e` liste `lst` odgovara pozivu: `show e`).

Primer 1

```
Poziv: tupJ (1, [])
Izlaz: "(1,[])"
```

Primer 2

```
Poziv: lstJ [1..5]
Izlaz: "[1,2,3,4,5]"
```

Primer 3

```
Poziv: flstJ null [[], [1,2], [], [3]]
Izlaz: ["[]","[1,2]","[]","[3]"]
```

Primer 4

```
Poziv: flstJ (all (>3)) [], [1,2], [], [3]  
Izlaz: ["[]"]
```

Primer 5

```
Poziv: flstJ (not.null) [], [1,2], [], [3]  
Izlaz: ["[1,2]", "[3]"]
```

Primer 6

```
Poziv: zipJ [], [1,2]  
Izlaz: [([], "[]"), ([1,2], "[1,2]")]
```

^aSerijalizacija predstavlja proces konverzije objekta u niz bajtova kako bi se stanje objekta sačuvalo i objekat mogao ponovo kreirati kasnije.

Zadatak 3.4.6

Kreirati biblioteku za serijalizaciju vremenskih podataka u format JSON. Biblioteka treba da omogući serijalizaciju kroz metod `ser` klase (osobine) `Serializable`. Implementirati:

1. tip podataka `Point` koji predstavlja par realnih koordinata (tipa `Float`) i koji se konstruiše putem konstruktora `MkPoint`;
2. tip podataka `WeatherInfo` koji se sastoji od lokacije (tipa `Point`) i temperature na datoj lokaciji (tipa `Float`) i koji se konstruiše putem konstruktora `MkWeatherInfo`.

Instancirati:

1. klasu `Serializable` za tip `Float`;
2. klasu `Serializable` za tip `Point` kao uređeni par koordinata;
3. klasu `Serializable` za tip `WeatherInfo` kao objekat sa atributima `loc` i `temp`.

Primer 1

```
POZIV: ser 3.2
IZLAZ: "3.2"
```

Primer 2

```
POZIV: ser (MkPoint 45.4 24.3)
IZLAZ: "(45.4,24.3)"
```

Primer 3

```
POZIV: ser (MkWeatherInfo (MkPoint 45.4 24.3) 27.0)
IZLAZ: "{ 'loc': (45.4,24.3), 'temp': 27.0 }"
```

Primer 4

```
POZIV: ser (MkWeatherInfo (MkPoint (-45.4) 24.3) (-27.0))
IZLAZ: "{ 'loc': (-45.4,24.3), 'temp': -27.0 }"
```

Zadatak 3.4.7

Napisati biblioteku koja pruža dodatne operacije za rad sa listama, implementirajući funkcije:

1. `safeHead lst`, čiji je tip:
`safeHead :: [a] -> Maybe a`
 koja vraća glavu liste `lst` ili `Nothing` ukoliko nije moguće izvući glavu liste;
2. `safeTail lst`, čiji je tip:
`safeTail :: [a] -> Either [a] String`
 koja vraća rep liste `lst` ili nisku "Empty" ukoliko nije moguće izvući rep liste;
3. `find e lst`, čiji je tip:
`find :: Eq a => a -> [a] -> Either Int String`
 koja vraća indeks elementa `e` u listi `lst` (počev od 1) ili nisku "404" ako se `e` ne nalazi u `lst`;
4. `foldf acc fs lst`, čiji je tip:
`foldf :: b -> [(b -> a -> b)] -> [a] -> b`
 koja akumulira listu `lst` tako što primenjuje funkcije iz liste `fs` redom na trenutnu vrednost akumulatora i trenutni element liste, i vraća finalnu vrednost akumulatora (ukoliko je broj elemenata listi `fs` i `lst` različit, broj operacija je jednak dužini kraće liste).

Primer 1

```
Poziv: safeHead [1..10]
Izlaz: Just 1
```

Primer 2

```
Poziv: safeHead []
Izlaz: Nothing
```

Primer 3

```
Poziv: safeTail [1..5]
Izlaz: Left [2,3,4,5]
```

Primer 4

```
Poziv: safeTail []
Izlaz: Right "Empty"
```

Primer 5

```
Poziv: find 3 [1..10]
Izlaz: Left 3
```

Primer 6

```
Poziv: find 100 [1..10]
Izlaz: Right "404"
```

Primer 7

```
Poziv: foldf 1 [(+),(*)]
      [5,10]
Izlaz: 60
```

Primer 8

```
Poziv: foldf 1 [(+),(*),(+)]
      [1..5]
Izlaz: 7
```

Zadatak 3.4.8

Napisati biblioteku koja pruža dodatne operacije za rad sa listama:

1. funkciju `takeCycled n lst`, čiji je tip:
`takeCycled :: Int -> [a] -> [a]`
 koja vraća prvih `n` elemenata liste dobijene cikličnim ponavljanjem liste `lst`;
2. funkciju `dropUntil p lst`, čiji je tip:
`dropUntil :: (a -> Bool) -> [a] -> [a]`
 koja uklanja elemente sa početka liste sve dok se ne zadovolji predikat `p`;
3. funkciju `sumLst lst`, čiji je tip: `sumLst :: Num a => [a] -> a`
 koja vraća sumu liste `lst` **bez** korišćenja rekurzije;
4. funkciju `split sep str`, čiji je tip:
`split :: Char -> String -> [String]`
 koja vraća listu podniski niske `str` razdvojene na mestima pojavljivanja karaktera (separatora) `sep`;
5. funkciju `join sep lst`, čiji je tip:
`join :: String -> [String] -> [String]`
 koja vraća nisku dobijenu nadovezivanjem niski iz liste `lst` sa separatorom `sep` umetnutim između svaka dva elementa (inverz `split` funkcije).

Primer 1

```
Poziv: takeCycled 5 [1,2]
Izlaz: [1,2,1,2,1]
```

Primer 2

```
Poziv: sumLst [1..5]
Izlaz: 15
```

Primer 3

```
Poziv: dropUntil (>2) [1,2,3,4,5]
Izlaz: [3,4,5]
```


Primer 4

```
Poziv: split ', ' "jedan,dva,,tri"  
Izlaz: ["jedan","dva",,"tri"]
```

Primer 5

```
Poziv: split '!' "jedan,dva,tri"  
Izlaz: ["jedan,dva,tri"]
```

Primer 6

```
Poziv: join "->" ["1","2","3","4","5"]  
Izlaz: "1->2->3->4->5"
```

Zadatak 3.4.9

Napisati biblioteku koja pomaže pri radu sa oblicima u 2D ravni. Implementirati:

1. tip podataka `Point2D` koji se može konstruisati putem konstruktora `MkPoint` i sadrži dva atributa `x` i `y`, tipa `Float`, kao i funkcije `x` i `y` koje služe kao *geteri* za polja `x` i `y`;
2. prikaz i poređenje — podrazumevano instancirati `Show` i `Eq` nad tipom `Point2D`;
3. funkciju `point p`, čiji je tip: `point :: (Float,Float) -> Point2D`
koja pravi tačku od prosleđenog para koordinata
4. funkciju `line p1 p2`, čiji je tip:
`line :: Point2D -> Point2D -> (Float,Float)`
koja vraća par (k,n) gde su k i n parametri prave $y = kx + n$ koja prolazi kroz tačke `p1` i `p2` (pretpostaviti da rezultat neće biti prava sa beskonačnim koeficijentom pravca);
5. funkciju `dist p1 p2`, čiji je tip:
`dist :: Point2D -> Point2D -> Float`
koja vraća euklidsko rastojanje između tačaka `p1` i `p2`;
6. funkciju `maxP f lst`, čiji je tip:
`maxP :: (Point2D -> Float) -> [Point2D] -> Float`
koja vraća maksimalnu vrednost od svih vrednosti dobijenih primenom funkcije `f` na tačke iz *neprazne* liste `lst`.

Primer 1

```
Poziv: line (MkPoint 2.0 0.0) (MkPoint 3.0 3.0)
Izlaz: (1.0,2.0)
```

Primer 2

```
Poziv: line (point (1.0,1.0)) (point (2.0,1.0))
Izlaz: (0.0,1.0)
```

Primer 3

```
Poziv: dist (MkPoint 1.0 1.0) (MkPoint 2.0 1.0)
Izlaz: 1.0
```

Primer 4

```
POZIV: dist (MkPoint 3.0 4.0) (MkPoint 0.0 0.0)
IZLAZ: 5.0
```

Primer 5

```
POZIV: maxP x [MkPoint 1.0 1.0, MkPoint 2.0 1.0]
IZLAZ: 2.0
```

Primer 6

```
POZIV: maxP y [MkPoint 1.0 1.0, MkPoint 2.0 1.0]
IZLAZ: 1.0
```

Zadatak 3.4.10

Napisati biblioteku funkcija koja će podržati implementacije raznih kartaških igara. Potrebno je modelirati kartu, skup izvučenih karata (u daljem tekstu *ruka*), kao i funkcije za rad sa definisanim modelom. Svaka karta se sastoji od znaka (*herc*, *karo*, *pik*, *tref*) vrednosti (od 2 do 10 za karte bez slike, dok žandar ima vrednost 12, kraljica 13, kralj 14, a kec 15).

Karte se porede po njihovim vrednostima, a u slučaju da su vrednosti iste, onda se posmatraju znakovi po sledećem poretku (od najmanjeg ka najvećem): *herc*, *karo*, *pik*, *tref*. Na primer, trojka tref je manja od petice herc, ali je trojka herc manja od trojke tref.

1. Svaku kartu posmatrati kao par gde levi element predstavlja celobrojnu vrednost karte a desni predstavlja znak (prvi karakter znaka, dakle: 'H', 'K', 'P', 'T'). Implementirati funkcije:

- (a) dodaj *ruka karta*, čiji je tip:

```
dodaj :: [(Int, Char)] -> (Int, Char) -> [(Int, Char)]
```

koja u trenutnu *sortiranu* ruku, dodaje novu kartu tako da je sortirani poredak održan i vraća potencijalno modifikovanu ruku (ako karta već postoji u ruci ne raditi ništa);

- (b) ukloni *ruka karta*, čiji je tip:

```
ukloni :: [(Int, Char)] -> (Int, Char) -> [(Int, Char)]
```

koja iz ruke uklanja odgovarajuću kartu i vraća potencijalno novu ruku (ako karta ne postoji u ruci ne raditi ništa);

- (c) `uporedi ruka1 ruka2`, čiji je tip:
`uporedi :: [(String, Int)] -> [(String, Int)]`
`-> Ordering`

koja poredi dve ruke i vraća poredak `LT` ako je prva ruka u leksikografskom poretku pre druge, `GT` ako je prva ruka u leksikografskom poretku posle druge, ili `EQ` ukoliko su jednake leksikografski (pretpostaviti da je broj karata u obe ruke jednak).

Primer 1

POZIV: `dodaj [(2, 'H'), (3, 'T'), (14, 'P')] (10, 'K')`
IZLAZ: `[(2, 'H'), (3, 'T'), (10, 'K'), (14, 'P')]`

Primer 2

POZIV: `ukloni [(2, 'H'), (3, 'T'), (10, 'K'), (14, 'P')] (10, 'K')`
IZLAZ: `[(2, 'H'), (3, 'T'), (14, 'P')]`

Primer 3

POZIV: `uporedi [(2, 'H'), (3, 'T'), (10, 'K'), (14, 'P')]`
`[(2, 'H'), (3, 'T'), (9, 'K'), (14, 'P')]`
IZLAZ: `GT`

2. Implementirati:

- (a) tip podataka **Znak** koji predstavlja znak karte — može se konstruisati konstruktorima `Herc`, `Karo`, `Pik` ili `Tref`;
- (b) prikazivi i jednačivi tip podataka **Karta** (podrazumevano instancira `Show` i `Eq`) sa konstruktorom `MkKarta` i poljima: **vrednost** (vrednost karte, ceo broj) i **znak** (**Znak** karte);
- (c) tip podataka **Ruka** koji predstavlja alias za listu karata;
- (d) funkciju `izlistaj znak ruka`, čiji je tip:
`izlistaj :: Znak -> Ruka -> [Karta]`
 koja vraća sve karte iz ruke (`ruka`) koje su odgovarajućeg znaka (`znak`);

- (e) dodaj ruka karta, čiji je tip:
dodaj :: Ruka -> Karta -> Ruka
koja u trenutnu sortiranu ruku dodaje novu kartu tako da je sortirani poredak održan i vraća potencijalno modifikovanu ruku (ako karta već postoji u ruci ne raditi ništa);
- (f) najjaca ruka, čiji je tip:
najjaca :: Ruka -> Karta
koja vraća najveću kartu (maksimalnu u sortiranom poretku) iz ruke ruka.

Primer 4

```
POZIV: izlistaj Pik  
      [MkKarta 3 Pik, MkKarta 2 Pik, MkKarta 10 Karo]  
IZLAZ: [MkKarta 3 Pik, MkKarta 2 Pik]
```

Primer 5

```
POZIV: dodaj [MkKarta 2 Pik, MkKarta 4 Pik, MkKarta 10 Karo]  
      (MkKarta 1 Pik)  
IZLAZ: [MkKarta 1 Pik, MkKarta 2 Pik, MkKarta 4 Pik,  
      MkKarta 10 Karo]
```

Primer 6

```
POZIV: najjaca [MkKarta 2 Pik, MkKarta 4 Pik, MkKarta 10 Karo]  
IZLAZ: MkKarta 10 Karo
```

Zadatak 3.4.11

Napisati biblioteku funkcija za olakšavanje rada s niskama. Neke od čestih operacija uključuju umanjivanje karaktera unutar niske, razdvajanje i spajanje niski u odnosu na karakter ili nisku koja predstavlja separator.

Implementirati funkcije:

1. `najduza lst`, čiji je tip:
`najduza :: [String] -> String`
 koja pronalazi najdužu nisku iz liste `lst`. Ako je više niski jednake dužine, vratiti **prvu**. Pretpostaviti da lista `lst` ima barem jedan element;
2. `umanji str`, čiji je tip:
`umanji :: String -> String`
 koja vraća nisku koja sadrži transformisane karaktere niske `str` — velika slova prebacuje u mala, dok ostale karaktere ne modifikuje;
3. `razdvoj sep str`, čiji je tip:
`razdvoj :: Char -> String -> [String]`
 koja vraća listu podniski niske `str` razdvojene na mestima pojavljivanja karaktera (separatora) `sep`;
4. `spoj sep lst`, čiji je tip:
`spoj :: String -> [String] -> String`
 koja vraća nisku dobijenu nadovezivanjem niski iz liste `lst` sa separatorom `sep` umetnutim između svaka dva elementa (inverz `razdvoj` funkcije).

Primer 1

```
Poziv: najduza ["cvet", "cvetic", "lala"]
Izlaz: "testtt"
```

Primer 2

```
Poziv: umanji "1 cveT,saT"
Izlaz: "1 abcd,efg"
```

Primer 3

```
Poziv: umanji "12345"
Izlaz: "12345"
```

Primer 4

```
Poziv: razdvoj ', ' "jedan,dva,tri,cetiri"
Izlaz: ["jedan","dva","tri","četiri"]
```

Primer 5

```
Poziv: razdvoj '!' "jedan,dva"
Izlaz: ["jedan,dva"]
```

Primer 6

```
Poziv: spoj "->" ["1","2","3"]
Izlaz: "1->2->3"
```

Zadatak 3.4.12

Napisati biblioteku funkcija za pretragu karaktera u niskama. Potrebno je isporučiti tip **Poklapanje** koji će predstavljati jedno pojavljivanje karaktera u nisci i sadržati u sebi informaciju o poziciji karaktera. Nad ovim tipom je neophodno implementirati dodatne funkcije za pretragu.

Implementirati:

1. prikazivi tip podataka **Poklapanje** (podrazumevano instancira **Show**) sa konstruktorom **Od** i poljima: **kar** (predstavlja karakter) i **poz** (predstavlja poziciju karaktera u nisci); postarati se da postoje funkcije **kar** i **poz** koje vraćaju vrednosti polja objekta tipa **Poklapanje**
2. funkciju **poklapanjeShow p**, čiji je tip:
`poklapanjeShow :: Poklapanje -> String`
 koja vraća nisku u formatu **karakter (pozicija)** za dato **poklapanje p**, npr: **"k (12)"**;
3. funkciju **poklapanjeM i str**, čiji je tip:
`poklapanjeM :: Int -> String -> Maybe Poklapanje`
 koja vraća **poklapanje karaktera na indeksu i u nisci str** ukoliko je **i** unutar granica niske **str**, a inače praznu vrednost;
4. funkciju **poklapanjeE i str**, čiji je tip:
`poklapanjeE :: Int -> String -> Either String Poklapanje`
 koja vraća **poklapanje karaktera na indeksu i u nisci str** ukoliko je **i** unutar granica niske **str**, a inače vraća nisku: **"Index error"**;
5. funkciju **pronadjiM k str**, čiji je tip:
`pronadjiM :: Poklapanje -> String -> Maybe Bool`
 koja, ukoliko je pozicija karaktera iz **poklapanja p** u granicama niske **str**, vraća indikator da li se taj **poklapanje** nalazi na poziciji iz **poklapanja p** u nisci **str**, a inače praznu vrednost;

6. funkciju `pronadjiE` k `str`, čiji je tip:

```
pronadjiE :: Poklapanje -> String -> Either String
Bool
```

koja, ukoliko je pozicija karaktera iz poklapanja `p` u granicama niske `str`, vraća indikator da li se taj karakter nalazi na poziciji iz poklapanja `p` u nisci `str`, a inače vraća nisku: `"Index error"`.

Primer 1

```
Poziv: poklapanjeShow
      (0d 'a' 2)
Izlaz: "a (2)"
```

Primer 2

```
Poziv: poklapanjeShow
      (0d 't' 24)
Izlaz: "t (24)"
```

Primer 3

```
Poziv: poklapanjeM 1 "abc"
Izlaz: Just (0d 'b' 1)
```

Primer 4

```
Poziv: poklapanjeM (-2) "abc"
Izlaz: Nothing
```

Primer 5

```
Poziv: poklapanjeE 1 "abc"
Izlaz: "Right (0d 'b' 1)"
```

Primer 6

```
Poziv: poklapanjeE (-2) "abc"
Izlaz: Left "Index error"
```

Primer 7

```
Poziv: pronadjiM (0d 'b' 1)
      "abc"
Izlaz: Just True
```

Primer 8

```
Poziv: pronadjiM (0d 'b' 2)
      "abc"
Izlaz: Just False
```

Primer 9

```
Poziv: pronadjiM (0d 'b' 20)
      "abc"
Izlaz: Nothing
```

Primer 10

```
Poziv: pronadjiE (0d 'b' 1)
      "abc"
Izlaz: Right True
```

Primer 11

```
Poziv: pronadjiM (0d 'b' 2)
      "abc"
Izlaz: Right False
```

Primer 12

```
Poziv: pronadjiE (0d 'b' 20)
      "abc"
Izlaz: Left "Index error"
```


Zadatak 3.4.13

U okviru jednog programskog rešenja su identifikovane česte operacije nad listama: pronalazak maksimuma, traženje indeksa elementa u listi i slično. Osim toga, u okviru projekta se umesto stekova koriste obične liste. Primećena je mogućnost generalizacije pomenutih koncepata u jednu zasebnu biblioteku koja bi se koristila kroz projekat i korisnicima pružila transparentni interfejs za rad sa listama ili stekovima. Implementirati:

1. funkciju `max1 lst`, čiji je tip:
`max1 :: Ord a => [a] -> a`
 koja vraća maksimalni element liste `lst`;
2. funkciju `ind x lst`, čiji je tip:
`ind :: Eq a => [a] -> a -> Int`
 koja vraća indeks elementa `x` u listi `lst` (indeks se broji od 1), a inače `-1` ukoliko se `x` ne nalazi u listi `lst`;
3. funkciju `presek lst1 lst2`, čiji je tip:
`presek :: Ord a => [a] -> [a] -> [a]`
 koja vraća *sortiranu* listu koja predstavlja presek dve *sortirane* liste (elemenata koji se nalaze u obe liste), pritom u listama `lst1` i `lst2` nema ponavljanja elemenata;
4. tip podataka (dozvoljen je i alias) `Stek a` koji će svoje elemente čuvati unutar liste, glava liste će biti vrh steka;
5. funkciju `push s x`, čiji je tip:
`push :: Stek a -> a -> Stek a`
 koja ubacuje element `x` na vrh steka `s`;
6. funkciju `top s`, čiji je tip:
`top :: Stek a -> Maybe a`
 koja vraća vrh steka `s` ako stek nije prazan, a inače praznu vrednost;
7. funkciju `pushMP lst1 lst2 s`, čiji je tip:
`pushMP :: Ord a => [a] -> [a] -> Stek a -> Stek a`
 koja maksimalni element preseka *sortiranih* listi `lst1` i `lst2` gura na vrh steka `s`, pritom u listama `lst1` i `lst2` nema ponavljanja elemenata.

Primer 1

```
Poziv: maxl [2,3,4,5,1,2,3,4]  
Izlaz: 5
```

Primer 2

```
Poziv: maxl "abcd"  
Izlaz: 'd'
```

Primer 3

```
Poziv: ind 3 [1..10]  
Izlaz: 3
```

Primer 4

```
Poziv: ind 100 [1..10]  
Izlaz: -1
```

Primer 5

```
Poziv: presek [1..10] [8..15]  
Izlaz: [8,9,10]
```

Primer 6

```
Poziv: presek [1..5] [6..10]  
Izlaz: []
```

Primer 7

```
Poziv: presek [] [1..10]  
Izlaz: []
```

Primer 8

```
Poziv: push [1,2,3] 4  
Izlaz: [4,1,2,3]
```

Primer 9

```
Poziv: push [] 1  
Izlaz: [1]
```

Primer 10

```
Poziv: top [1,2,3]  
Izlaz: Just 1
```

Primer 11

```
Poziv: top []  
Izlaz: Nothing
```

Primer 12

```
Poziv: top [(1,1)]  
Izlaz: Just (1,1)
```

Primer 13

```
Poziv: pushMP [1,2] [2,4] [9]  
Izlaz: [2,9]
```

Primer 14

```
Poziv: pushMP [] [2,4] [9]  
Izlaz: [9]
```

Primer 15

```
Poziv: pushMP [1] [1] [2]  
Izlaz: [1,2]
```

Primer 16

```
Poziv: pushMP [3] [1..5] []  
Izlaz: [3]
```

Zadatak 3.4.14

Napisati biblioteku funkcija koje pružaju korisniku rad sa prozorima — tzv. *menadžer prozora*. Menadžer prozora treba da pruži korisniku mogućnost otvaranja novog prozora, selektovanja aktivnog prozora, zatvaranja prozora, kao i cikličnog pomeranja kroz prozore. Prozori se čuvaju u listi i reprezentuju se kao par (`id_prozora`, `naslov_prozora`). Aktivni prozor držati u glavi liste prozora zarad efikasnog pristupa/zamene.

Implementirati funkcije (sve funkcije vraćaju ažuriranu listu prozora):

1. `aktivni lst`, čiji je tip:
`aktivni :: [(Int, String)] -> String`
koja vraća naslov trenutno aktivnog prozora u listi `lst`;
2. `otvori x str lst`, čiji je tip:
`otvori :: Int -> String -> [(Int, String)] -> [(Int, String)]`
koja novo-otvoreni prozor sa identifikatorom `x` i naslovom `str` dodaje u listu prozora `lst` i postavlja taj prozor kao trenutno aktivni prozor. Ako se prozor sa datim identifikatorom već nalazi u listi, ne raditi ništa;
3. `zatvori x lst`, čiji je tip:
`zatvori :: Int -> [(Int, String)] -> [(Int, String)]`
koja iz liste prozora `lst` uklanja prozor sa identifikatorom `x`;
4. `tabUnapred n lst`, čiji je tip:
`tabUnapred :: [(Int, String)] -> Int -> [(Int, String)]`
koja ciklično pomera listu aktivnih prozora za `n` mesta ulevo, tako da prozor na mestu `n` postane aktivni prozor;
5. `zatvoriSve str lst`, čiji je tip:
`zatvoriSve :: String -> [(Int, String)] -> [(Int, String)]`
koja iz liste prozora `lst` uklanja sve prozore koji u svom naslovu sadrže nisku `str` (bez obzira na mala/velika slova);
6. `fokusiraj x lst`, čiji je tip:
`fokusiraj :: Int -> [(Int, String)] -> [(Int, String)]`
koja pomera prozor sa identifikatorom `x` iz liste prozora `lst` na

mesto aktivnog prozora. Redosled ostalih prozora ostaje isti. Ako se *x* ne nalazi u listi prozora, ne raditi ništa.

Primer 1

```
Poziv: aktivni [(1,"vi"),(5,"VS")]
Izlaz: "vi"
```

Primer 2

```
Poziv: otvori 1 "IDEA" [(2,"vim")]
Izlaz: [(1,"IDEA"),(2,"vim")]
```

Primer 3

```
Poziv: otvori 1 "IDEA" [(1,"vim")]
Izlaz: [(1,"vim")]
```

Primer 4

```
Poziv: zatvori 3 [(1,"vim"),(2,"IDEA"),(3,"sh")]
Izlaz: [(1,"vim"),(2,"IDEA")]
```

Primer 5

```
Poziv: tabUnapred 2 [(1,"vim"),(2,"IDEA"),(3,"sh")]
Izlaz: [(3,"sh"),(1,"vim"),(2,"IDEA")]
```

Primer 6

```
Poziv: tabUnapred 5 [(1,"vim"),(2,"IDEA"),(3,"sh")]
Izlaz: [(3,"sh"),(1,"vim"),(2,"IDEA")]
```

Primer 7

```
Poziv: zatvoriSve "ViM"
      [(1,"nvim"),(2,"IDEA"),(3,"VIM"),(4,"vi")]
Izlaz: [(2,"IDEA"),(4,"vi")]
```

Primer 8

```
Poziv: zatvoriSve "vi" [(1,"nvim"),(2,"IDEA"),(3,"vim"),(4,"vi")]
Izlaz: [(2,"IDEA")]
```

Primer 9

```
Poziv: fokusiraj 3 [(1,"nvim"),(2,"IDEA"),(3,"vim")]
Izlaz: [(3,"vim"),(1,"nvim"),(2,"IDEA")]
```

Zadatak 3.4.15

Napisati biblioteku funkcija koje pružaju korisniku rad sa prozorima — tzv. *menadžer prozora*. Potrebno je isporučiti tip `Prozor` i tip `Prozori`, kao i funkciju za otvaranje novog prozora.

Implementirati:

1. tip podataka `Prozor` sa konstruktorom `MkProzor` i poljima: `ind` (predstavlja indeks prozora) i `naziv` (predstavlja naziv prozora); postarati se da postoje funkcije `ind` i `naziv` koje vraćaju vrednosti polja objekta tipa `Prozor`;
2. tip `Prozori` koji predstavlja listu prozora (tipa `Prozor`);
3. klasu `Ord` za tip `Prozor` tako da se prozori porede po indeksu;
4. klasu `Show` za tip `Prozor` tako da se prozor prikaže u formatu `[indeks]: naziv`;
5. funkciju `otvori` str `lst`, čiji je tip:
`otvori :: String -> Prozori -> Prozori`
koja kreira `Prozor` sa naslovom `str` i prvim slobodnim pozitivnim indeksom koji nije zauzet (prozor sa tim indeksom se ne nalazi u listi `lst`) i postavlja ga kao aktivni prozor. Funkcija vraća izmenjenu listu prozora.

Primer 1

```
Poziv: show (MkProzor 1 "IDEA")
Izlaz: "[1]: IDEA"
```

Primer 2

```
Poziv: (MkProzor 1 "sh") > (MkProzor 2 "sh")
Izlaz: False
```

Primer 3

```
Poziv: otvori "vim" [(1,"IDEA"),(3,"sh")]
Izlaz: [(2,"vim"),(1,"IDEA"),(3,"sh")]
```

Zadatak 3.4.16

Napisati biblioteku funkcija koje pružaju korisniku podršku za rad sa prostim brojevima. Implementirati finkcije:

1. `delioci n`, čiji je tip:
`delioci :: Int -> [Int]`
 koja vraća rastuće sortiranu listu pravih delilaca broja `n`;
2. `prost n`, čiji je tip:
`prost :: Int -> Bool`
 koja vraća `True` ukoliko je `n` prost, a `False` inače;
3. `generisiProste n`, čiji je tip:
`generisiProste :: Int -> [Int]`
 koja generiše proste brojeve u intervalu `[2..n]`;
4. `sumaProstih a b`, čiji je tip:
`sumaProstih :: Int -> Int -> Int`
 koja vraća sumu prostih brojeva u intervalu `[a..b]`;
5. `faktori n`, čiji je tip:
`faktori :: Int -> [Int]`
 koja vraća listu uređenih parova koji predstavljaju proste faktore broja `n` kao i broj njihovih ponavljanja.

Primer 1

Poziv: `delioci 2`
Izlaz: `[]`

Primer 2

Poziv: `delioci 20`
Izlaz: `[2,4,5,10]`

Primer 3

Poziv: `prost 17`
Izlaz: `True`

Primer 4

Poziv: `prost 4`
Izlaz: `False`

Primer 5

Poziv: `prost 9929`
Izlaz: `True`

Primer 6

Poziv: `generisiProste 10`
Izlaz: `[2,3,5,7]`

Primer 7

Poziv: `sumaProstih 2 10`
Izlaz: `17`

Primer 8

Poziv: `sumaProstih 8 10`
Izlaz: `0`

Primer 9

Poziv: `faktori 50`
Izlaz: `[(2,1),(5,2)]`

Zadatak 3.4.17

Napisati biblioteku funkcija koje pružaju korisniku podršku za rad sa geometrijskim oblicima. Implementirati:

1. tip podataka `Oblik` koji se može konstruisati pomoću konstruktora (**bez** geter funkcija za polja):

- (a) `Krug r` — predstavlja krug poluprečnika `r`;
- (b) `Trougao a` — predstavlja jednakostranični trougao stranice `a`;
- (c) `Prav a b` — predstavlja pravougaonik sa stranicama `a` i `b`;
- (d) `Kvadrat a` — predstavlja kvadrat stranice `a`.

(Dužine stranica modelovati tipom `Float`)

2. podrazumevano instanciranje klase `Show` za `Oblik`;
3. klasu `PlanarniOblik` koja se sastoji od funkcija `obim` i `povrsina`;
4. instanciranje klase `PlanarniOblik` za `Oblik` tako da funkcije `obim` i `povrsina` vrate ispravne vrednosti za odgovarajuće vrste `Oblik`-a (`Krug`, `Trougao` itd.);
5. funkciju `ukupnaP lst`, čiji je tip:
`ukupnaP :: (PlanarniOblik a) => [a] -> Float`
 koja vraća ukupnu površinu svih oblika iz liste `lst`.

Primer 1

```
Poziv: show (Krug 3)
Izlaz: "Krug 3.0"
```

Primer 2

```
Poziv: show (Prav 3 4)
Izlaz: "Prav 3.0 4.0"
```

Primer 3

```
Poziv: obim (Prav 3 4)
Izlaz: 14.0
```

Primer 4

```
Poziv: ukupnaP []
Izlaz: 0.0
```

Primer 5

```
Poziv: ukupnaP [Prav 3 4, Krug 1]
Izlaz: 15.141593
```

Zadatak 3.4.18

Napisati biblioteku funkcija za rad sa bojama. Svaka boja se opisuje pomoću tri cela broja u opsegu $[0, 255]$ koji opisuju crvenu, zelenu i plavu komponentu boje.

NAPOMENA: Komponenta *k* se u funkcijama ispod zadaje kao karakter 'r', 'g' ili 'b'. Sve ostale karaktere tretirati kao nevalidne komponente. Pretpostaviti da su na ulazu samo mala slova. Za nevalidne komponente vratiti vrednost 0.

Implementirati funkcije:

- komponenta *k* *b*, čiji je tip:
`komponenta :: Char -> (Int, Int, Int) -> Int`
 koja vraća vrednost komponente *k* boje *b*;
- ponisti *k* *b*, čiji je tip:
`ponisti :: Char -> (Int, Int, Int) -> (Int, Int, Int)`
 koja poništava komponentu *k* boje *b* (postavlja njenu vrednost na 0, za nevalidne komponente vratiti neizmenjen argument);
- pomesaj *b1* *b2*, čiji je tip:
`pomesaj :: (Int, Int, Int) -> (Int, Int, Int) -> (Int, Int, Int)`
 koja formira novu boju tako što sabere odgovarajuće vrednosti komponenti boja *b1* i *b2* (ukoliko komponente u zbiru prelaze maksimalnu vrednost za komponentu boje, rezultat njihovog zbira je maksimalna vrednost komponente);
- dodaj *k* *v* *b*, čiji je tip:
`dodaj :: Char -> Int -> (Int, Int, Int) -> (Int, Int, Int)`
 koja dodaje vrednost *v* na komponentu *k* boje *b* (ukoliko komponente u zbiru prelaze maksimalnu vrednost za komponentu boje, rezultat njihovog zbira je maksimalna vrednost komponente; za nevalidne komponente vratiti neizmenjen argument);
- lbFilter *k* *v* boje, čiji je tip:
`lbFilter :: Char -> Int -> [(Int, Int, Int)] -> [(Int, Int, Int)]`
 koja iz liste boje uklanja one boje (održavajući originalni redosled) čija je vrednost komponente *k* manja od *v* (za nevalidne komponente vratiti neizmenjen argument).

Pretpostaviti da su argumenti funkcija koji predstavljaju boje ispravni, tj. da su vrednosti za komponente boje u opsegu [0, 255]. Rezultat rada funkcija mora biti boja koja takođe zadovoljava ovo svojstvo.

Primer 1

```
Poziv: komponenta 'r'  
(123, 122, 121)  
Izlaz: 123
```

Primer 2

```
Poziv: komponenta 'z'  
(123, 122, 121)  
Izlaz: 0
```

Primer 3

```
Poziv: ponisti 'g'  
(123, 10, 30)  
Izlaz: (123, 0, 30)
```

Primer 4

```
Poziv: pomesaj (10, 10, 10)  
(200, 200, 200)  
Izlaz: (210, 210, 210)
```

Primer 5

```
Poziv: pomesaj (100, 100, 100)  
(200, 200, 200)  
Izlaz: (255, 255, 255)
```

Primer 6

```
Poziv: dodaj 'r' 10  
(123, 123, 123)  
Izlaz: (133, 123, 123)
```

Primer 7

```
Poziv: dodaj 'r' 10  
(250, 100, 100)  
Izlaz: (255, 100, 100)
```

Primer 8

```
Poziv: lbFilter 'b' 200  
[(210, 10, 10)]  
Izlaz: []
```

Primer 9

```
Poziv: lbFilter 'r' 100 [(50, 20, 20), (150, 0, 0)]  
Izlaz: [(150, 0, 0)]
```

Zadatak 3.4.19

Napisati biblioteku funkcija za rad sa bojama. Svaka boja se opisuje pomoću tri cela broja u opsegu $[0, 255]$ koji opisuju crvenu, zelenu i plavu komponentu boje.

Implementirati:

1. implicitno prikazivi i jednačivi suma tip *Komponenta* (podrazumevano instancira *Show* i *Eq*) sa konstruktorima *R*, *G* i *B* (konstruktori nemaju argumente);
2. implicitno prikazivi i jednačivi tip *Boja* (podrazumevano instancira *Show* i *Eq*) sa poljima polja *r*, *g* i *b* (tipa *Int*) i ternarnim konstruktorom *MkBoja* koji prima celobrojne vrednosti za crvenu, zelenu i plavu komponentu boje, redom;
3. funkciju *komponenta k b*, čiji je tip:
`komponenta :: Komponenta -> Boja -> Int`
koja vraća vrednost komponente *k* boje *b*;
4. funkciju *ponisti k b*, čiji je tip:
`ponisti :: Komponenta -> Boja -> Boja`
koja poništava komponentu *k* boje *b* (postavlja njenu vrednost na 0);
5. funkciju *pomesaj b1 b2*, čiji je tip:
`pomesaj :: Boja -> Boja -> Maybe Boja`
koja formira novu boju tako što sabere odgovarajuće vrednosti komponenti boja *b1* i *b2*, a praznu vrednost ukoliko se desilo prekoračenje opsega za bilo koju od komponenti;
6. funkciju *maska komponente boja*, čiji je tip:
`maska :: [Komponenta] -> Boja -> Boja`
koja iz boje *b* poništava komponente (postavlja im vrednosti na 0) koje se ne nalaze u listi *komponente*.

Pretpostaviti da su argumenti funkcija koji predstavljaju boje ispravni, tj. da su vrednosti za komponente boje u opsegu $[0, 255]$. Rezultat rada funkcija mora biti boja koja takođe zadovoljava ovo svojstvo.

Primer 1

POZIV: `show (MkBoja 10 20 30)`

IZLAZ: `"MkBoja {r=10, g=20, b=30}"`

Primer 2

POZIV: komponenta R
(MkBoja 10 20 30)
IZLAZ: 10

Primer 3

POZIV: komponenta R
(MkBoja 10 20 30)
IZLAZ: 10

Primer 4

POZIV: ponisti R (MkBoja 10 20 30)
IZLAZ: MkBoja {r=0, g=20, b=30}

Primer 5

POZIV: pomesaj (MkBoja 10 10 10) (MkBoja 200 200 200)
IZLAZ: Just (MkBoja {r=210, g=210, b=210})

Primer 6

POZIV: pomesaj (MkBoja 100 100 100) (MkBoja 200 200 200)
IZLAZ: Nothing

Primer 7

POZIV: maska [B, G, R] (MkBoja 30 30 30)
IZLAZ: MkBoja {r=30, g=30, b=30}

Primer 8

POZIV: maska [R] (MkBoja 200 200 200)
IZLAZ: MkBoja {r=200, g=0, b=0}

Primer 9

POZIV: maska [G, R] (MkBoja 200 200 200)
IZLAZ: MkBoja {r=200, g=200, b=0}

Primer 10

POZIV: maska [G, G, R] (MkBoja 200 200 200)
IZLAZ: MkBoja {r=200, g=200, b=0}

Zadatak 3.4.20

Dat je spisak bodova studenata sa više kolokvijuma. Napisati funkcije:

1. `saberiKolokvijume :: [[Int]] -> [Int]`
koja za svakog studenta sabira bodove sa svih kolokvijuma;

Primer 1

```
Poziv: saberiKolokvijume [[10,5,5],[8,7,10],[0,0,0]]
Izlaz: [20,25,0]
```

Primer 2

```
Poziv: saberiKolokvijume [[3,3,4],[7,0,8]]
Izlaz: [10,15]
```

2. `maksimum :: [Int] -> Int`
koja vraća najveći broj bodova u listi, ili 0 ako je lista prazna;

Primer 3

```
Poziv: maksimum [20,25,0]
Izlaz: 25
```

Primer 4

```
Poziv: maksimum [10,10,10]
Izlaz: 10
```

3. `normalizuj :: [Int] -> [Float]`
koja normalizuje bodove tako da se izraze kao procenat u odnosu na najveći broj bodova. Ako je maksimalan broj bodova 0, svi rezultati su 0;

Primer 5

```
Poziv: normalizuj [20,25,0]
Izlaz: [80.0,100.0,0.0]
```

Primer 6

```
Poziv: normalizuj [10,15,5]
Izlaz: [66.67,100.0,33.33]
```

4. `boljiOdProcentualno :: [Int] -> [Float]`
koja za svakog studenta vraća procenat studenata koje je on nadmašio po ukupnim bodovima. Ako je lista prazna, vraća praznu listu;

Primer 7

```
Poziv: boljiOdProcentualno [20,25,0]
Izlaz: [33.3333,66.6667,0.0]
```

Primer 8

```
Poziv: boljiOdProcentualno [10,15,5,15]  
Izlaz: [25.0,50.0,0.0,50.0]
```

5. prilagodiPoene :: [Int] -> [Float]

koja za svakog studenta prilagođava njegove ukupne bodove po sledećoj krivoj:

- Prvo se za svaki rezultat izračuna kvadratni koren, što znači da se niži rezultati "pojačavaju", a viši se blago smanjuju u odnosu na originalnu skalu.
- Nakon toga se svi rezultati skaliraju tako da najveći rezultat postane 100, a ostali se proporcionalno prilagođavaju.
- Ako je lista prazna ili svi rezultati su 0, vraća se prazna lista.

Primer 9

```
Poziv: prilagodiPoene [20,25,0]  
Izlaz: [89.4427,100.0,0.0]
```

Primer 10

```
Poziv: prilagodiPoene [10,15,5,15]  
Izlaz: [81.6497,100.0,57.7350,100.0]
```

Zadatak 3.4.21

Definisati tip `DnevniPodaci` koji predstavlja podatke o jednom danu: `temperatura :: Int` i `padavine :: Int`. Napisati funkcije:

1. `suviDani :: [DnevniPodaci] -> [Int]`
koja vraća temperaturu dana bez padavina;

Primer 1

```
POZIV: suviDani [DnevniPodatak 10 2, DnevniPodatak 12 0,  
                DnevniPodatak 8 5, DnevniPodatak 15 0]  
IZLAZ: [12,15]
```

Primer 2

```
POZIV: suviDani [DnevniPodatak 0 0, DnevniPodatak 5 3,  
                DnevniPodatak 7 0]  
IZLAZ: [0,7]
```

2. `rasponSuvihDana :: [DnevniPodaci] -> Int`
koja vraća razliku u temperaturi između najtoplijeg i najhladnijeg dana koji nemaju padavine, ili 0 ako nema takvih dana;

Primer 3

```
POZIV: rasponSuvihDana  
       [DnevniPodatak 10 2, DnevniPodatak 12 0,  
        DnevniPodatak 8 5, DnevniPodatak 15 0]  
IZLAZ: 3
```

Primer 4

```
POZIV: rasponSuvihDana [DnevniPodatak 0 0,  
                        DnevniPodatak 5 3, DnevniPodatak 7 0]  
IZLAZ: 7
```

3. `topliPadavniDani :: [DnevniPodaci] -> Int -> [DnevniPodaci]`
koja vraća sve dane sa padavinama čija temperatura je iznad zadate granice;

Primer 5

```
POZIV: topliPadavniDani [DnevniPodatak 7 2, DnevniPodatak 8 0,
DnevniPodatak 9 5, DnevniPodatak 6 0] 7
IZLAZ: [DnevniPodatak 8 0, DnevniPodatak 9 5]
```

Primer 6

```
POZIV: topliPadavniDani [DnevniPodatak 0 0, DnevniPodatak 5 3,
DnevniPodatak 7 0] 4
IZLAZ: [DnevniPodatak 5 3]
```

4. `stvarniOsecaj :: [DnevniPodaci] -> [DnevniPodaci]`
 koja vraca listu dana sa istom vrednošću padavina, ali sa temperaturom prilagođenom prema formuli stvarnog osećaja. Za svaki dan računamo:

$$T_{\text{osećaj}} = \lfloor T_{\text{temperatura}} - 0.3 \cdot P_{\text{padavine}} \rfloor$$

Primer 7

```
POZIV: stvarniOsecaj [DnevniPodatak 10 2, DnevniPodatak 12 0,
DnevniPodatak 8 5, DnevniPodatak 15 0]
IZLAZ: [DnevniPodatak 9 2, DnevniPodatak 12 0,
DnevniPodatak 6 5, DnevniPodatak 15 0]
```

Primer 8

```
POZIV: stvarniOsecaj [DnevniPodatak 0 0,
DnevniPodatak 5 3, DnevniPodatak 7 0]
IZLAZ: [DnevniPodatak 0 0, DnevniPodatak 4 3,
IZLAZ: DnevniPodatak 7 0]
```

Zadatak 3.4.22

Nemanji je potrebna pomoć oko kompresovanja i dekompresovanja matrica koje su mu potrebe za njegov projekat. Matrice sa kojima on radi sadrže pretežno nule ali su neki elementi brojevi koji su stepen broja 2. Svaka matrica dimenzije $m \times n$ predstavljena je listom koja sadrži m listi brojeva (svaki broj je 0 ili stepen broja 2) dužine n (npr. jedna matrica dimenzije 2×3 je predstavljena sa

$[[0,0,8],[16,0,4]]$). Matrica se kompresuje na sledeći način: svaka vrsta (podlista) se zamenjuje listom parova (*pozicija, stepen*) gde pozicija predstavlja poziciju na kojoj se nalazi element koji nije nula u toj vrsti a stepen je broj koji govori koji stepen dvojke je taj element (npr. ako se u određenoj vrsti na drugoj poziciji nalazi element 8 njemu će odgovarati uređen par (2,3)). Napisati funkcije:

1. `kompresuj :: [[Int]] -> [[(Int, Int)]]`

koja kao argument dobija nekompresovanu matricu, a kao rezultat vraća kompresovanu matricu.

Primer 1

```
Poziv: kompresuj [[0,0,0,16,0], [4,0,8,0,0]]
Izlaz: [[(3,4)], [(0,2), (2,3)]]
```

Primer 2

```
Poziv: kompresuj [[0,0,0], [0,8,0]]
Izlaz: [[], [(1,3)]]
```

2. `dekompresuj :: [[(Int, Int)]] -> Int -> [[Int]]`

koja kao argument dobija kompresovanu matricu i dimenziju vrste, a kao rezultat vraća nekompresovanu matricu.

Primer 3

```
Poziv: dekompresuj [(3,4)], [(0,2), (2,3)] 5
Izlaz: [[0,0,0,16,0], [4,0,8,0,0]]
```

Primer 4

```
Poziv: dekompresuj [], [(1,3)] 3
Izlaz: [[0,0,0], [0,8,0]]
```


Zadatak 3.4.23

U prodavnici se dobija broj koji služi za čekanje u redu (što manji broj bićete pre usluženi). Svaka osoba opisuje se svojim imenom i dobijenim brojem.

U redu već stoji nekoliko ljudi (ne znamo da li poštuju redosled brojeva). U prodavnicu može da dođe osoba koja uzme svoj broj ali ga ne poštuje i staje u red na poziciju na koju ona želi. Takođe, može da dođe osoba koja zna da u redu ne poštuju svi svoj broj ali koja staje tačno na mesto na kojem bi njen broj trebao biti (kada bi red bio sortiran, staje na poziciju koja odgovara dobijenom broju prilikom sortiranja). Dešava se i da u prodavnicu uđe više ljudi odjednom i da svako stane gde želi.

Napisati funkcije za održavanje reda (liste) kupaca:

1. `ubaciNaPoz :: [(String,Int)] -> Int -> (String,Int) -> [(String,Int)]`
 koja u red dodaje novog kupca (3. argument) koji želi na poziciju `k` (brojeći od 0, 2. argument) iako to možda nije pozicija na kojoj bi trebao biti. Pretpostaviti da je `k` ispravno zadat;

Primer 1

```
Poziv: ubaciNaPoz [("marko",3),("marija",5)] 1 ("nevena",6)
Izlaz: [("marko",3),("nevena",6),("marija",5)]
```

Primer 2

```
Poziv: ubaciNaPoz [("marko",2)] 0 ("nevena",1)
Izlaz: [("nevena",1),("marko",2)]
```

2. `ubaciURed :: [(String,Int)] -> (String,Int) -> [(String,Int)]`
 koja u red dodaje novog kupca (2. argument) na poziciju na kojoj bi on trebao da bude da svi poštuju red;

Primer 3

```
Poziv: ubaciURed [("marko",3),("marija",5)] ("nevena",6)
Izlaz: [("marko",3),("marija",5),("nevena",6)]
```

Primer 4

```
Poziv: ubaciURed [("marko",2)] ("nevena",1)
Izlaz: [("nevena",1),("marko",2)]
```

Primer 5

```
POZIV: ubaciURed [("marko",4),("ana",1),("milica",2)]  
        ("milan",3)  
IZLAZ: [("marko",4),("ana",1),("milan",3),("milica",2)]
```

Primer 6

```
POZIV: ubaciURed [("marija",11),("milica",12)] ("milan",17)  
IZLAZ: [("marija",11),("milica",12),("milan",17)]
```

3. `ubaciVise :: [(String,Int)] -> [Int] -> [(String,Int)] -> [(String,Int)]`

koja u red koji je zadata kao prvi argument ubacuje više novih kupaca koji ne poštuju red, tako da *i*-ti element druge liste odgovara poziciji na koju treba ubaciti *i*-tog po redu novog kupca iz treće liste. Pretpostaviti da su argumenti ispravni i druga i treća lista istih dimenzija.

Primer 7

```
POZIV: ubaciVise [("marko",3),("marija",5)] [1] [("nevena",6)]  
IZLAZ: [("marko",3),("nevena",6),("marija",5)]
```

Primer 8

```
POZIV: ubaciVise [("marko",3)] [1,0] [("ana",6),("kosta",8)]  
IZLAZ: [("kosta",8),("marko",3),("ana",6)]
```

Zadatak 3.4.24

Podaci o temperaturama za narednih nekoliko dana u gradu su zadati kao alias: `type Temperature = [Double]`. Definirati funkcije

1. `promena :: Temperature -> Double -> Int -> Temperature`

koja za date podatke o temperaturama, realan broj `d` i ceo broj `n` menja temperature dana za `d` stepeni počev od `n`-tog. Pretpostaviti da je indeks `n` ispravno zadat i da se broji od 0 u listi.

Primer 1

```
Poziv: promena [10,11.5,12,13,13,9.5] 1.4 3
Izlaz: [10,11.5,12,14.4,14.4,10.9]
```

Primer 2

```
Poziv: promena [15.5,15.2,13.9,16,17,15.8] -1 4
Izlaz: [15.5,15.2,13.9,16.0,16.0,14.8]
```

2. `topliji :: Temperature -> Temperature -> Double -> Temperature`

koja menja temperature svih dana iz grada u kom ima više toplijih dana za `d` stepeni i vraća novodobijene temperature za taj grad. Pretpostaviti da u datim listama, podaci na istim pozicijama odgovaraju istim danima i da su liste jednakih dužina. U slučaju da je broj toplijih dana isti, modifikovati temperature za prvi grad.

Primer 3

```
Poziv: topliji [12,11.5,12.23] [11.9,11.4,13.9] 2
Izlaz: [14,13.5,14.23]
```

Primer 4

```
Poziv: topliji [11,14.0,19.9,20] [10.5,15.0,21.23,22] -1
Izlaz: [9.5,14.0,20.23,21]
```

Zadatak 3.4.25

U svakom krugu igre *Muzička stolica* ispada po jedan učesnik. Učesnici igre se čuvaju u vidu liste imena gde redni broj učesnika odgovara njegovoj poziciji u listi (pozicije se broje od 0). Napisati funkcije koje određuju preostale učesnike nakon određenog broja krugova igre *Muzička stolica* ili rekonstruišu podatke o učesnicima:

1. `izbaci :: [[Char]] -> Int -> [[Char]]`

koja iz liste učesnika izbacuje *k*-tog (brojeći od 0). Pretpostaviti da je lista učesnika neprazna i da je drugi argument *k* ispravno zadat.

Primer 1

```
POZIV: izbaci ["ana","marko","milan"] 2
IZLAZ: ["ana","marko"]
```

Primer 2

```
POZIV: izbaci ["ana","marko","milan","jelena"] 1
IZLAZ: ["ana","milan","jelena"]
```

Primer 3

```
POZIV: izbaci ["ana"] 0
IZLAZ: []
```

2. `izbaciVise :: [[Char]] -> [Int] -> [[Char]]`

koja iz liste učesnika koja je zadata kao prvi argument izbacuje one čije su početne pozicije date kao elementi opadajuće liste zadate kao drugi argument. Pretpostaviti da su argumenti ispravni.

Primer 4

```
POZIV: izbaciVise ["ana","marko","milan"] [2,1]
IZLAZ: ["ana"]
```

Primer 5

```
POZIV: izbaciVise ["ana","marko","milan"] [2,0]
IZLAZ: ["marko"]
```

Primer 6

```
POZIV: izbaciVise ["ana"] [0]  
IZLAZ: []
```

3. `rekonstruisi :: [[Char]] -> [[Char]] -> [Int] -> [[Char]]`

koja na osnovu liste učesnika koji su preostali koja je zadata kao prvi argument i liste učesnika koji su izbačeni do sada, zajedno sa pozicijama (rastuća lista) sa kojih su izbačeni, zadati kao drugi i treći argument redom, rekonstruiše listu učesnika sa početka igre. Pretpostaviti da su argumenti ispravni.

Primer 7

```
POZIV: rekonstruisi ["ana"] ["marko","milan"] [1,2]  
IZLAZ: ["ana","marko","milan"]
```

Primer 8

```
POZIV: rekonstruisi ["marko"] ["milan","ana"] [0,2]  
IZLAZ: ["milan","marko","ana"]
```

Primer 9

```
POZIV: rekonstruisi [] ["ana"] [0]  
IZLAZ: ["ana"]
```

Zadatak 3.4.26

Podaci o broju poena na ispitu u jednom roku za nekoliko studenata su zadati kao `type Poeni = [Double]`. Definirati funkcije:

1. `promena :: Poeni -> Double -> Int -> Poeni`
koja za date podatke o poenima, realan broj `d` i ceo broj `n`, dodaje svakom studentu `d` poena, počev od `n`-tog studenta u spisku poena. Pretpostaviti da je indeks `n` ispravno zadat i da se broji od 0 u listi.

Primer 1

```
Poziv: promena [40,51.5,62,73,83,89.5] 1.4 3  
Izlaz: [40,51.5,62,74.4,84.4,90.9]
```

Primer 2

```
Poziv: promena [15.5,65.2,43.9,16,87,25.8] -1 4  
Izlaz: [15.5,65.2,43.9,16.0,86.0,24.8]
```

2. `bolji :: Poeni -> Poeni -> Double -> Poeni`
koja za poene studenata iz dva ispitna roka menja za `d` sve poene u onom roku u kom studenti ukupno imaju više poena i vraća novodobijene poene za taj rok. U slučaju da je ukupan broj poena za oba roka isti, modifikovati poene za prvi rok.

Primer 3

```
Poziv: bolji [62,71.5,72.5] [81.0,11.5,63.5] 2  
Izlaz: [64,73.5,74.5]
```

Primer 4

```
Poziv: bolji [11,14,90,20] [10.5,95,21,22] -1  
Izlaz: [9.5,94.0,20,21]
```

Zadatak 3.4.27

Kreirati tip podataka `Student` koji se može konstruisati putem konstruktora `MkStudent` i sadrži dva atributa `ime` (`String`) i `poeni` (`Int`), kao i funkcije `ime` i `poeni` koje omogućavaju pristup poljima `ime` i `poeni`. Za kreirani tip instancirati klasu `Show` (podrazumevano) i klasu `Ord` tako da se studenti porede po broju ostvarenih poena. Implementirati funkcije:

1. `rangLista :: [Student] -> [Student]`
koja na ulazu dobija listu studenata i sortira je opadajuće po broju poena koje je student ostvario na ispitu.

Primer 1

```
Poziv: rangLista [MkStudent "Marko" 75, MkStudent "Vuk" 50  
          MkStudent "Ana" 91]  
Izlaz: [MkStudent "Ana" 91, MkStudent "Marko" 75,  
        MkStudent "Vuk" 50]
```

Primer 2

```
Poziv: rangLista [MkStudent "Ilija" 75, MkStudent "Mila" 86,  
          MkStudent "Kristina" 53, MkStudent "Jovan" 96]  
Izlaz: [MkStudent "Jovan" 96, MkStudent "Mila" 86,  
        MkStudent "Ilija" 75, MkStudent "Kristina" 53]
```

2. `poeniStudenata :: [Student] -> [(String, Maybe Int)]`
koja na ulazu dobija listu studenata i za svakog od njih u slučaju položenog ispita vraća ime i poene, a inače ime i praznu vrednost.

Primer 3

```
Poziv: poeniStudenata [MkStudent "Ilija" 75,  
          MkStudent "Mila" 42, MkStudent "Kristina" 53]  
Izlaz: [("Ilija", Just 75), ("Mila", Nothing),  
        ("Kristina", Just 53)]
```

Primer 4

```
Poziv: poeniStudenata  
       [MkStudent "Marko" 75, MkStudent "Vuk" 50  
        MkStudent "Ana" 91, MkStudent "Milica" 53]  
Izlaz: [("Marko", Just 75), ("Vuk", Nothing),  
        ("Ana", Just 91), ("Milica", Just 53)]
```

Zadatak 3.4.28

Napraviti Haskell biblioteku koja pomaže kupcima pri kupovini u supermarketu.

1. Napisati funkciju

```
unesiProizvod :: (String, Float) -> [(String, Float)] -> [(String, Float)]
```

koja u listu proizvoda sortiranu po ceni rastuće unosi novi proizvod na odgovarajuće mesto.

Primer 1

```
Poziv: unesiProizvod ("Mleko", 70)
      [("Hleb", 50), ("Sir", 100)]
Izlaz: [("Hleb", 50), ("Mleko", 70), ("Sir", 100)]
```

Primer 2

```
Poziv: unesiProizvod ("Jabuke", 30) []
Izlaz: [("Jabuke", 30)]
```

2. Napisati funkciju

```
ukloniProizvod :: String -> [(String, Float)] -> [(String, Float)]
```

koja iz liste proizvoda uklanja dati proizvod.

Primer 3

```
Poziv: ukloniProizvod "Sir"
      [("Hleb", 50), ("Sir", 100), ("Mleko", 70)]
Izlaz: [("Hleb", 50), ("Mleko", 70)]
```

Primer 4

```
Poziv: ukloniProizvod "Jabuke"
      [("Hleb", 50), ("Sir", 100), ("Mleko", 70)]
Izlaz: [("Hleb", 50), ("Sir", 100), ("Mleko", 70)]
```

3. Napisati funkciju

```
primeniPopust :: [(String, Float)] -> [(String, Float)]
```

koja na sve proizvode čija je cena veća od 1000 dinara primenjuje popust od 10% i vraća listu proizvoda sa ažuriranim cenama.

Primer 5

```
Poziv: primeniPopust [("Televizor", 24675), ("Laptop", 67340)
                     ("Slusalice", 560)]
Izlaz: [("Televizor", 22207.5), ("Laptop", 60606.0),
        ("Slusalice", 560.0)]
```

Primer 6

```
Poziv: primeniPopust [("Hleb", 50), ("Sir", 100),
                     ("Mleko", 70)]
Izlaz: [("Hleb", 50.0), ("Sir", 100.0), ("Mleko", 70.0)]
```

4. Kreirati tip podataka **Proizvod** koji se može konstruisati putem konstruktora **MkProizvod** i sadrži dva atributa **naziv** (**String**) i **cena** (**Float**), kao i funkcije **naziv** i **cena** koje omogućavaju pristup poljima **naziv** i **cena**. Za kreirani tip instancirati klasu **Show** (podrazumevano).

5. Napisati funkciju

```
izbaciSkupeProizvode :: Float -> [Proizvod] ->
[Proizvod]
```

koja prima određenu cenu i listu proizvoda, i vraća listu proizvoda čije cene ne prelaze zadatu cenu.

Primer 7

```
Poziv: izbaciSkupeProizvode 200 [MkProizvod "Hleb" 120,
                                MkProizvod "Mleko" 90, MkProizvod "Sir" 450,
                                MkProizvod "Cokolada" 200, MkProizvod "Sok" 350]
Izlaz: [Proizvod { naziv = "Hleb", cena = 120.0 },
        Proizvod { naziv = "Mleko", cena = 90.0 },
        Proizvod { naziv = "Cokolada", cena = 200.0 }]
```

Primer 8

```
Poziv: izbaciSkupeProizvode 150 [MkProizvod "Kafa" 150,
                                MkProizvod "Secer" 100, MkProizvod "Caj" 80,
                                MkProizvod "Biskvit" 120]
Izlaz: [Proizvod { naziv = "Kafa", cena = 150.0 }
        Proizvod { naziv = "Secer", cena = 100.0 }
        Proizvod { naziv = "Caj", cena = 80.0 }
        Proizvod { naziv = "Biskvit", cena = 120.0 }]
```

6. Napisati funkciju

```
pronadjiProizvod :: String -> [Proizvod] -> Either  
String Proizvod
```

koja nalazi proizvod sa zadatim nazivom u listi proizvoda i vraća proizvod ukoliko on postoji u listi, a inače poruku greške.

Primer 9

```
Poziv: pronadjiProizvod "Laptop"  
      [MkProizvod "Televizor" 24675,  
       MkProizvod "Laptop" 67340, MkProizvod "Slusalice" 560]  
Izlaz: Right (Proizvod naziv = "Laptop", cena = 67340)
```

Primer 10

```
Poziv: pronadjiProizvod "Mikrofon"  
      [MkProizvod "Televizor" 24675,  
       MkProizvod "Laptop" 67340, MkProizvod "Slusalice" 560]  
Izlaz: Left "Proizvod sa nazivom Mikrofon nije pronađen."
```

Zadatak 3.4.29

Napisati biblioteku funkcija za rad sa log porukama. Svaka log poruka se predstavlja uređenim parom (`tip_poruke`, `poruka`), tipa (`Int`, `String`). Tip poruke može biti *debug* (vrednost 0), *info* (vrednost 1), *warn* (vrednost 2), ili *error* (vrednost 3). Implementirati:

1. `izdvojNivo n poruke`, čiji je tip:
`izdvojNivo :: Int -> [(Int, String)] -> [(Int, String)]`
koja iz liste poruke uklanja sve poruke nivoa različitog od `n`.

Primer 1

```
Poziv:  izdvojNivo 0 [(0,"start"),(0,"init"),(3,"crash")]
Izlaz:  [(0,"start"),(0,"init")]
```

Primer 2

```
Poziv:  izdvojNivo 2 [(0,"start"),(1,"ok"),(3,"fail")]
Izlaz:  []
```

Primer 3

```
Poziv:  izdvojNivo 3 [(0,"init"),(3,"db fail"),(1,"login ok")]
Izlaz:  [(3,"db fail")]
```

Primer 4

```
Poziv:  izdvojNivo 1
        [(1,"login ok"),(2,"slow req"),(1,"logout ok")]
Izlaz:  [(1,"login ok"),(1,"logout ok")]
```

Primer 5

```
Poziv:  izdvojNivo 3 []
Izlaz:  []
```

2. `brojGresaka poruke`, čiji je tip:
`brojGresaka :: [(Int, String)] -> Int`
koja vraća broj poruka koje predstavljaju greške.

Primer 6

```
Poziv:  brojGresaka [(0,"start"),(1,"req ok"),(2,"slow req")]
Izlaz:  0
```

Primer 7

```
Poziv: brojGresaka [(0,"init"),(3,"db fail"),(3,"timeout")]
Izlaz: 2
```

Primer 8

```
Poziv: brojGresaka [(3,"disk err"),(3,"oom"),(3,"conn lost")]
Izlaz: 3
```

Primer 9

```
Poziv: brojGresaka []
Izlaz: 0
```

3. grupisi poruke, čiji je tip:

```
grupisi :: [(Int, String)] -> [(Int, [String])]
```

koja grupiše poruke istog nivoa. Rezultat je lista parova (nivo, poruke) sortirana rastuće po nivou. Redosled poruka u okviru jednog nivoa odgovara redosledu njihovog pojavljivanja u ulaznoj listi.

Primer 10

```
Poziv: grupisi [(0,"init"),(3,"db fail"),(1,"login ok"),
               (3,"timeout"),(2,"message received")]
Izlaz: [(0,["init"]), (1,["login ok"]),
        (2,["message received"]), (3,["db fail","timeout"])]
```

Primer 11

```
Poziv: grupisi
       [(2,"slow req"),(0,"start"),(2,"retry"),(0,"init")]
Izlaz: [(0,["start","init"]), (2,["slow req","retry"])]
```

Primer 12

```
Poziv: grupisi [(1,"login ok"),(1,"logout ok"),(1,"refresh")]
Izlaz: [(1,["login ok","logout ok","refresh"])]
```

Primer 13

```
Poziv: grupisi []
Izlaz: []
```

4. najcesciNivo poruke, čiji je tip:
najcesciNivo :: [(Int, String)] -> Maybe Int
koja određuje nivo poruke koji se pojavljuje najviše puta. Za praznu listu vratiti Nothing. Ako više nivoa ima isti maksimalan broj pojavljivanja, vratiti najmanji nivo.

Primer 14

```
Poziv: najcesciNivo [(0,"a"),(3,"x"),(3,"y"),(1,"z")]
Izlaz: Just 3
```

Primer 15

```
Poziv: najcesciNivo [(1,"a"),(2,"b"),(1,"c"),(2,"d")]
Izlaz: Just 1
```

Primer 16

```
Poziv: najcesciNivo []
Izlaz: Nothing
```

5. kompresuj poruke, čiji je tip:
kompresuj :: [(Int, String)] -> [(Int, String, Int)]
koja uzastopne identične poruke grupiše u trojke oblika (nivo, sadržaj, broj_ponavljanja).

Primer 17

```
Poziv: kompresuj
      [(1,"ok"),(1,"ok"),(3,"fail"),(3,"fail"),(3,"fail")]
Izlaz: [(1,"ok",2),(3,"fail",3)]
```

Primer 18

```
Poziv: kompresuj
      [(0,"start"),(1,"ok"),(0,"start")]
Izlaz: [(0,"start",1),(1,"ok",1),(0,"start",1)]
```

Primer 19

```
Poziv: kompresuj []
Izlaz: []
```

Zadatak 3.4.30

Napisati biblioteku funkcija za rad sa log porukama. Implementirati:

1. Tip `LogPoruka`, koji se konstruiše konstruktorima `Debug`, `Info`, `Warn` i `Error`. Svaki konstruktor prima sadržaj poruke tipa `String`. Podrazumevano instancirati klase `Eq` i `Ord`. Poruke se porede prvo po nivou, redom `Debug < Info < Warn < Error`, a zatim leksikografski po sadržaju.

Primer 1

```
POZIV: Debug "init ok" == Debug "init ok"  
IZLAZ: True
```

Primer 2

```
POZIV: Info "user logout" == Info "user login"  
IZLAZ: False
```

Primer 3

```
POZIV: Debug "cache hit" > Info "message received"  
IZLAZ: False
```

Primer 4

```
POZIV: Info "login success" < Warn "high latency"  
IZLAZ: True
```

Primer 5

```
POZIV: Error "disk full" > Warn "timeout occurred"  
IZLAZ: True
```

Primer 6

```
POZIV: Warn "slow request" < Error "db connection failed"  
IZLAZ: True
```

2. Instancirati klasu `Show` za tip `LogPoruka`. Poruke se prikazuju u formatu "[DEBUG] s", "[INFO] s", "[WARN] s" i "[ERROR] s".

Primer 7

```
POZIV: show (Debug "init ok")
IZLAZ: "[DEBUG] init ok"
```

Primer 8

```
POZIV: show (Warn "high latency detected")
IZLAZ: "[WARN] high latency detected"
```

Primer 9

```
POZIV: show (Info "login success")
IZLAZ: "[INFO] login success"
```

Primer 10

```
POZIV: show (Error "db connection failed")
IZLAZ: "[ERROR] db connection failed"
```

Primer 11

```
POZIV: show (Debug "cache miss")
IZLAZ: "[DEBUG] cache miss"
```

3. greska poruke, čiji je tip:
- ```
greska :: [LogPoruka] -> Maybe String
```
- koja vraća sadržaj poslednje greške u listi `poruke`, posmatrano prema rastuće-sortiranom poretku. Ako lista ne sadrži greške, vratiti `Nothing`.

## Primer 12

```
POZIV: greska [Info "start", Error "db fail",
 Warn "slow req", Error "timeout"]
IZLAZ: Just "timeout"
```

## Primer 13

```
POZIV: greska [Debug "init ok", Info "login ok",
 Warn "cache miss"]
IZLAZ: Nothing
```

## Primer 14

```
POZIV: greska [Error "disk full", Error "db crash",
 Info "recovery started"]
IZLAZ: Just "disk full"
```

## Primer 15

```
POZIV: greska [Warn "slow req", Error "timeout occurred",
 Debug "trace", Error "connection lost"]
IZLAZ: Just "timeout occurred"
```

4. `sortirajPoDuzini` poruke, čiji je tip:  
`sortirajPoDuzini :: [LogPoruka] -> [LogPoruka]`  
koja sortira poruke rastuće po dužini sadržaja. Ako dve poruke imaju istu dužinu sadržaja, koristiti podrazumevani poredak za tip `LogPoruka`.

## Primer 16

```
POZIV: sortirajPoDuzini
 [Error "disk full", Debug "ok", Info "hello"]
IZLAZ: [[DEBUG] ok, [INFO] hello, [ERROR] disk full]
```

## Primer 17

```
POZIV: sortirajPoDuzini [Warn "abc", Debug "xyz", Error "aa"]
IZLAZ: [[ERROR] aa, [DEBUG] xyz, [WARN] abc]
```

## Primer 18

```
POZIV: sortirajPoDuzini []
IZLAZ: []
```



**Zadatak 3.4.31**

Napisati biblioteku funkcija za rad sa rezervacijama učionica. Svaka rezervacija je uređena trojka (sala, početak, trajanje), tipa (String, Int, Int). Vrednost početak predstavlja sat u kom rezervacija počinje, a trajanje broj sati koliko rezervacija traje. Implementirati:

1. rezervacijeSale sala rezervacije, čiji je tip:  
rezervacijeSale :: String -> [(String, Int, Int)] -> [(String, Int, Int)]  
koja iz liste rezervacije uklanja sve rezervacije koje nisu vezane za salu sala.

**Primer 1**

```
Poziv: rezervacijeSale "A1"
 [("A1",8,2),("B1",10,3),("A1",12,1)]
Izlaz: [("A1",8,2),("A1",12,1)]
```

**Primer 2**

```
Poziv: rezervacijeSale "A3" [("A1",8,2),("B1",10,3)]
Izlaz: []
```

**Primer 3**

```
Poziv: rezervacijeSale "B1"
 [("B1",9,2),("A2",10,1),("B1",14,3)]
Izlaz: [("B1",9,2),("B1",14,3)]
```

**Primer 4**

```
Poziv: rezervacijeSale "C1"
 [("C1",8,1),("C1",9,1),("A1",10,2)]
Izlaz: [("C1",8,1),("C1",9,1)]
```

**Primer 5**

```
Poziv: rezervacijeSale "A1" []
Izlaz: []
```

2. `ukupnoTrajanje rezervacije`, čiji je tip: `ukupnoTrajanje :: [(String, Int, Int)] -> Int`  
 koja vraća ukupan broj sati zauzetosti sala.

## Primer 6

```
Poziv: ukupnoTrajanje [("A1",8,2),("B1",10,3),("A1",12,1)]
Izlaz: 6
```

## Primer 7

```
Poziv: ukupnoTrajanje []
Izlaz: 0
```

## Primer 8

```
Poziv: ukupnoTrajanje [("C1",9,2),("C1",12,2),("C1",16,1)]
Izlaz: 5
```

## Primer 9

```
Poziv: ukupnoTrajanje [("A2",18,1)]
Izlaz: 1
```

3. `rasporedSala rezervacije`, čiji je tip:  
`rasporedSala :: [(String, Int, Int)] -> [(String, [(Int, Int)])]`  
 koja grupiše rezervacije po salama. Rezultat je lista parova (sala, [(početak, trajanje)]), sortirana rastuće po oznaci sale. Redosled rezervacija unutar svake sale odgovara redosledu pojavljivanja u ulaznoj listi.

## Primer 10

```
Poziv: rasporedSala
 [("B1",10,2),("A1",8,1),("B1",13,1),("C1",9,2)]
Izlaz: [("A1",[(8,1)]),("B1",[(10,2),(13,1)]),("C1",[(9,2)])]
```

## Primer 11

```
Poziv: rasporedSala
 [("C1",8,2),("A2",10,1),("C1",14,2),("A2",16,1)]
Izlaz: [("A2",[(10,1),(16,1)]),("C1",[(8,2),(14,2)])]
```

## Primer 12

```
Poziv: rasporedSala [("A1",8,2),("A1",11,1),("A1",13,2)]
Izlaz: [("A1",[(8,2),(11,1),(13,2)])]
```

## Primer 13

```
Poziv: rasporedSala []
Izlaz: []
```

4. najzauzetijaSala rezervacije, čiji je tip:  
najzauzetijaSala :: [(String, Int, Int)] -> Maybe  
String  
koja određuje salu čije rezervacije imaju najveće ukupno  
trajanje. Za praznu listu vratiti Nothing. Ako više sala ima  
isto najveće ukupno trajanje, vratiti leksikografski najmanju  
oznaku sale.

## Primer 14

```
Poziv: najzauzetijaSala [("A1",8,2),("B1",10,3),("A1",12,2)]
Izlaz: Just "A1"
```

## Primer 15

```
Poziv: najzauzetijaSala []
Izlaz: Nothing
```

## Primer 16

```
Poziv: najzauzetijaSala
 [("B1",9,2),("A2",10,2),("B1",14,1),("A2",16,1)]
Izlaz: Just "A2"
```

5. kompresujRezervacije rezervacije, čiji je tip:  
kompresujRezervacije :: [(String, Int, Int)] ->  
[(String, Int, Int, Int)]  
koja uzastopne identične rezervacije grupiše u četvorke oblika  
(sala, početak, trajanje, broj-ponavljanja).

## Primer 17

```
Poziv: kompresujRezervacije
 [("C1",9,1),("C1",9,1),("C1",9,1)]
Izlaz: [("C1",9,1,3)]
```

## Primer 18

```
Poziv: kompresujRezervacije
 [("A1",8,2),("A1",8,2),("B1",10,1)]
Izlaz: [("A1",8,2,2),("B1",10,1,1)]
```

## Primer 19

```
Poziv: kompresujRezervacije
 [("A1",8,2),("B1",10,1),("A1",8,2)]
Izlaz: [("A1",8,2,1),("B1",10,1,1),("A1",8,2,1)]
```

## Primer 20

```
Poziv: kompresujRezervacije []
Izlaz: []
```

## Zadatak 3.4.32

Napisati biblioteku funkcija za rad sa terminima nastave i provera znanja. Implementirati:

1. Tip **Termin**, koji se konstruiše konstruktorima **Predavanje**, **Vezbe**, **Kolokvijum** i **Ispit**. Svaki konstruktor prima oznaku sale tipa **String** i trajanje termina u satima tipa **Int**. Podrazumevano instancirati klase **Eq** i **Ord**. Termini se porede redom **Predavanje < Vezbe < Kolokvijum < Ispit**; za iste konstruktore porede se oznaka sale, pa trajanje.

## Primer 1

```
Poziv: Predavanje "A1" 2 == Predavanje "A1" 2
Izlaz: True
```

## Primer 2

```
Poziv: Kolokvijum "A1" 2 == Kolokvijum "A1" 3
Izlaz: False
```

## Primer 3

```
Poziv: Predavanje "A2" 2 > Vezbe "A2" 2
Izlaz: False
```

## Primer 4

```
POZIV: Vezbe "B1" 2 < Kolokvijum "B1" 2
IZLAZ: True
```

## Primer 5

```
POZIV: Ispit "C1" 3 > Kolokvijum "C1" 2
IZLAZ: True
```

## Primer 6

```
POZIV: Vezbe "C1" 1 < Ispit "A1" 4
IZLAZ: True
```

2. Instancirati klasu `Show` za tip `Termin`. Termini se prikazuju u formatu "[PREDAVANJE] s th", "[VEZBE] s th", "[KOLOKVIJUM] s th" i "[ISPIT] s th".

## Primer 7

```
POZIV: show (Predavanje "A1" 2)
IZLAZ: "[PREDAVANJE] A1 2h"
```

## Primer 8

```
POZIV: show (Kolokvijum "C1" 2)
IZLAZ: "[KOLOKVIJUM] C1 2h"
```

## Primer 9

```
POZIV: show (Vezbe "B1" 3)
IZLAZ: "[VEZBE] B1 3h"
```

## Primer 10

```
POZIV: show (Ispit "A2" 4)
IZLAZ: "[ISPIT] A2 4h"
```

## Primer 11

```
POZIV: show (Predavanje "C1" 1)
IZLAZ: "[PREDAVANJE] C1 1h"
```

3. `poslednjiIspit` termini, čiji je tip:

```
poslednjiIspit :: [Termin] -> Maybe (String, Int)
```

koja vraća salu i trajanje poslednjeg ispita nakon rastućeg sortiranja liste prema poretku za tip `Termin`. Ako lista ne sadrži ispit, vratiti `Nothing`.

## Primer 12

```
Poziv: poslednjiIspit [Predavanje "A1" 2, Ispit "B1" 3,
 Vezbe "A2" 2, Ispit "A1" 2]
Izlaz: Just ("B1",3)
```

## Primer 13

```
Poziv: poslednjiIspit
 [Predavanje "A1" 2, Vezbe "B1" 2, Kolokvijum "C1" 2]
Izlaz: Nothing
```

## Primer 14

```
Poziv: poslednjiIspit
 [Ispit "A1" 2, Ispit "A1" 4, Kolokvijum "A1" 2]
Izlaz: Just ("A1",4)
```

## Primer 15

```
Poziv: poslednjiIspit
 [Ispit "C1" 2, Ispit "A2" 4, Predavanje "C1" 2]
Izlaz: Just ("C1",2)
```

4. `sortirajPoTrajanju` termini, čiji je tip:

```
sortirajPoTrajanju :: [Termin] -> [Termin]
```

koja sortira termine rastuće po trajanju. Ako dva termina imaju isto trajanje, koristiti podrazumevani poredak za tip `Termin`.

## Primer 16

```
Poziv: sortirajPoTrajanju
 [Ispit "A1" 4, Predavanje "C1" 1, Vezbe "B1" 2]
Izlaz: [[PREDAVANJE] C1 1h, [VEZBE] B1 2h, [ISPIT] A1 4h]
```

## Primer 17

```
Poziv: sortirajPoTrajanju
 [Ispit "A1" 2, Predavanje "C1" 2, Vezbe "A2" 2]
Izlaz: [[PREDAVANJE] C1 2h, [VEZBE] A2 2h, [ISPIT] A1 2h]
```

## Primer 18

```
POZIV: sortirajPoTrajanju [Kolokvijum "B1" 3]
IZLAZ: [[KOLOKVIJUM] B1 3h]
```

## Primer 19

```
POZIV: sortirajPoTrajanju []
IZLAZ: []
```

**Zadatak 3.4.33**

Napisati biblioteku funkcija za rad sa hemijskim elementima. Svaki element je uređen par (simbol, atomski\_broj), tipa (String, Int). Implementirati:

1. `odrediPeriode elementi`, čiji je tip:  
`odrediPeriode :: [(String, Int)] -> [(String, Int)]`  
koja za svaki element određuje periodu na osnovu atomskog broja. Perioda je 1 za atomske brojeve iz intervala [1,2], 2 za [3,10], 3 za [11,18], 4 za [19,36], 5 za [37,54], 6 za [55,86], a 7 za atomske brojeve veće ili jednake 87.

**Primer 1**

```
Poziv: odrediPeriode [("H",1),("O",8),("Fe",26)]
Izlaz: [("H",1),("O",2),("Fe",4)]
```

**Primer 2**

```
Poziv: odrediPeriode [("Au",79),("U",92)]
Izlaz: [("Au",6),("U",7)]
```

**Primer 3**

```
Poziv: odrediPeriode []
Izlaz: []
```

2. `brojRadioaktivnih elementi`, čiji je tip:  
`brojRadioaktivnih :: [(String, Int)] -> Int`  
koja vraća broj elemenata čiji je atomski broj veći ili jednak 83.

**Primer 4**

```
Poziv: brojRadioaktivnih [("U",92),("H",1),("Pu",94)]
Izlaz: 2
```

**Primer 5**

```
Poziv: brojRadioaktivnih [("H",1),("O",8)]
Izlaz: 0
```



## Primer 6

```
Poziv: brojRadioaktivnih []
Izlaz: 0
```

## 3. grupisiPoPeriodi elementi, čiji je tip:

```
grupisiPoPeriodi :: [(String, Int)] -> [(Int,
[String])]
```

koja grupiše elemente po periodu. Rezultat je lista parova (perioda, simboli) sortirana rastuće po periodu. Redosled simbola u svakoj grupi odgovara redosledu pojavljivanja u ulaznoj listi.

## Primer 7

```
Poziv: grupisiPoPeriodi [("H",1),("O",8),("Fe",26),("Ne",10)]
Izlaz: [(1,["H"]), (2,["O","Ne"]), (4,["Fe"])]
```

## Primer 8

```
Poziv: grupisiPoPeriodi [("Au",79),("U",92),("Pb",82)]
Izlaz: [(6,["Au","Pb"]), (7,["U"])]
```

## Primer 9

```
Poziv: grupisiPoPeriodi []
Izlaz: []
```

## 4. najcescaPerioda elementi, čiji je tip:

```
najcescaPerioda :: [(String, Int)] -> Maybe Int
```

koja određuje periodu koja se pojavljuje najviše puta. Za praznu listu vratiti Nothing. Ako više perioda ima isti maksimalan broj pojavljivanja, vratiti najmanju periodu.

## Primer 10

```
Poziv: najcescaPerioda [("H",1),("O",8),("Ne",10)]
Izlaz: Just 2
```

## Primer 11

```
Poziv: najcescaPerioda [("H",1),("He",2),("Li",3),("Ne",10)]
Izlaz: Just 1
```

## Primer 12

```
Poziv: najcescaPerioda []
Izlaz: Nothing
```

5. kompresuj elementi, čiji je tip:  
kompresuj :: [(String, Int)] -> [(String, Int, Int)]  
koja uzastopne identične elemente grupiše u trojke oblika  
(simbol, atomski\_broj, broj\_ponavljanja).

## Primer 13

```
Poziv: kompresuj [("H",1),("H",1),("O",8)]
Izlaz: [("H",1,2),("O",8,1)]
```

## Primer 14

```
Poziv: kompresuj [("H",1),("O",8),("H",1)]
Izlaz: [("H",1,1),("O",8,1),("H",1,1)]
```

## Primer 15

```
Poziv: kompresuj [("Fe",26),("Fe",26),("Fe",26)]
Izlaz: [("Fe",26,3)]
```

## Primer 16

```
Poziv: kompresuj []
Izlaz: []
```

**Zadatak 3.4.34**

Napisati biblioteku funkcija za rad sa hemijskim elementima. Implementirati:

1. Tip `Element`, koji se konstruiše konstruktorima `Metal`, `Nemetal`, `Gas` i `Radioaktivan`. Svaki konstruktor prima naziv elementa tipa `String` i valencu tipa `Int`. Podrazumevano instancirati klase `Eq` i `Ord`.

**Primer 1**

```
POZIV: Metal "Fe" 2 == Metal "Fe" 2
IZLAZ: True
```

**Primer 2**

```
POZIV: Gas "He" 0 == Gas "Ar" 0
IZLAZ: False
```

**Primer 3**

```
POZIV: Metal "Fe" 2 < Gas "Ne" 0
IZLAZ: True
```

**Primer 4**

```
POZIV: Radioaktivan "U" 6 > Nemetal "O" 2
IZLAZ: True
```

2. Instancirati klasu `Show` za tip `Element`. Elementi se prikazuju u formatu "[METAL] s (v)", "[NEMETAL] s (v)", "[GAS] s (v)" i "[RADIOAKTIVAN] s (v)".

**Primer 5**

```
POZIV: show (Metal "Fe" 2)
IZLAZ: "[METAL] Fe (2)"
```

**Primer 6**

```
POZIV: show (Gas "Ne" 0)
IZLAZ: "[GAS] Ne (0)"
```

3. `pronadjiRadioaktivan` elementi, čiji je tip:  
`pronadjiRadioaktivan :: [Element] -> Maybe String`  
 koja vraća naziv poslednjeg radioaktivnog elementa u listi, posmatrano prema rastuće-sortiranom poretku. Ako lista ne sadrži radioaktivne elemente, vratiti `Nothing`.

## Primer 7

```
POZIV: pronadjiRadioaktivan [Gas "Ne" 0, Metal "Au" 3]
IZLAZ: Nothing
```

## Primer 8

```
POZIV: pronadjiRadioaktivan []
IZLAZ: Nothing
```

## Primer 9

```
POZIV: pronadjiRadioaktivan
 [Metal "Fe" 2, Radioaktivan "U" 6, Radioaktivan "Pu" 4]
IZLAZ: Just "U"
```

## Primer 10

```
POZIV: pronadjiRadioaktivan
 [Radioaktivan "Pu" 4, Radioaktivan "U" 6]
IZLAZ: Just "U"
```

4. `sortirajPoValenci` elementi, čiji je tip:  
`sortirajPoValenci :: [Element] -> [Element]`  
 koja sortira elemente rastuće po valenci. Ako dva elementa imaju istu valencu, koristiti podrazumevani poredak za tip `Element`.

## Primer 11

```
POZIV: sortirajPoValenci
 [Radioaktivan "U" 6, Gas "Ne" 0, Metal "Fe" 2]
IZLAZ: [[GAS] Ne (0), [METAL] Fe (2), [RADIOAKTIVAN] U (6)]
```

## Primer 12

```
POZIV: sortirajPoValenci
 [Metal "Fe" 2, Gas "He" 2, Nemetal "O" 2]
IZLAZ: [[METAL] Fe (2), [NEMETAL] O (2), [GAS] He (2)]
```

## Primer 13

```
POZIV: sortirajPoValenci [Gas "Xe" 0]
IZLAZ: [[GAS] Xe (0)]
```

## Primer 14

```
POZIV: sortirajPoValenci []
IZLAZ: []
```

**Zadatak 3.4.35**

Napisati biblioteku funkcija koje pružaju podršku za obradu predatih seminarskih radova. Svaki zapis o predatom seminarskom radu je uređena trojka (`student`, `poeni`, `kasnjenje`), tipa (`String`, `Int`, `Int`), gde su `poeni` nenegativan ceo broj, a `kasnjenje` broj dana kašnjenja. Implementirati:

1. `validniSeminarskiRadovi maxKasnjenje radovi`, čiji je tip:  
`validniSeminarskiRadovi :: Int -> [(String, Int, Int)] -> [(String, Int, Int)]`  
koja zadržava u listi samo one radove čije kašnjenje nije veće od `maxKasnjenje`. Za svaki zadržani rad umanjiti poene za dva poena po danu kašnjenja, ali ne ispod nule. Rezultat treba da sadrži parove (`student`, `noviPoeni`).

**Primer 1**

```
POZIV: validniSeminarskiRadovi 0
 [("Ana",20,0),("Marko",15,2),("Iva",12,0)]
IZLAZ: [("Ana",20),("Iva",12)]
```

**Primer 2**

```
POZIV: validniSeminarskiRadovi 2
 [("Ana",20,0),("Marko",15,2),("Iva",12,3)]
IZLAZ: [("Ana",20),("Marko",11)]
```

**Primer 3**

```
POZIV: validniSeminarskiRadovi 5 [("Mia",10,5)]
IZLAZ: [("Mia",0)]
```

2. `ukupnoPoenaBezKasnjenja radovi`, čiji je tip:  
`ukupnoPoenaBezKasnjenja :: [(String, Int, Int)] -> Int`  
koja vraća ukupan broj poena samo iz radova bez kašnjenja.

**Primer 4**

```
POZIV: ukupnoPoenaBezKasnjenja [("Ana",20,0),("Marko",15,2)]
IZLAZ: 20
```

## Primer 5

```
POZIV: ukupnoPoenaBezKasnjenja
 [("Ana",10,0),("Ana",12,1),("Iva",8,0)]
IZLAZ: 18
```

## Primer 6

```
POZIV: ukupnoPoenaBezKasnjenja []
IZLAZ: 0
```

3. `poeniPoStudentu` radovi, čiji je tip:  
`poeniPoStudentu :: [(String, Int, Int)] -> [(String, Int)]`  
koja za svakog studenta računa ukupan broj poena. Rezultat treba da bude sortiran leksikografski po imenu studenta.

## Primer 7

```
POZIV: poeniPoStudentu
 [("Ana",20,0),("Marko",15,2),("Ana",5,1)]
IZLAZ: [("Ana",25),("Marko",15)]
```

## Primer 8

```
POZIV: poeniPoStudentu
 [("Zoe",8,0),("Bojan",7,2),("Zoe",2,3),("Ana",4,1)]
IZLAZ: [("Ana",4),("Bojan",7),("Zoe",10)]
```

## Primer 9

```
POZIV: poeniPoStudentu []
IZLAZ: []
```

4. `najboljiStudent` radovi, čiji je tip:  
`najboljiStudent :: [(String, Int, Int)] -> Maybe String`  
koja vraća ime studenta sa najvećim ukupnim brojem poena, ako takav postoji. Ako više studenata ima isti broj poena, vratiti leksikografski najmanje ime.

## Primer 10

```
Poziv: najboljiStudent
 [("Ana",20,0),("Marko",15,2),("Ana",5,1)]
Izlaz: Just "Ana"
```

## Primer 11

```
Poziv: najboljiStudent [("Mia",10,0),("Noa",10,1)]
Izlaz: Just "Mia"
```

## Primer 12

```
Poziv: najboljiStudent []
Izlaz: Nothing
```

5. `kompresujSeminarskeRadove` radovi, čiji je tip:
- ```
kompresujSeminarskeRadove :: [(String, Int, Int)] ->
[(String, Int, Int, Int)]
```
- koja uzastopne identične zapise o seminarskim radovima grupiše u četvorke (`student`, `poeni`, `kasnjenje`, `brojPonavljanja`).

Primer 13

```
Poziv: kompresujSeminarskeRadove
      [("Ana",20,0),("Ana",20,0),("Marko",15,2)]
Izlaz: [("Ana",20,0,2),("Marko",15,2,1)]
```

Primer 14

```
Poziv: kompresujSeminarskeRadove
      [("Ana",20,0),("Marko",15,2),("Ana",20,0)]
Izlaz: [("Ana",20,0,1),("Marko",15,2,1),("Ana",20,0,1)]
```

Primer 15

```
Poziv: kompresujSeminarskeRadove []
Izlaz: []
```


Zadatak 3.4.36

Napisati biblioteku funkcija koje pružaju podršku za obradu predatih seminarskih radova.

1. Definirati tip `SeminarskiRad`, koji se konstruiše konstruktorima `PredatNaVreme` i `PredatSaKasnjenjem`. Konstruktor `PredatNaVreme` prima ime studenta tipa `String` i broj poena tipa `Int`. Konstruktor `PredatSaKasnjenjem` prima ime studenta tipa `String`, broj poena tipa `Int` i broj dana kašnjenja tipa `Int`. Podrazumevano instancirati klase `Eq` i `Ord`.

Primer 1

```
Poziv: PredatNaVreme "Ana" 20 ==  
       PredatNaVreme "Ana" 20  
Izlaz: True
```

Primer 2

```
Poziv: PredatNaVreme "Ana" 20 ==  
       PredatNaVreme "Ana" 15  
Izlaz: False
```

Primer 3

```
Poziv: PredatNaVreme "Marko" 15 >  
       PredatSaKasnjenjem "Ana" 30 1  
Izlaz: False
```

Primer 4

```
Poziv: PredatSaKasnjenjem "Mia" 18 1 >  
       PredatNaVreme "Zoe" 25  
Izlaz: True
```

Primer 5

```
Poziv: PredatNaVreme "Ana" 10 <  
       PredatSaKasnjenjem "Ana" 10 0  
Izlaz: True
```

Primer 6

```
Poziv: PredatSaKasnjenjem "Ana" 20 1 <  
       PredatSaKasnjenjem "Ana" 20 2  
Izlaz: True
```

2. Instancirati klasu `Show` za tip `SeminarskiRad`. Radovi se prikazuju u formatima `š: p poena` i `š: p poena (k dana kasnjenja)`".

Primer 7

```
Poziv: show (PredatNaVreme "Ana" 20)
Izlaz: "Ana: 20 poena"
```

Primer 8

```
Poziv: show (PredatNaVreme "Marko" 15)
Izlaz: "Marko: 15 poena"
```

Primer 9

```
Poziv: show (PredatSaKasnjenjem "Iva" 18 2)
Izlaz: "Iva: 18 poena (2 dana kasnjenja)"
```

Primer 10

```
Poziv: show (PredatSaKasnjenjem "Anastasija" 10 5)
Izlaz: "Anastasija: 10 poena (5 dana kasnjenja)"
```

3. `najslabijiSeminarskiRad` radovi, čiji je tip:
`najslabijiSeminarskiRad :: [SeminarskiRad] -> Maybe String`
 koja vraća ime studenta čiji seminarski rad ima najmanji efektivni broj poena. Efektivni broj poena za radove predate sa kašnjenjem dobija se oduzimanjem dva poena za svaki dan kašnjenja, pri čemu rezultat ne može biti manji od nule. Za praznu listu vratiti `Nothing`. Ako više studenata ima isti efektivni broj poena, vratiti leksikografski najmanje ime.

Primer 11

```
Poziv: najslabijiSeminarskiRad
      [PredatNaVreme "Ana" 20,
       PredatSaKasnjenjem "Marko" 25 3]
Izlaz: Just "Marko"
```

Primer 12

```
POZIV: najslabijiSeminarskiRad
      [PredatSaKasnjenjem "Ana" 14 2,
       PredatNaVreme "Marko" 10]
IZLAZ: Just "Ana"
```

Primer 13

```
POZIV: najslabijiSeminarskiRad
      [PredatNaVreme "Mia" 10,
       PredatNaVreme "Ana" 10]
IZLAZ: Just "Ana"
```

Primer 14

```
POZIV: najslabijiSeminarskiRad []
IZLAZ: Nothing
```

4. `sortirajRadovePoEfektivnimPoenima` radovi, čiji je tip:
`sortirajRadovePoEfektivnimPoenima :: [SeminarskiRad]`
`-> [SeminarskiRad]`
 koja sortira radove rastuće po efektivnom broju poena. Efektivni broj poena za radove predate sa kašnjenjem dobija se oduzimanjem dva poena za svaki dan kašnjenja, pri čemu rezultat ne može biti manji od nule. Ako dva rada imaju isti efektivni broj poena, koristiti podrazumevani poredak za tip `SeminarskiRad`.

Primer 15

```
POZIV: sortirajRadovePoEfektivnimPoenima
      [PredatNaVreme "Ana" 20,
       PredatSaKasnjenjem "Marko" 15 2,
       PredatSaKasnjenjem "Mia" 8 5]
IZLAZ: [PredatSaKasnjenjem "Mia" 8 5,
        PredatSaKasnjenjem "Marko" 15 2,
        PredatNaVreme "Ana" 20]
```

Primer 16

```
POZIV: sortirajRadovePoEfektivnimPoenima
      [PredatNaVreme "Ana" 10,
       PredatSaKasnjenjem "Marko" 14 2]
IZLAZ: [PredatNaVreme "Ana" 10,
        PredatSaKasnjenjem "Marko" 14 2]
```

Zadatak 3.4.37

Napisati biblioteku funkcija za obradu stanja robe u magacinu. Stanje magacina predstavlja se listom uređenih parova (**proizvod**, **kolicina**), tipa `[(String, Int)]`.

1. Implementirati funkciju `ukupnoArtikala` čiji je tip:

`ukupnoArtikala :: [(String, Int)] -> Int`

Funkcija prima kao argument **stanje** i vraća ukupan broj artikala čija je količina strogo veća od nule.

Primer 1

```
Poziv: ukupnoArtikala [("olovka",10),("sveska",5)]
Izlaz: 15
```

Primer 2

```
Poziv: ukupnoArtikala [("gumica",0),("lenjir",7),("blok",-3)]
Izlaz: 7
```

Primer 3

```
Poziv: ukupnoArtikala []
Izlaz: 0
```

2. Implementirati funkciju `azurirajKolicinu`, čiji je tip:

`azurirajKolicinu :: String -> Int -> [(String, Int)] -> [(String, Int)]`

Funkcija prima kao argumente **proizvod** **promena** **stanje** i uvećava količinu zadatog proizvoda za vrednost **promena**. Ako proizvod ne postoji u stanju **stanje**, dodati ga samo ako je **promena** strogo pozitivna. Posle izmene ukloniti proizvode čija količina nije strogo pozitivna. Rezultat treba da bude sortiran leksikografski po nazivu proizvoda.

Primer 4

```
Poziv: azurirajKolicinu "olovka" 3
      [("olovka",10),("sveska",5)]
Izlaz: [("olovka",13),("sveska",5)]
```

Primer 5

```
Poziv: azurirajKolicinu "gumica" 4
      [("olovka",10),("sveska",5)]
Izlaz: [("gumica",4),("olovka",10),("sveska",5)]
```

Primer 6

```
Poziv: azurirajKolicinu "sveska" (-5)
      [("olovka",10),("sveska",5)]
Izlaz: [("olovka",10)]
```

3. Implementirati funkciju `proizvodiNaStanju`, čiji je tip:
`proizvodiNaStanju :: [(String, Int)] -> [String]`
Funkcija prima kao argument `stanje` i vraća nazive proizvoda
čija je količina strogo veća od nule, sortirane leksikografski.

Primer 7

```
Poziv: proizvodiNaStanju
      [("olovka",10),("lenjir",0),("sveska",5)]
Izlaz: ["olovka","sveska"]
```

Primer 8

```
Poziv: proizvodiNaStanju
      [("rezac",1),("lenjir",2),("pernica",0)]
Izlaz: ["lenjir","rezac"]
```

Primer 9

```
Poziv: proizvodiNaStanju [("lenjir",0)]
Izlaz: []
```

4. Implementirati funkciju `nedostajuciProizvodi`, čiji je tip:
`nedostajuciProizvodi :: [String] -> [(String, Int)] -> [String]`
Funkcija prima kao argumente `trazeni stanje` i vraća
proizvode iz liste `trazeni` koji ne postoje u magacinu ili im
je količina jednaka nuli. Redosled rezultata treba da prati
redosled iz liste `trazeni`.

Primer 10

```
POZIV: nedostajuciProizvodi ["sveska","lenjir","gumica"]  
      [(("olovka",10),("lenjir",0),("sveska",5))]  
IZLAZ: ["lenjir","gumica"]
```

Primer 11

```
POZIV: nedostajuciProizvodi ["lenjir","gumica","sveska"]  
      [(("gumica",0),("lenjir",2))]  
IZLAZ: ["gumica","sveska"]
```

Primer 12

```
POZIV: nedostajuciProizvodi [] [(("gumica",1))]  
IZLAZ: []
```

5. Implementirati funkciju `najzastupljeniji`, čiji je tip:
`najzastupljeniji :: [(String, Int)] -> Maybe String`
Funkcija prima kao argument `stanje` i vraća proizvod sa najvećom količinom. Za praznu listu vratiti `Nothing`. Ako više proizvoda ima istu količinu, vratiti leksikografski najmanji naziv.

Primer 13

```
POZIV: najzastupljeniji  
      [(("olovka",10),("sveska",5),("gumica",10))]  
IZLAZ: Just "gumica"
```

Primer 14

```
POZIV: najzastupljeniji  
      [(("ranac",1),("gumica",2),("pernica",2))]  
IZLAZ: Just "gumica"
```

Primer 15

```
POZIV: najzastupljeniji []  
IZLAZ: Nothing
```

Zadatak 3.4.38

Napisati biblioteku funkcija za obradu stanja robe u magacinu.

1. Definirati tip `Artikal`, koji se može konstruisati pomoću konstruktora (redom) `NaStanju` ili `Rasprodat`. Konstruktor `NaStanju` prima naziv proizvoda (tipa `String`) i količinu na stanju (tipa `Int`), dok konstruktor `Rasprodat` prima samo naziv proizvoda (tipa `String`). Podrazumevano instancirati klase `Eq` i `Ord` za tip `Artikal` (artikli se porede prvo po konstruktoru u redosledu definicije konstruktora, dakle `NaStanju < Rasprodat`, a zatim leksikografski po parametrima konstruktora).

Primer 1

```
POZIV: NaStanju "olovka" 10 == NaStanju "olovka" 10
IZLAZ: True
```

Primer 2

```
POZIV: NaStanju "olovka" 10 < Rasprodat "olovka"
IZLAZ: True
```

Primer 3

```
POZIV: Rasprodat "blok" > NaStanju "sveska" 5
IZLAZ: True
```

2. Instancirati klasu `Show` za tip `Artikal` u sledećem formatu:

```
NaStanju s k  → s: k kom
Rasprodat s   → s: rasprodato
```

Primer 4

```
POZIV: show (NaStanju "olovka" 10)
IZLAZ: "olovka: 10 kom"
```

Primer 5

```
POZIV: show (Rasprodat "gumica")
IZLAZ: "gumica: rasprodato"
```

3. Implementirati funkciju `pronadjiArtikal`, čiji je tip:

```
pronadjiArtikal :: String -> [Artikal] -> Either  
String Int
```

koja za dati naziv proizvoda pretražuje listu artikala. Ako postoji vrednost oblika `NaStanju naziv kolicina`, vratiti `Right kolicina`. Ako postoji vrednost oblika `Rasprodat naziv`, vratiti `Left "Rasprodat"`. Ukoliko se proizvod ne nalazi u listi, vratiti `Left "Nepostojeci artikal"`.

Primer 6

```
POZIV: pronadjiArtikal "olovka" [NaStanju "olovka" 10,  
      Rasprodat "gumica"]  
IZLAZ: Right 10
```

Primer 7

```
POZIV: pronadjiArtikal "gumica" [NaStanju "olovka" 10,  
      Rasprodat "gumica"]  
IZLAZ: Left "Rasprodat"
```

Primer 8

```
POZIV: pronadjiArtikal "blok" [NaStanju "olovka" 10]  
IZLAZ: Left "Nepostojeci artikal"
```

4. Implementirati funkciju `sortirajPoKolicini`, čiji je tip:

```
sortirajPoKolicini :: [Artikal] -> [Artikal]
```

koja sortira artikle rastuće po količini. Smatrati da artikli konstruisani pomoću `Rasprodat` imaju količinu jednaku nuli. U slučaju jednake količine koristiti podrazumevani poredak nad tipom `Artikal`.

Primer 9

```
POZIV: sortirajPoKolicini [NaStanju "olovka" 10,  
      Rasprodat "gumica", NaStanju "sveska" 5]  
IZLAZ: [Rasprodat "gumica", NaStanju "sveska" 5,  
      NaStanju "olovka" 10]
```


Primer 10

```
POZIV: sortirajPoKolicini  
       [NaStanju "lenjir" 3, NaStanju "gumica" 3]  
IZLAZ: [NaStanju "gumica" 3, NaStanju "lenjir" 3]
```

Primer 11

```
POZIV: sortirajPoKolicini []  
IZLAZ: []
```